FÁBRICA DE SOFTWARE | DOCUMENTAÇÃO TÉCNICA

WAREHOUSEFLOW

SPRINTS 01–10 | RELEASE CANDIDATE

GRUPO 26 • TURMA 8MB

DOCUMENTAÇÃO ACUMULADA

Histórico integral das Sprints 01 a 09, seguido da Sprint 10 e avaliação da RC.

EQUIPE

Gabriel George de Araújo Figueiredo	01605236
Guilherme Branco Ferrario	01596391
João Pedro Silva de Araujo	01606470
Sérgio José de Araújo Júnior	01590694
Thiago de Morais Gonçalves	01609695

FÁBRICA DE SOFTWARE

DOCUMENTAÇÃO TÉCNICA WAREHOUSEFLOW

SPRINTS 01 · 09

Planejamento | Desenvolvimento | Testes | Consolidação

INFORMAÇÕES DO PROJETO

Projeto: WarehouseFlow — Sistema de Gestão de Armazém e Estoque
Grupo: 26
Turma: 8MB
Disciplina: Fábrica de Software
Entrega: Sprints 01 a 09

EQUIPE

Gabriel George de Araújo Figueiredo — 01605236
João Pedro Silva de Araujo — 01606470
Sérgio José de Araújo Júnior — 01590694
Thiago de Morais Gonçalves — 01609695
Guilherme Branco Ferrario — 01596391

FÁBRICA DE SOFTWARE

SPRINT 01 — PLANEJAMENTO DO PROJETO

WarehouseFlow

Nome inicial do projeto: WarehouseFlow — Sistema de Gestão de Armazém e Estoque.
Turma: 8MB

2. Escolha do tema

O projeto será desenvolvido no contexto de logística e gestão de armazéns, mais especificamente na área de controle de estoque (Warehouse Management). Esse contexto envolve o recebimento, armazenamento, movimentação, contagem e expedição de produtos dentro de um centro de distribuição ou depósito, sendo um domínio rico em processos, regras de negócio e oportunidades de automação.

3. Definição do problema

3.1 Qual problema a equipe pretende resolver?

Muitas pequenas e médias empresas ainda controlam seus estoques por meio de planilhas eletrônicas ou anotações manuais, o que gera divergências entre o estoque físico e o estoque registrado, dificuldade em localizar produtos dentro do armazém, falta de visibilidade sobre produtos com estoque baixo ou próximos da validade, e retrabalho na hora de gerar relatórios gerenciais. O WarehouseFlow pretende resolver esse problema oferecendo um sistema centralizado, que permita cadastrar produtos e localizações, registrar entradas, saídas e movimentações internas, e acompanhar o saldo de estoque em tempo real.

3.2 Por que esse problema é relevante?

O controle inadequado de estoque impacta diretamente o resultado financeiro das empresas, gerando perdas por produtos vencidos ou extraviados, rupturas de estoque que afetam vendas, e retrabalho operacional. Além disso, é um problema recorrente em pequenas e médias empresas, que muitas vezes não têm orçamento para adquirir sistemas de gestão robustos, tornando relevante o desenvolvimento de uma solução mais enxuta e acessível, focada especificamente na gestão de armazém.

4. Objetivos do sistema

4.1 Objetivo geral

Desenvolver um sistema para gestão de estoque e armazém que permita controlar, de forma centralizada e confiável, o cadastro de produtos, as movimentações de entrada, saída e transferência, e a geração de relatórios gerenciais que apoiem a tomada de decisão.

4.2 Principais resultados que a solução pretende alcançar

- Reduzir divergências entre o estoque físico e o estoque registrado no sistema.
- Aumentar a agilidade na localização de produtos dentro do armazém.
- Diminuir perdas por produtos vencidos ou parados em estoque.
- Fornecer informações gerenciais confiáveis para apoiar decisões de compra e venda.
- Padronizar os processos de recebimento, armazenagem e expedição de mercadorias.

5. Público-alvo

O sistema é voltado principalmente para pequenas e médias empresas que possuem um armazém, depósito ou centro de distribuição próprio, como comércios, indústrias de pequeno porte e distribuidoras. Dentro dessas empresas, os principais usuários do sistema serão:

- Operadores de estoque — responsáveis pelo recebimento, armazenagem, movimentação e expedição de produtos no dia a dia.
- Conferentes — responsáveis pela contagem física e conferência do inventário.
- Administradores/gestores — responsáveis por cadastros gerais, definição de permissões e acompanhamento de relatórios gerenciais.

6. Requisitos Funcionais

A tabela a seguir apresenta uma primeira relação das principais funcionalidades previstas para o sistema, que poderão ser detalhadas e ajustadas nas próximas Sprints.

Código	Descrição
RF01	Cadastrar, editar, consultar e inativar produtos (SKU, descrição, categoria, unidade de medida, código de barras).
RF02	Cadastrar e gerenciar localizações físicas do armazém (corredor, prateleira, nível, posição).
RF03	Registrar entradas de mercadorias (recebimento) vinculadas a fornecedor e nota/pedido de compra.
RF04	Registrar saídas de mercadorias (expedição) vinculadas a pedidos de venda ou requisições internas.
RF05	Movimentar estoque entre localizações dentro do armazém (transferência interna).
RF06	Controlar saldo de estoque em tempo real por produto e por localização.
RF07	Gerar alertas automáticos de estoque mínimo/ruptura e de produtos próximos da validade.
RF08	Realizar contagem cíclica/inventário rotativo e apontar divergências entre saldo físico e sistêmico.
RF09	Gerar relatórios gerenciais (giro de estoque, acurácia de inventário, produtos parados, curva ABC).
RF10	Emitir etiquetas e documentos de apoio (romaneio, etiqueta de localização, etiqueta de produto).
RF11	Controlar perfis de acesso (administrador, operador de estoque, conferente) com permissões distintas.
RF12	Registrar histórico (log) de todas as movimentações de estoque, com usuário e data/hora.

7. Requisitos Não Funcionais

Além das funcionalidades, o sistema deverá atender às seguintes características técnicas e de qualidade:

Código	Categoria	Descrição
RNF01	**Usabilidade**	A interface deve ser simples e objetiva, priorizando o uso rápido por operadores de armazém, inclusive em dispositivos móveis/coletores.
RNF02	**Desempenho**	Consultas de saldo de estoque devem responder em até 2 segundos, mesmo com grande volume de itens cadastrados.
RNF03	**Segurança**	Autenticação obrigatória por usuário e senha, com controle de perfis de acesso e criptografia de senhas.
RNF04	**Confiabilidade**	O sistema deve garantir a integridade do saldo de estoque mesmo em caso de falhas, evitando movimentações duplicadas.
RNF05	**Disponibilidade**	O sistema deve estar disponível para uso durante o horário de operação do armazém, com meta de disponibilidade de 99%.
RNF06	**Portabilidade**	A aplicação deve ser acessível via navegador web, sem necessidade de instalação local nas estações de trabalho.
RNF07	**Manutenibilidade**	O código deve seguir padrões de arquitetura em camadas, facilitando manutenção e evolução por diferentes desenvolvedores.
RNF08	**Escalabilidade**	O sistema deve suportar o crescimento do volume de produtos e movimentações sem perda relevante de desempenho.

8. Casos de Uso

A seguir são apresentados os principais casos de uso identificados nesta etapa inicial do planejamento.

Código	Caso de uso	Ator principal	Descrição resumida
UC01	Registrar entrada de mercadoria	Operador de estoque	Dar entrada de produtos recebidos de fornecedores, atualizando o saldo e a localização no armazém.
UC02	Registrar saída de mercadoria	Operador de estoque	Baixar produtos do estoque referentes a pedidos de venda ou requisições internas.
UC03	Movimentar produto entre localizações	Operador de estoque	Transferir um produto de uma posição para outra dentro do próprio armazém.
UC04	Consultar saldo de estoque	Operador de estoque / Administrador	Verificar a quantidade disponível de um produto, por localização ou de forma consolidada.
UC05	Realizar contagem de inventário	Conferente	Conferir fisicamente os produtos e registrar eventuais divergências com o sistema.
UC06	Cadastrar produto	Administrador	Incluir um novo produto no catálogo do sistema, com seus respectivos atributos.
UC07	Gerar relatórios gerenciais	Administrador	Emitir relatórios de desempenho do armazém, como giro de estoque e acurácia de inventário.
UC08	Gerenciar usuários e permissões	Administrador	Cadastrar usuários do sistema e definir seus perfis de acesso.

9. Product Backlog

O Product Backlog inicial organiza, em ordem de prioridade, os principais itens previstos para o desenvolvimento do sistema. Esta lista será refinada e detalhada em backlog de sprint (Sprint Backlog) ao longo do semestre.

#	Item do backlog	Prioridade	Requisito relacionado
1	Cadastro de produtos	Alta	RF01
2	Cadastro de localizações do armazém	Alta	RF02
3	Registro de entrada de mercadorias	Alta	RF03
4	Registro de saída de mercadorias	Alta	RF04
5	Controle de saldo de estoque em tempo real	Alta	RF06
6	Movimentação interna entre localizações	Média	RF05
7	Autenticação e controle de perfis de acesso	Alta	RF11
8	Alertas de estoque mínimo e validade	Média	RF07
9	Módulo de contagem/inventário rotativo	Média	RF08
10	Relatórios gerenciais (giro, curva ABC, acurácia)	Média	RF09
11	Emissão de etiquetas e romaneios	Baixa	RF10
12	Histórico (log) de movimentações	Média	RF12

10. Cronograma inicial

O cronograma abaixo apresenta uma organização preliminar das etapas previstas ao longo do semestre, que poderá ser ajustada conforme o andamento do projeto e as orientações recebidas.

Etapa	Principais atividades previstas
Sprint 01	Planejamento do projeto: tema, problema, objetivos, requisitos, backlog inicial e criação do repositório.
Sprint 02	Modelagem: diagramas de casos de uso detalhados, modelo de dados/ER e protótipo de telas (wireframes).
Sprint 03	Desenvolvimento do módulo de cadastros (produtos, localizações, usuários) e autenticação.
Sprint 04	Desenvolvimento das movimentações de estoque: entrada, saída e transferência interna.
Sprint 05	Desenvolvimento de alertas, contagem de inventário e ajustes de estoque.
Sprint 06	Desenvolvimento dos relatórios gerenciais e refinamento da interface.
Sprint 07	Testes integrados, correção de bugs e ajustes de desempenho.
Sprint 08	Entrega final, documentação do sistema e apresentação do projeto.

11. Repositório GitHub

O repositório da equipe já está criado e disponível no seguinte endereço:

https://github.com/thiagodemorais87/WarehouseFlow

Sumário

1. Introdução e visão da solução

2. Arquitetura do sistema

3. Diagrama de classes

4. Modelo Entidade-Relacionamento (MER)

5. Modelo relacional

6. Protótipo das telas principais

7. Banco de dados

8. Projeto estruturado no GitHub

9. Considerações finais

1. Introdução e visão da solução

O WarehouseFlow é um sistema proposto para apoiar o gerenciamento de operações de armazéns e estoques, centralizando informações de produtos, estoque, posições, pedidos e tarefas operacionais. O diferencial da solução é a incorporação de um motor de otimização capaz de analisar os dados disponíveis e produzir recomendações para tornar a execução das atividades mais eficiente.

A proposta busca reduzir deslocamentos desnecessários, melhorar a organização e a priorização das tarefas e auxiliar os operadores na tomada de decisão. Entre as funcionalidades previstas estão o cadastro de produtos, controle de estoque, gerenciamento de posições, registro de pedidos, controle de tarefas, sugestão de posições, sequenciamento de separação e otimização de deslocamentos.

1.1 Escopo funcional

Módulo	Responsabilidades principais
Produtos	Cadastro, consulta, atualização e exclusão de produtos.
Estoque	Controle de quantidade, entradas, saídas e localização dos produtos.
Armazém	Cadastro de armazéns e posições e controle de ocupação.
Pedidos	Cadastro, itens, status e prioridade dos pedidos.
Tarefas	Separação, movimentação, distribuição, priorização e controle de status.
Otimização	Sugestão de posição, priorização, sequenciamento e redução de deslocamentos.

2. Arquitetura do Sistema

A arquitetura adota uma separação lógica entre apresentação, aplicação, persistência e processamento computacional. Essa organização permite que as regras de negócio sejam centralizadas no back-end e que o motor de otimização seja integrado ao fluxo operacional sem depender diretamente da interface.

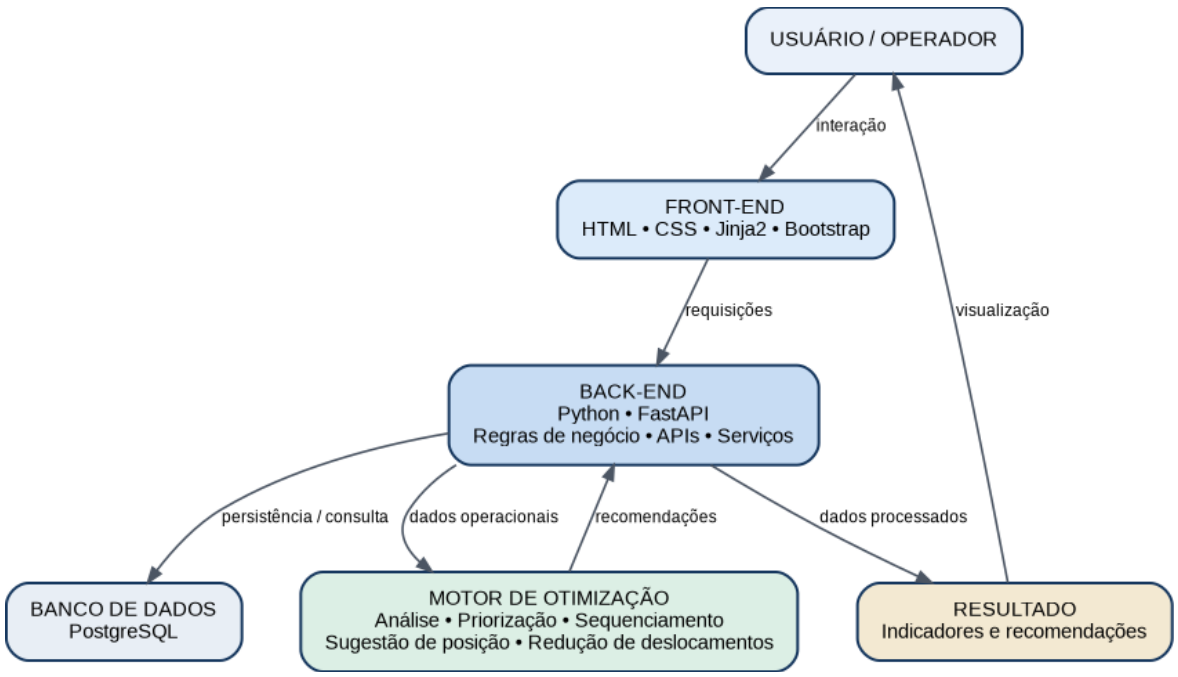


Figura 1 — Arquitetura lógica do WarehouseFlow.

2.1 Tecnologias

Camada	Tecnologias	Papel
Front-end	HTML, CSS, Jinja2, Bootstrap	Renderização das telas e interação com o usuário.
Back-end	Python, FastAPI, SQLAlchemy, Pydantic	APIs, regras de negócio, serviços, validação e acesso aos dados.
Dados	PostgreSQL	Persistência estruturada das informações do sistema.
Otimização	Motor de otimização	Análise dos dados operacionais e geração de recomendações.
Testes	Pytest	Automação de testes do código.
Infraestrutura	Docker, Docker Compose	Padronização e execução dos serviços.
Versionamento	Git, GitHub	Controle de versão e colaboração.

2.2 Integração com o componente computacional avançado

O motor de otimização recebe do back-end dados necessários para análise, como localização dos produtos, distâncias entre posições, prioridade dos pedidos, quantidade e disponibilidade de estoque, capacidade das posições e tarefas pendentes. A partir desses dados, o componente produz uma recomendação que retorna ao fluxo da aplicação e pode ser apresentada ao operador.

Fluxo lógico:

Pedidos pendentes → Produtos necessários → Localizações → Distâncias e prioridades → Algoritmo de otimização → Sequência/recomendação de tarefas

2.3 Requisitos não funcionais arquiteturais

- Separação de responsabilidades entre interface, aplicação, dados e otimização.
- Validação de dados na camada de aplicação.

- Persistência em banco relacional.
- Possibilidade de execução padronizada por containers.
- Versionamento e integração das alterações por Git/GitHub.

3. Diagrama de Classes

O diagrama de classes representa o domínio da solução e os principais objetos necessários para suportar os módulos descritos no projeto. As classes abaixo devem orientar a implementação do domínio e a persistência.

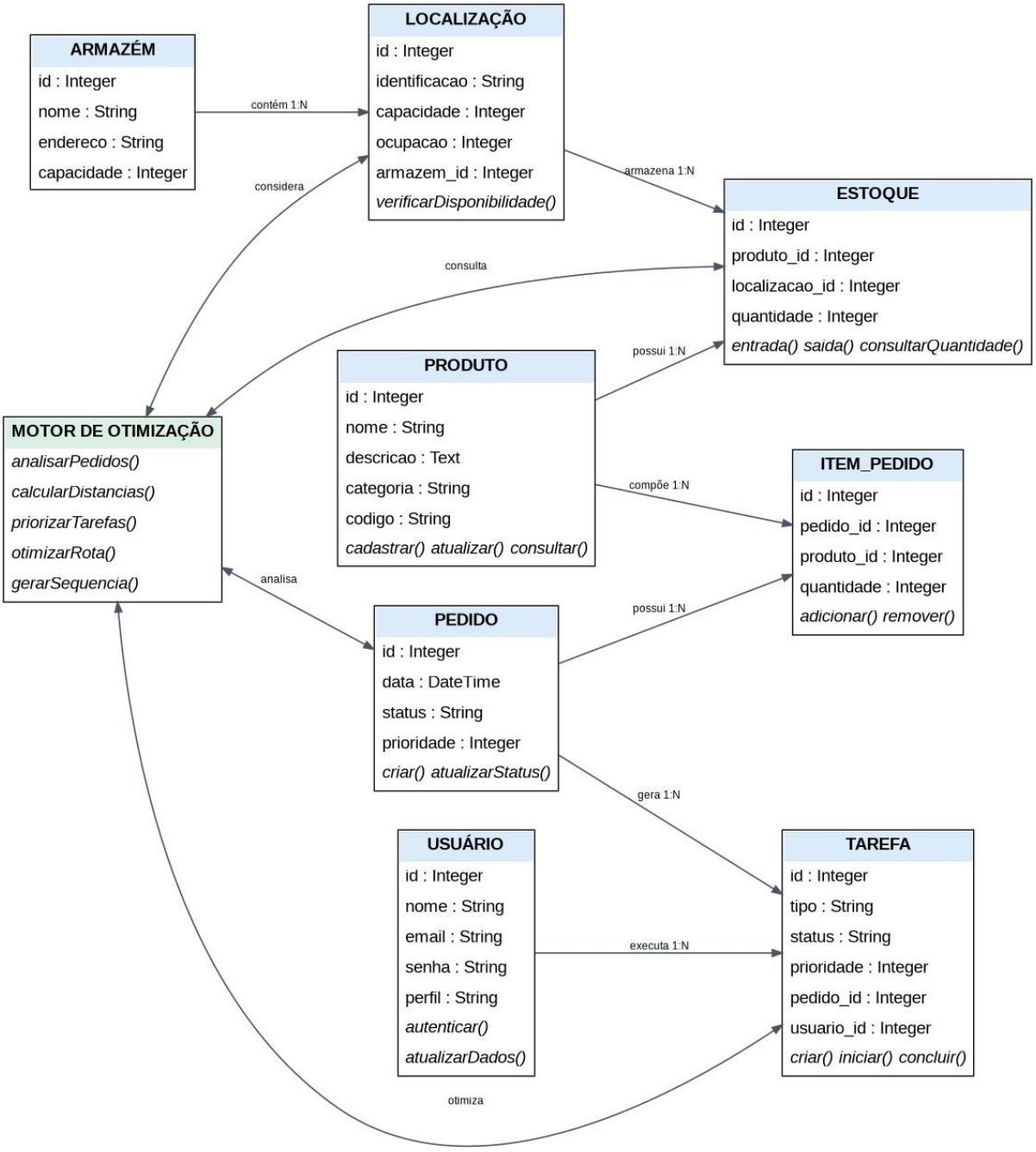


Figura 2 — Diagrama de classes conceitual do WarehouseFlow.

3.1 Responsabilidades das classes

Classe	Responsabilidade
Usuário	Representar o operador e seus dados de acesso/perfil.
Produto	Representar os itens administrados pelo armazém.
Armazém	Representar a unidade física e sua capacidade.
Localização	Representar uma posição física dentro do armazém.
Estoque	Relacionar produto, localização e quantidade disponível.
Pedido	Representar uma solicitação de produtos e sua prioridade/status.
ItemPedido	Representar cada produto e quantidade dentro de um pedido.
Tarefa	Representar uma atividade operacional derivada do fluxo de pedidos.
Motor de Otimização	Analisar dados operacionais e gerar recomendações para tarefas.

3.2 Relacionamentos principais

- Um armazém contém várias localizações (1:N).
- Uma localização pode possuir vários registros de estoque (1:N).
- Um produto pode possuir vários registros de estoque (1:N).
- Um pedido possui vários itens de pedido (1:N).
- Um produto pode aparecer em vários itens de pedido (1:N).
- Um pedido pode gerar várias tarefas (1:N).
- Um usuário pode executar várias tarefas (1:N).

4. Modelo Entidade-Relacionamento (MER)

O MER apresenta as entidades persistentes e os relacionamentos necessários para representar o domínio do WarehouseFlow. O modelo privilegia a separação entre cadastro, localização física, saldo de estoque, pedidos e execução operacional.

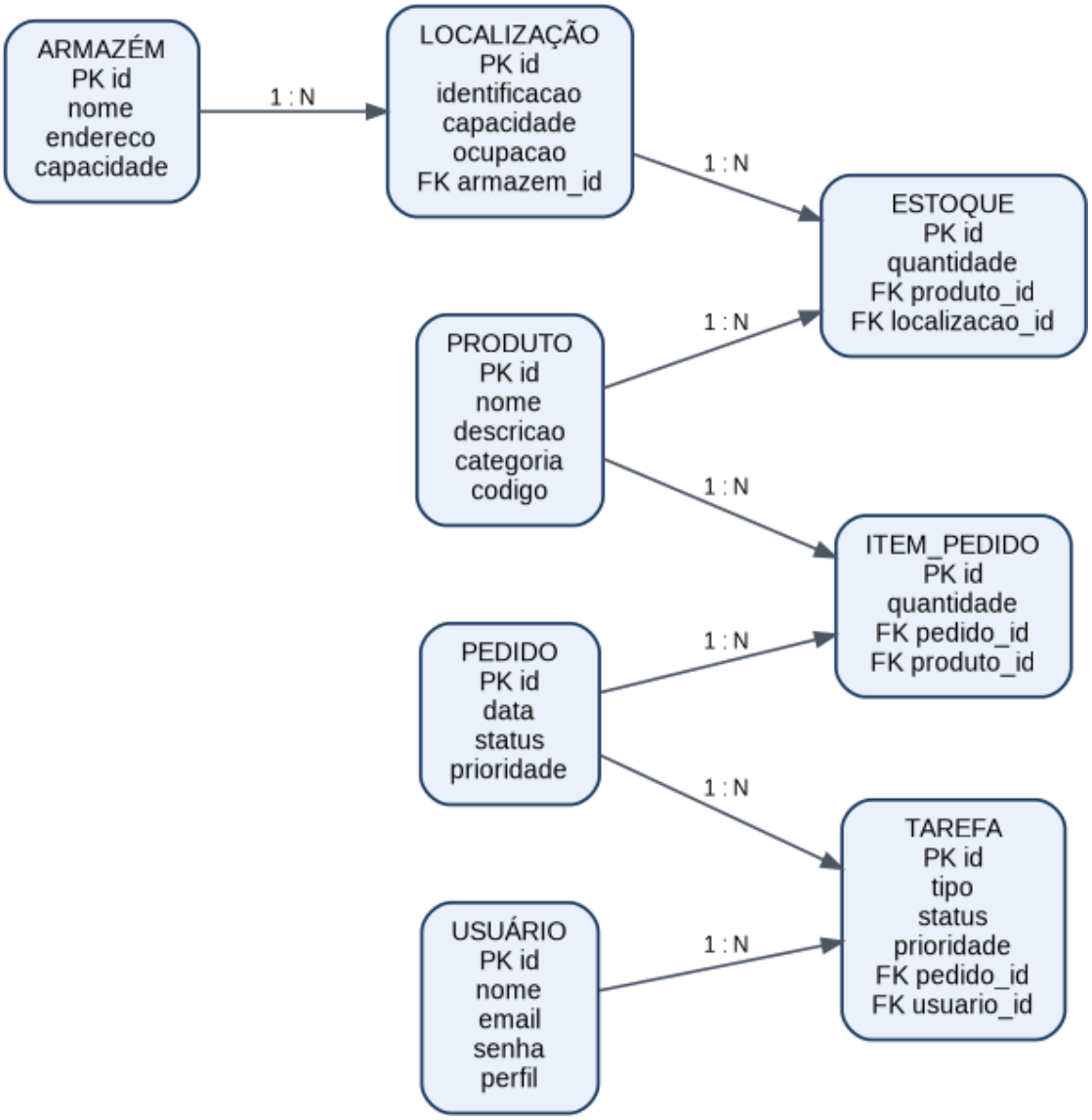


Figura 3 — Modelo Entidade-Relacionamento conceitual.

4.1 Entidades e atributos

Entidade	Atributos principais
USUÁRIO	id, nome, email, senha, perfil
ARMAZÉM	id, nome, endereço, capacidade
LOCALIZAÇÃO	id, identificação, capacidade, ocupação, armazém
PRODUTO	id, nome, descrição, categoria, código
ESTOQUE	id, produto, localização, quantidade
PEDIDO	id, data, status, prioridade
ITEM_PEDIDO	id, pedido, produto, quantidade
TAREFA	id, tipo, status, prioridade, pedido, usuário

5. Modelo Relacional

A transformação do modelo conceitual para o modelo relacional resulta nas tabelas abaixo. As chaves primárias identificam unicamente os registros, enquanto as chaves estrangeiras preservam os vínculos entre as entidades.

USUARIO

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
nome	VARCHAR	
email	VARCHAR	UNIQUE
senha	VARCHAR	
perfil	VARCHAR	

ARMAZEM

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
nome	VARCHAR	
endereco	VARCHAR	
capacidade	INTEGER	

LOCALIZACAO

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
identificacao	VARCHAR	
capacidade	INTEGER	
ocupacao	INTEGER	
armazem_id	INTEGER	FK → ARMAZEM.id

PRODUTO

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
nome	VARCHAR	
descricao	TEXT	
categoria	VARCHAR	
codigo	VARCHAR	UNIQUE

ESTOQUE

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
produto_id	INTEGER	FK → PRODUTO.id
localizacao_id	INTEGER	FK → LOCALIZACAO.id
quantidade	INTEGER	

PEDIDO

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
data	DATETIME	
status	VARCHAR	
prioridade	INTEGER	

ITEM_PEDIDO

Atributo	Tipo	Chave / restrição

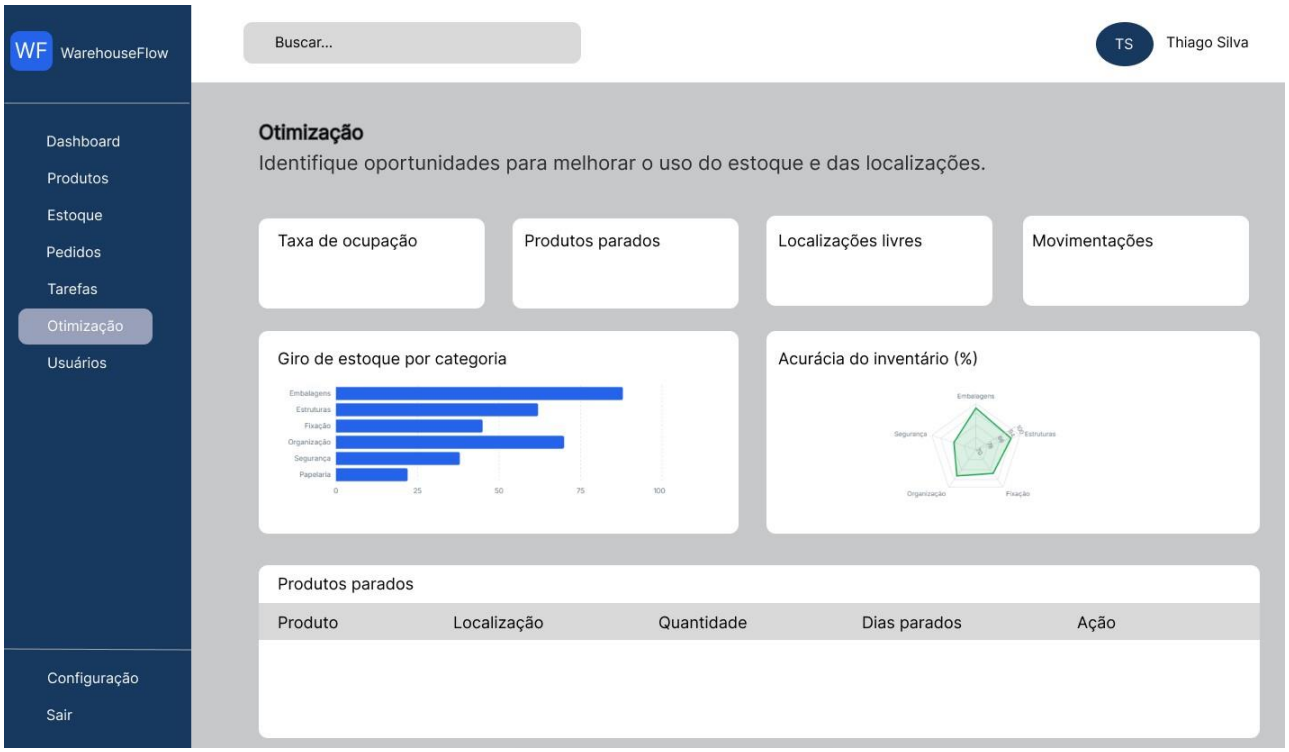
id	INTEGER	PK
pedido_id	INTEGER	FK → PEDIDO.id
produto_id	INTEGER	FK → PRODUTO.id
quantidade	INTEGER	

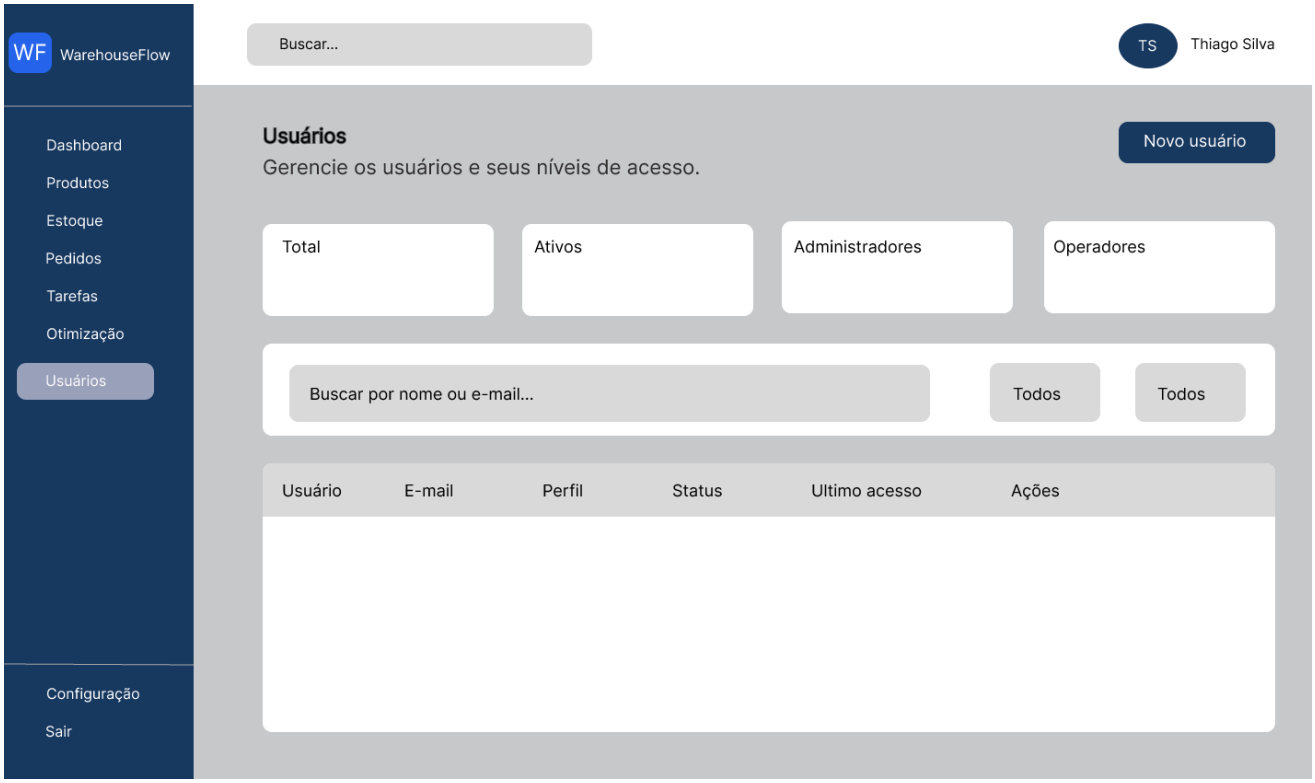
TAREFA

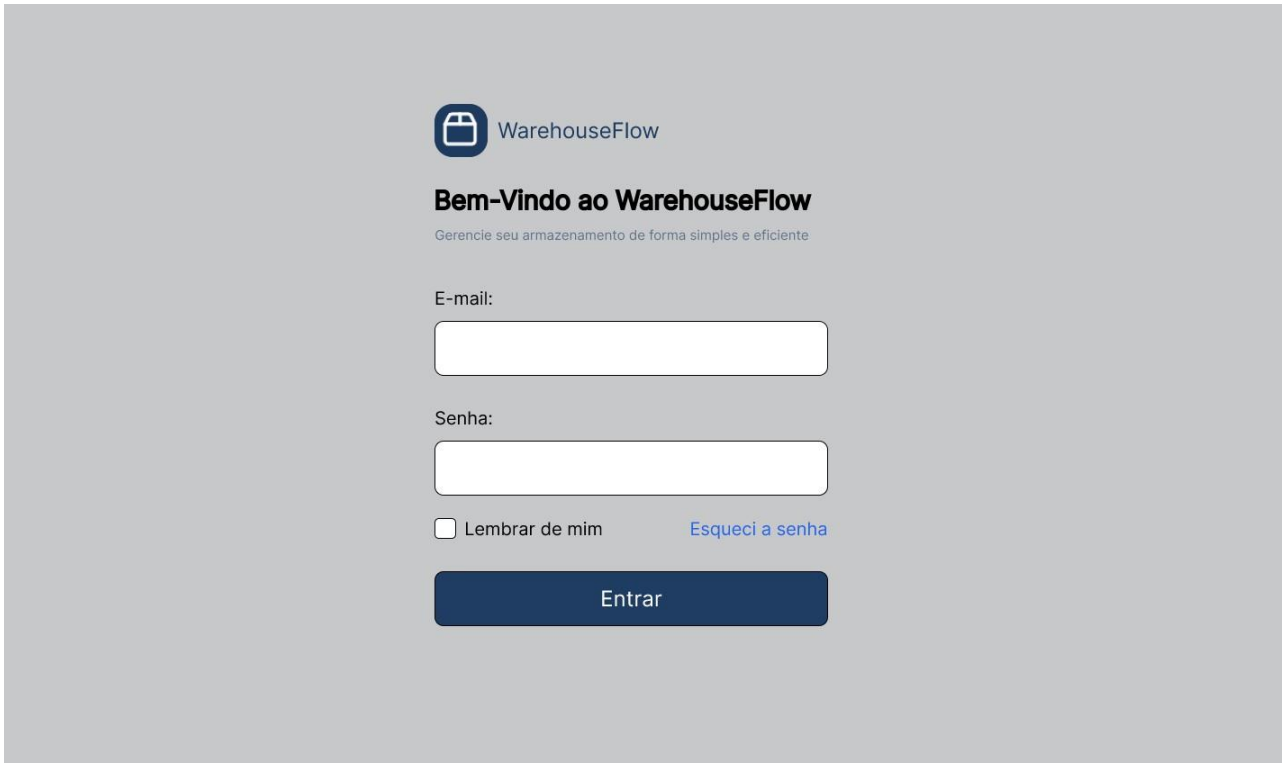
Atributo	Tipo	Chave / restrição
id	INTEGER	PK
tipo	VARCHAR	
status	VARCHAR	
prioridade	INTEGER	
pedido_id	INTEGER	FK → PEDIDO.id
usuario_id	INTEGER	FK → USUARIO.id

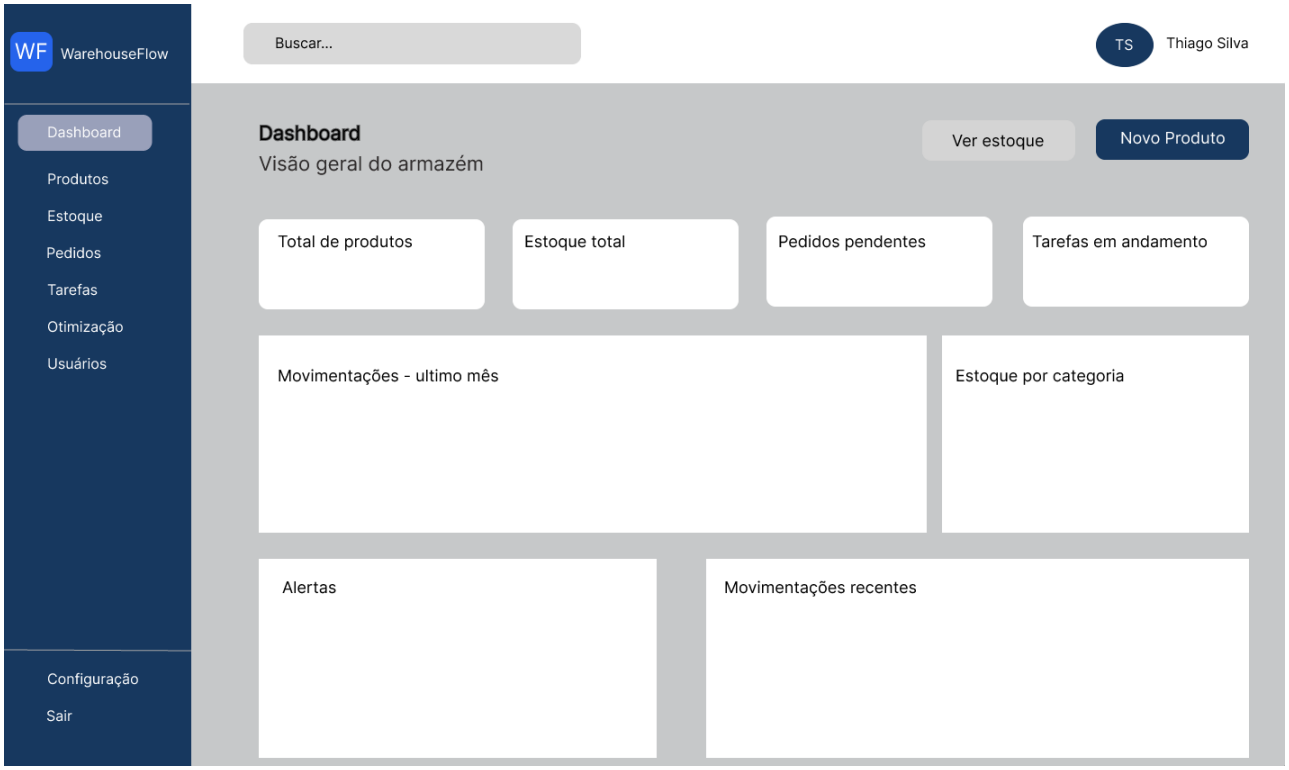
6. Protótipo das Telas Principais

O protótipo visual está no Figma e representa a navegação por Dashboard, Produtos, Estoque, Pedidos, Tarefas, Otimização, Usuários, Configuração e Sair. Para esta versão, foram incorporados ao documento os prints disponibilizados pela equipe, mantendo o link do protótipo completo como referência.

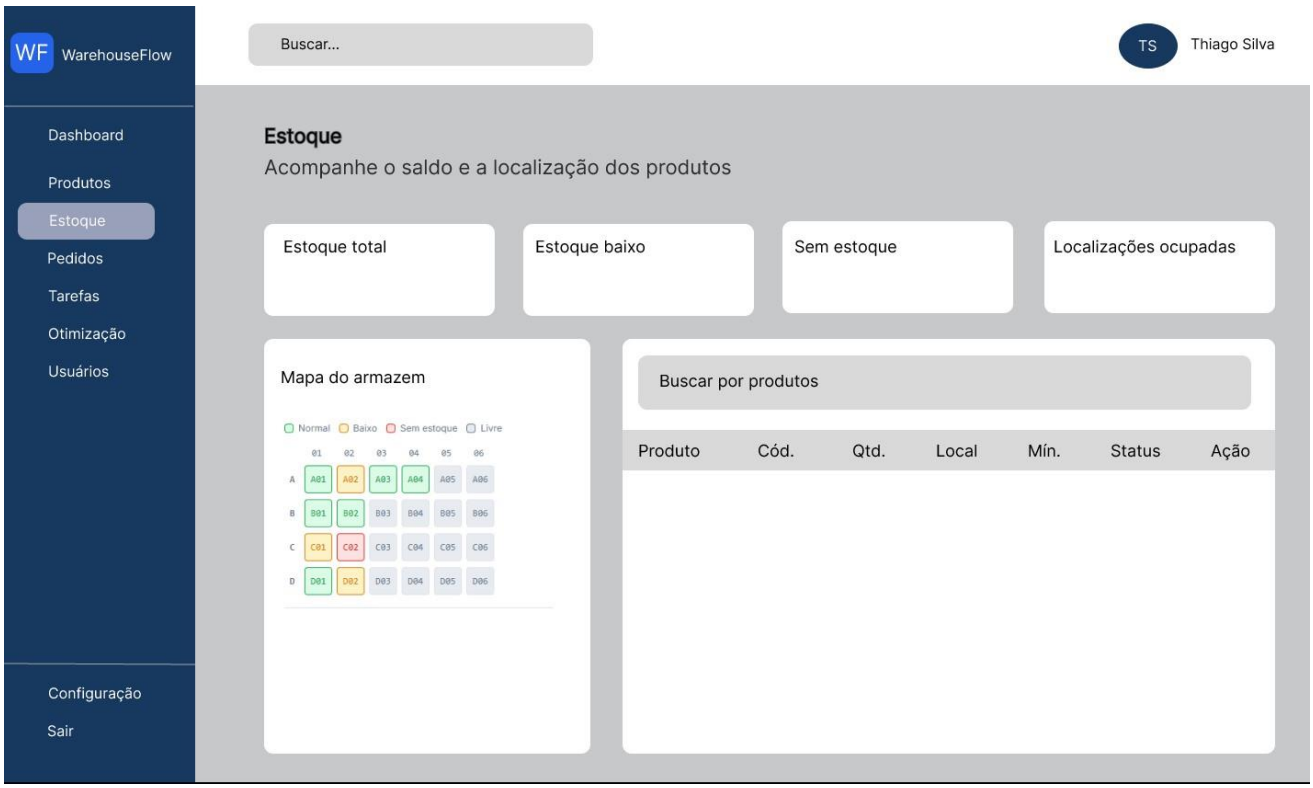



WF
WarehouseFlow
Buscar...
TS
Thiago Silva
Dashboard
Produtos
Estoque
Pedidos
Tarefas
Otimização
Usuários
Configuração
Sair
Usuários
Gerencie os usuários e seus níveis de acesso.
Novo usuário
Total
Ativos
Administradores
Operadores
Buscar por nome ou e-mail...
Todos
Todos
Usuário
E-mail
Perfil
Status
Ultimo acesso
Ações


WarehouseFlow
Bem-Vindo ao WarehouseFlow
Gerencie seu armazenamento de forma simples e eficiente
E-mail:
Senha:
Lembrar de mim
Esqueci a senha
Entrar


WF
WarehouseFlow
Buscar...
TS
Thiago Silva
Dashboard
Produtos
Estoque
Pedidos
Tarefas
Otimização
Usuários
Configuração
Sair
Dashboard
Visão geral do armazém
Ver estoque
Novo Produto
Total de produtos
Estoque total
Pedidos pendentes
Tarefas em andamento
Movimentações - ultimo mês
Estoque por categoria
Alertas
Movimentações recentes


WF
WarehouseFlow
Buscar...
TS
Thiago Silva
Dashboard
Produtos
Estoque
Pedidos
Tarefas
Otimização
Usuários
Configuração
Sair
Pedidos
Acompanhe recebimentos, expedições e pedidos internos.
Novo Pedido
todos
Entradas
Saídas
Pendentes
Concluídos
Buscar por número ou fornecedor/cliente...
pedido
Tipo
Fornecedor/cliente
itens
data
Status
ações


WF
WarehouseFlow
Buscar...
TS
Thiago Silva
Dashboard
Produtos
Estoque
Pedidos
Tarefas
Otimização
Usuários
Configuração
Sair
Estoque
Acompanhe o saldo e a localização dos produtos
Estoque total
Estoque baixo
Sem estoque
Localizações ocupadas
Mapa do armazem
Normal
Baixo
Sem estoque
Livre
Buscar por produtos
Produto
Cód.
Qtd.
Local
Mín.
Status
Ação



Link: https://www.figma.com/design/uGceagehdqafcFckYSlWZY/Sem-t%C3%ADtulo?node-id=5-569Ct=kWlwVzC0cuJRlaCp-1

7. Banco de Dados criado

O banco de dados desta Sprint 02 não é apenas uma proposta conceitual. O repositório atual contém o schema e o seed utilizados para inicialização do PostgreSQL, além do Docker Compose que monta esses arquivos no container do banco

7.1 Artefatos do banco

backend/database/schema.sql	**DDL das 10 tabelas, chaves primárias/estrangeiras, restrições e tabela optimization_results.**
backend/database/seed.sql	**Dados iniciais para roles, usuários, armazém, localizações, produtos, estoque, pedidos, tarefas e resultado de otimização.**
docker-compose.yml	**Serviço PostgreSQL 16, volume persistente e montagem automática de schema.sql + seed.sql.**
SQLAlchemy / DATABASE_URL	**Integração do back-end Python com PostgreSQL.**

7.2 Evidência verificável no GitHub

Na versão atual do repositório, a existência do banco é verificável diretamente pelos arquivos publicados. O schema.sql define roles, users, products, warehouses, locations, stock, orders, order_items, tasks e optimization_results. O seed.sql insere dados iniciais nessas estruturas. O docker-compose.yml configura o PostgreSQL e monta os dois scripts para inicialização

Trecho representativo do schema.sql

```
CREATE TABLE roles (
id SERIAL PRIMARY KEY,
name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE users (
id SERIAL PRIMARY KEY,
name VARCHAR(100) NOT NULL,
email VARCHAR(100) UNIQUE NOT NULL,
password_hash VARCHAR(255) NOT NULL,
role_id INTEGER REFERENCES roles(id) ON DELETE SET NULL
);

CREATE TABLE optimization_results (
id SERIAL PRIMARY KEY,
task_id INTEGER REFERENCES tasks(id) ON DELETE CASCADE,
suggested_location_id INTEGER REFERENCES locations(id),
suggested_route JSONB,
score DECIMAL(10,4)
);
```

Links de conferência:
https://github.com/thiagodemorais87/WarehouseFlow/blob/main/backend/database/schema.sql •
https://github.com/thiagodemorais87/WarehouseFlow/blob/main/backend/database/seed.sql •
https://github.com/thiagodemorais87/WarehouseFlow/blob/main/docker-compose.yml

8. Projeto Estruturado no GitHub

Repositório oficial: https://github.com/thiagodemorais87/WarehouseFlow

O repositório atual apresenta uma estrutura concreta, não apenas planejada. A raiz contém backend, database, docs, frontend, tests, Dockerfile, docker-compose.yml, requirements.txt, pytest.ini e README.md. Dentro de backend/database estão schema.sql e seed.sql; dentro de backend/optimization está o motor próprio de otimização.

Estrutura	Função
backend/app/	FastAPI, routers CRUD, schemas e serviços.
backend/database/	schema.sql e seed.sql.
backend/optimization/	Motor Python de otimização (Nearest Neighbor + 2-opt).
frontend/	Templates Jinja e arquivos estáticos.
docs/	Proposta, cronograma e documentação técnica do motor.
tests/	Testes automatizados.
docker-compose.yml	PostgreSQL + API em ambiente local.
pytest.ini / requirements.txt	Configuração e dependências dos testes.

8.1 Estratégia de versionamento

A implementação própria está em backend/optimization/, sem uso de OR-Tools ou outro solver externo. O pipeline utiliza Nearest Neighbor para construir uma rota gulosa e, em seguida, 2-opt para melhoria local. A distância é calculada pela métrica Manhattan.

Métrica	Significado
distance_before	Distância da rota original.
nearest_neighbor_distance	Distância após o Nearest Neighbor.
two_opt_distance / distance_after	Distância após o 2-opt.
distance_reduction	Ganho absoluto de distância: before − after.
reduction_percent	Ganho percentual em relação à rota original.
execution_time_ms	Tempo de execução do motor em milissegundos.

9. Considerações Finais

A revisão desta documentação corrige os pontos de coerência identificados na versão anterior. A identificação da equipe está corrigida, com Guilherme Branco Ferrario na matrícula 01596391; o modelo relacional foi alinhado ao schema atual; a seção de banco de dados passa a registrar evidências concretas do repositório; e o diferencial computacional deixou de ser descrito de forma genérica. A entrega também explicita o motor próprio de rota — Nearest Neighbor + 2-opt —, suas métricas distance_before, reduction_percent e execution_time_ms, além do endpoint POST /optimization/route. Dessa forma, arquitetura, banco, modelo relacional, GitHub e componente computacional são apresentados de maneira consistente com os artefatos atualmente publicados.

Referências

• GitHub — https://github.com/thiagodemorais87/WarehouseFlow • Schema — https://github.com/thiagodemorais87/WarehouseFlow/blob/main/backend/database/schema.sql • Seed — https://github.com/thiagodemorais87/WarehouseFlow/blob/main/backend/database/seed.sql • Docker Compose — https://github.com/thiagodemorais87/WarehouseFlow/blob/main/docker-compose.yml • Motor — https://github.com/thiagodemorais87/WarehouseFlow/blob/main/backend/optimization/README.md • Protótipo — https://www.figma.com/design/uGceagehdqafcFckYSlWZY/Sem-título?node-id=5-569

1. Identificação e objetivo da Sprint 03

A Sprint 03 representa a etapa de desenvolvimento do WarehouseFlow voltada à implementação das funcionalidades de autenticação, cadastro e controle de perfis, além da integração da aplicação com o banco de dados e da disponibilização do CRUD principal. Nesta etapa, a estrutura planejada nas Sprints anteriores passa a ser demonstrada por meio de funcionalidades executáveis e evidências de uso.

1.1 Escopo desta entrega

- Autenticação de usuários por meio de login.
- Cadastro de usuários e gerenciamento de perfis de acesso.
- Cadastro e manutenção dos registros principais do sistema, com operações de CRUD.
- Persistência dos dados em PostgreSQL.
- Execução local do sistema por meio da estrutura definida no projeto.
- Organização e atualização do código no repositório GitHub.

1.2 Relação com o planejamento anterior

Nas Sprints 01 e 02 foram definidos o problema, objetivos, requisitos, arquitetura, modelos de dados, protótipo e organização do projeto. A Sprint 03 dá continuidade a esse planejamento, materializando principalmente os requisitos relacionados a cadastro de produtos, autenticação e controle de perfis.

2. Breve descrição da estrutura implementada

O WarehouseFlow está organizado em módulos que separam a aplicação, a persistência, a interface, os testes e os artefatos de infraestrutura. A raiz do projeto contém os diretórios backend, database, docs, frontend e tests, além dos arquivos de configuração e execução.

- backend/ — APIs, regras de negócio, persistência e serviços.
- backend/database/ — schema e dados iniciais do PostgreSQL.
- backend/optimization/ — motor próprio de otimização.
- frontend/ — interface da aplicação.
- tests/ — testes automatizados.
- docs/ — documentação e artefatos.
- docker-compose.yml — orquestração dos serviços.
- requirements.txt — dependências Python.
- pytest.ini — configuração dos testes.

A implementação mantém a arquitetura apresentada nas Sprints anteriores, com Python/FastAPI no back-end, PostgreSQL na persistência e interface web baseada em HTML/CSS/Jinja2/Bootstrap.

3. Evidências do banco de dados conectado

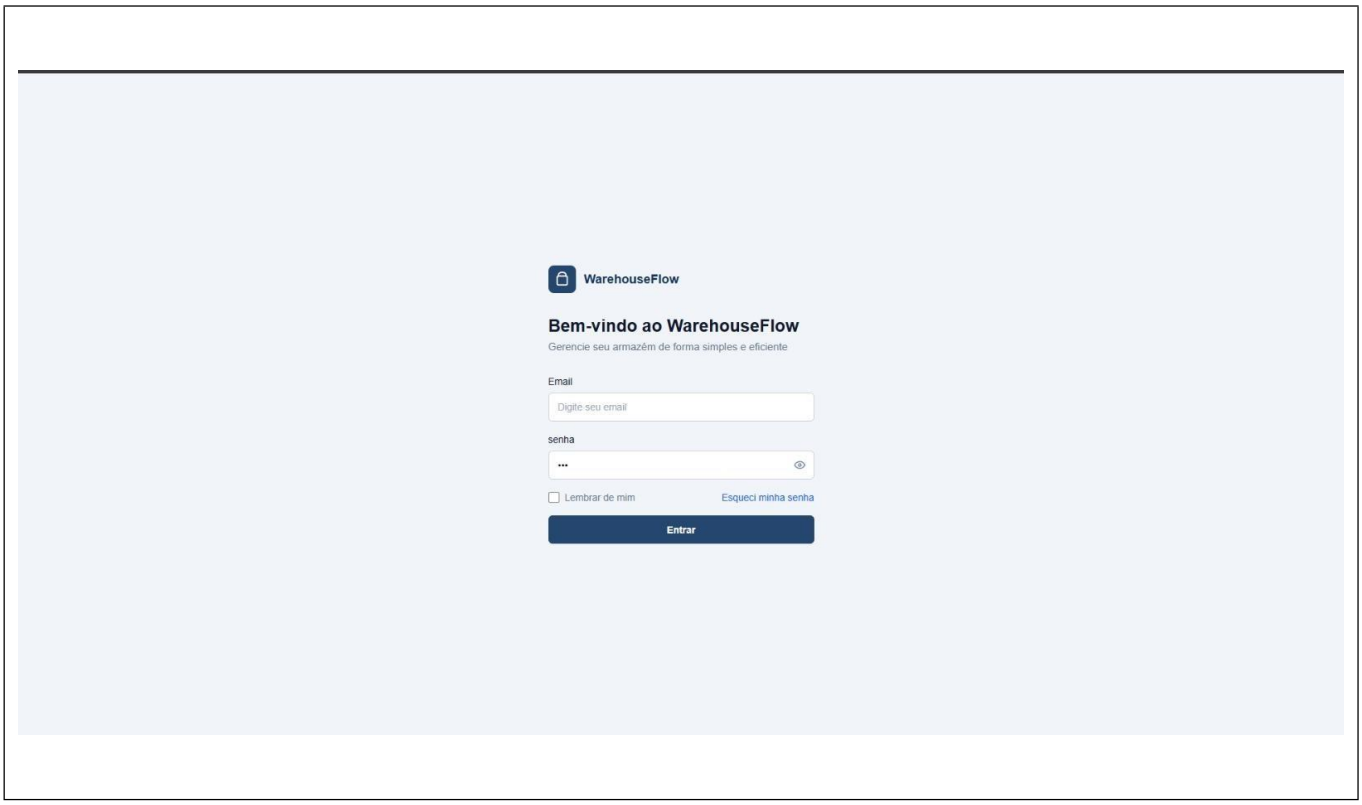
O sistema utiliza PostgreSQL como banco de dados relacional. A estrutura do projeto contém os artefatos necessários para criação e carga inicial do banco, e a aplicação utiliza a conexão configurada para persistir e consultar os dados dos módulos desenvolvidos.

.

A Sprint 03 contempla a autenticação dos usuários e o controle de acesso por perfil. O fluxo de autenticação identifica o usuário antes do acesso às funcionalidades protegidas, enquanto o cadastro e os perfis organizam os diferentes níveis de acesso previstos para o sistema.

4.1 Login

A tela de login é a porta de entrada do sistema. O usuário informa suas credenciais e, quando válidas, é direcionado para a aplicação. A autenticação deve impedir o acesso às áreas protegidas por usuários não autenticados.



4.2 Cadastro de usuário

O cadastro permite registrar novos usuários para utilização do sistema. A implementação deve validar os dados informados e armazená-los de forma persistente, respeitando as regras de acesso definidas.

4.3 Controle de perfis

O controle de perfis associa usuários a funções/permissões do sistema. Essa separação permite que funcionalidades administrativas sejam disponibilizadas de acordo com o perfil do usuário autenticado.

5. Evidências do CRUD principal funcionando

O CRUD (Create, Read, Update e Delete) representa as operações de criação, consulta, alteração e exclusão de registros. Nesta Sprint, o CRUD principal deve ser demonstrado preferencialmente no módulo de Produtos, por ser uma das funcionalidades centrais do WarehouseFlow.

Create — Criar

Registro de um novo produto no sistema.

Read — Consultar

Visualização dos produtos cadastrados e seus dados.

Update — Atualizar
Alteração de informações de um produto existente.

Delete — Excluir
Remoção de um produto conforme as regras de negócio

.6. Procedimento de execução local

O projeto pode ser executado localmente a partir da pasta raiz WarehouseFlow. Como a configuração do projeto utiliza Docker Compose, o procedimento recomendado é utilizar os arquivos de infraestrutura versionados no repositório. Caso a equipe utilize execução manual do Python, devem ser respeitadas as dependências e configurações definidas no projeto.

6.1 Pré-requisitos
- Python 3 instalado, quando utilizada a execução manual do back-end.
- Docker e Docker Compose instalados, quando utilizada a execução por containers.
- Git, caso seja necessário atualizar o código a partir do GitHub.

6.2 Execução por Docker Compose
Na pasta raiz do projeto, executar o Docker Compose conforme as instruções atuais do README. A execução deve iniciar os serviços necessários e disponibilizar a aplicação para acesso pelo navegador.

Comando de referência: docker compose up --build

Após a inicialização, acessar o endereço local informado pelo projeto/terminal e verificar o login e as funcionalidades desenvolvidas.



7. Link do repositório GitHub atualizado
Repositório oficial: https://github.com/thiagodemorais87/WarehouseFlow

8. Dificuldades encontradas
Durante a Sprint 03, as principais dificuldades estão relacionadas à integração entre as diferentes camadas da aplicação, configuração do ambiente local, conexão com o PostgreSQL, autenticação e organização dos perfis de acesso

9. Próximos passos
- Ampliar a cobertura de testes automatizados e testes integrados.
- Consolidar as movimentações de estoque e suas regras de negócio.
- Implementar/refinar alertas, inventário e ajustes de estoque conforme o backlog.
- Aprimorar relatórios gerenciais e indicadores operacionais.

- Realizar correções de bugs e ajustes de desempenho.
- Manter documentação, código e modelo de dados alinhados ao estado efetivamente implementado.

10. Considerações finais

A Sprint 03 representa a transição da etapa de modelagem para a comprovação prática das funcionalidades do WarehouseFlow. A implementação de autenticação, cadastro, controle de perfis, persistência e CRUD estabelece a base operacional necessária para as próximas etapas do projeto.

Antes da exportação final para PDF, os espaços reservados devem ser substituídos pelas evidências reais da execução. Recomenda-se que cada print seja acompanhado de uma legenda objetiva e mostre claramente a funcionalidade descrita na seção correspondente.

11. Referências

GitHub — WarehouseFlow: https://github.com/thiagodemorais87/WarehouseFlow

Figma — WarehouseFlow: https://www.figma.com/design/uGceagehdqafcFckYSlWZY/Sem-título?node-id=5-569

Documentação das Sprints 01 e 02 — material produzido pela equipe.

1. Objetivo do Sprint 04

O Sprint 04 tem como foco a consolidação do Módulo 1 — Produtos e Estoque (com posições), contemplando as operações de produtos, estoque e localizações/posições, além da validação da API por meio de cenários de teste.

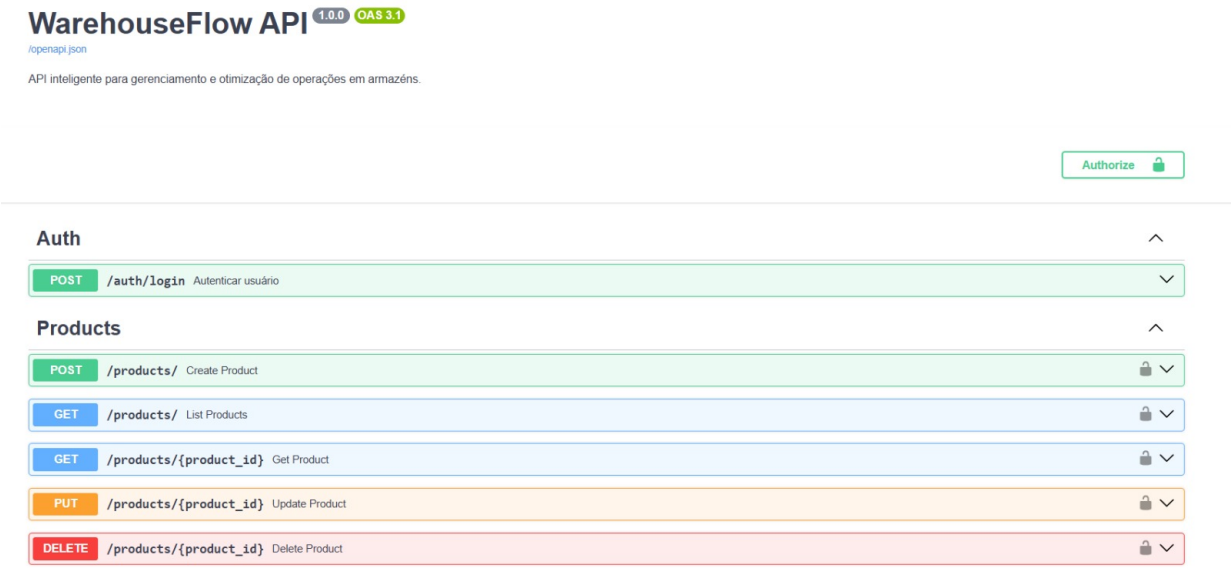
2. Escopo

- Cadastro, consulta, edição e exclusão de produtos.
- Criação/ajuste de estoque associado a produto e localização.
- Cadastro e listagem de localizações/posições.
- Validações de entrada, incluindo quantidade de estoque não negativa.
- Execução dos cenários pelo Postman contra a API local.

3. Integração e execução

Os cenários foram executados utilizando o Postman contra a API local do WarehouseFlow. As evidências apresentam os códigos HTTP e os corpos JSON retornados pela aplicação. A API conta ainda com documentação interativa (Swagger/OpenAPI), utilizada como referência para a execução dos testes.

API → Produtos → Estoque → Localizações/Posições



Evidência 01 — Documentação interativa da API (Swagger/OpenAPI) do WarehouseFlow, com os endpoints de autenticação (/auth/login) e de produtos (/products/).

4. Operações evidenciadas

Funcionalidade	Operação	Resultado
Produtos	POST /products/	201 Created
Produtos	PUT /products/{product_id}	200 OK
Produtos	DELETE /products/{product_id}	204 No Content

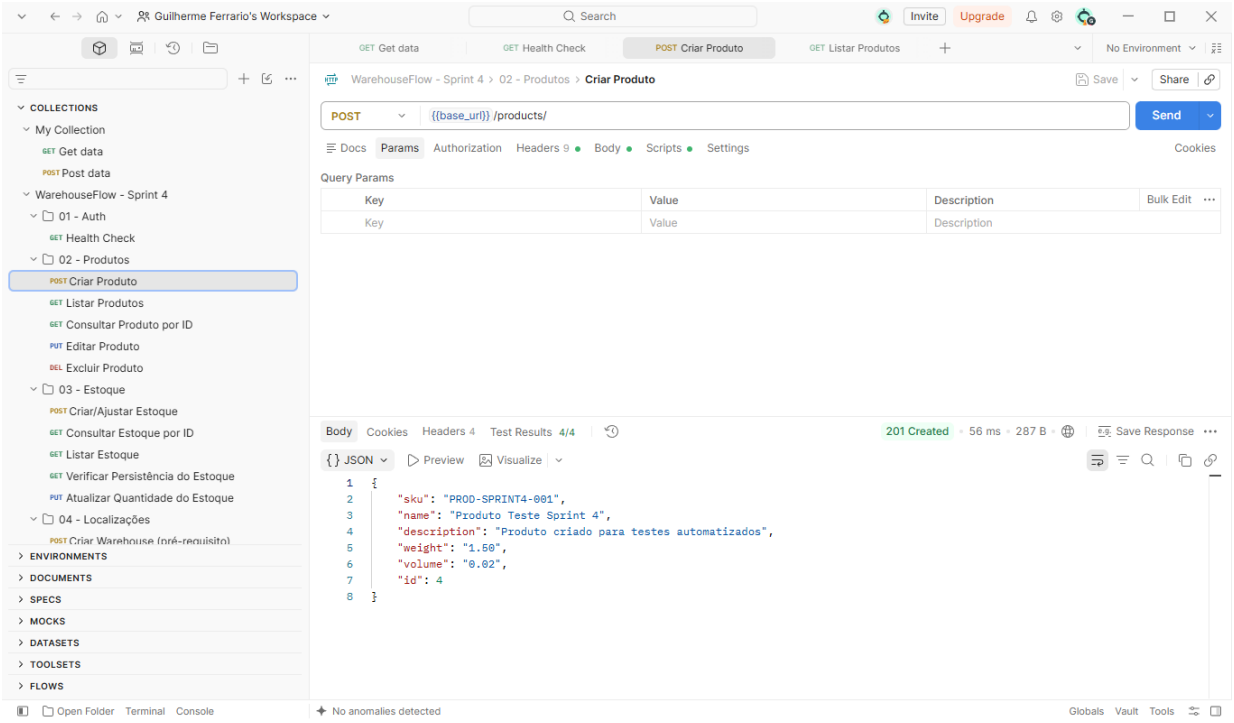
Funcionalidade	Operação	Resultado
Estoque	POST /stock/	201 Created
Localizações	POST /locations/	201 Created
Localizações	GET /locations/	200 OK
Validação	POST /stock/ com quantity = -10	422 Unprocessable Entity

5. Cenários de teste

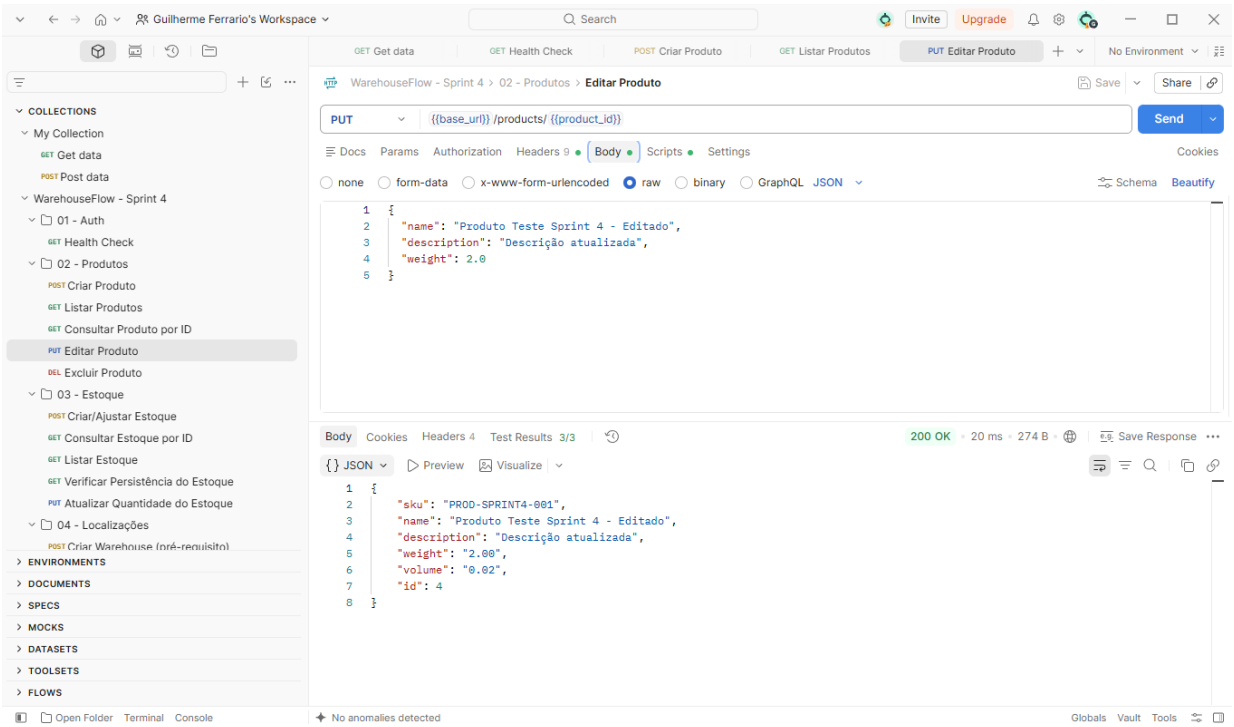
ID	Cenário	Resultado esperado	Resultado observado
CT01	Cadastrar produto válido	Produto criado.	201 Created — Aprovado
CT02	Editar produto	Dados atualizados.	200 OK — Aprovado
CT03	Excluir produto	Produto removido.	204 No Content — Aprovado
CT04	Criar/ajustar estoque	Registro criado com produto, localização e quantidade.	201 Created — Aprovado
CT05	Criar localização	Localização vinculada ao armazém criada.	201 Created — Aprovado
CT06	Informar estoque negativo	Operação rejeitada pela validação.	422 Unprocessable Entity — Aprovado
CT07	Listar localizações	Localizações retornadas pela API.	200 OK — Aprovado

6. Evidências dos testes

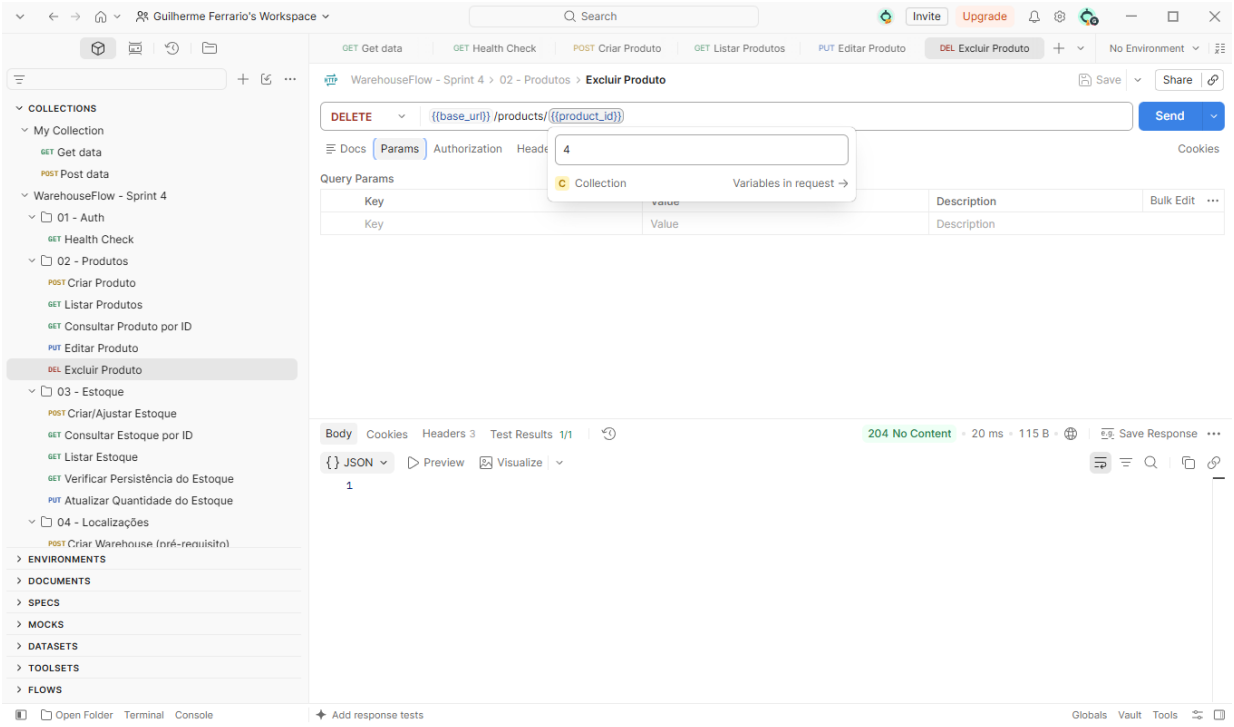
As imagens abaixo foram fornecidas após a execução dos testes no Postman e registram os resultados observados.



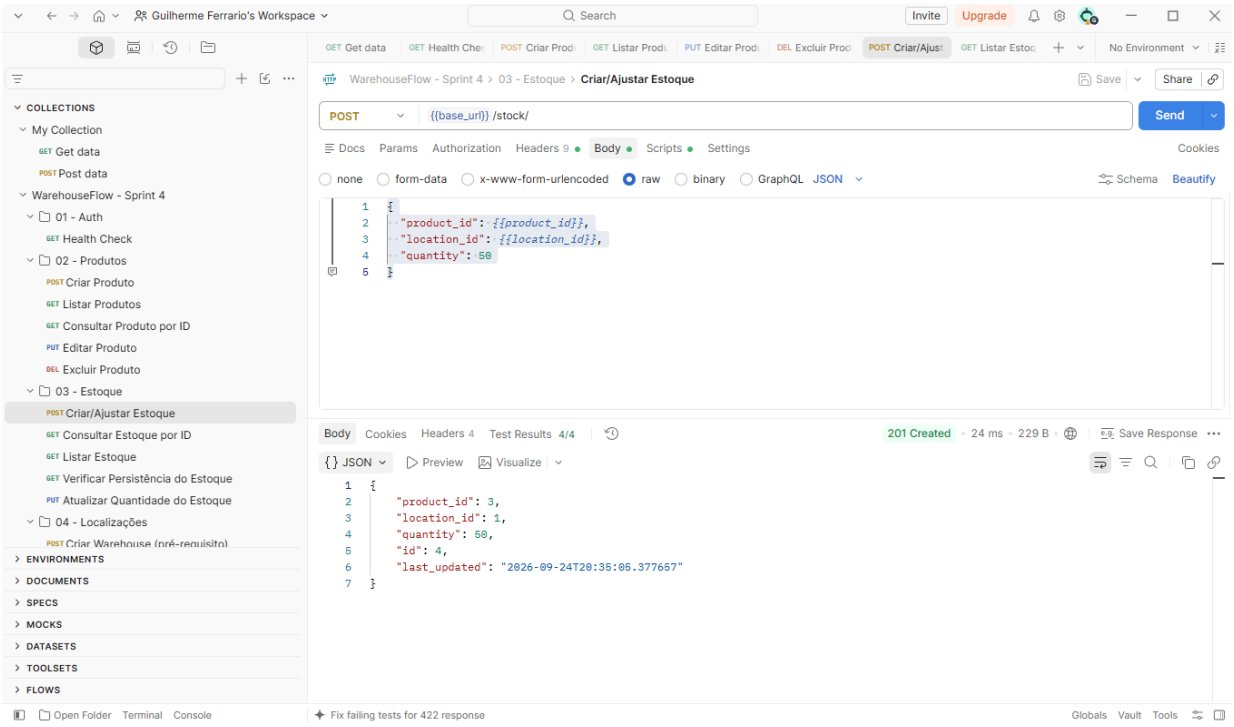
CT 01 — Cadastro de produto válido. Retorno 201 Created.



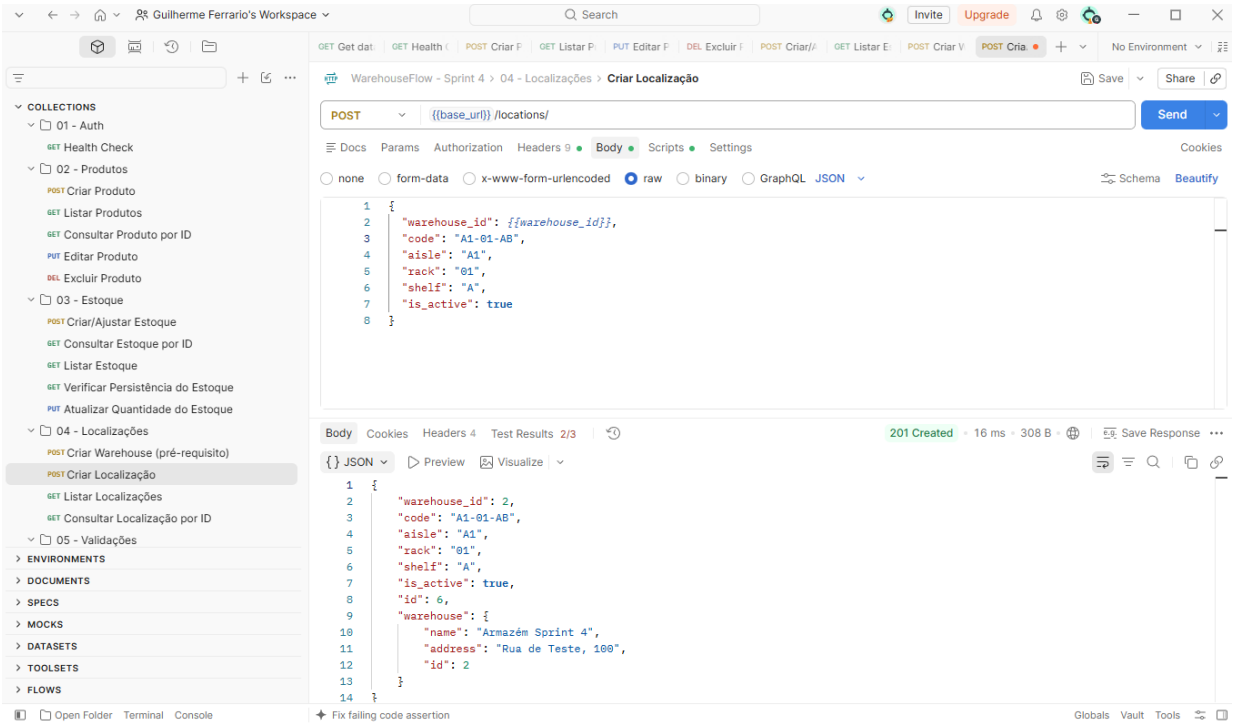
CT 02 — Edição de produto. Retorno 200 OK.



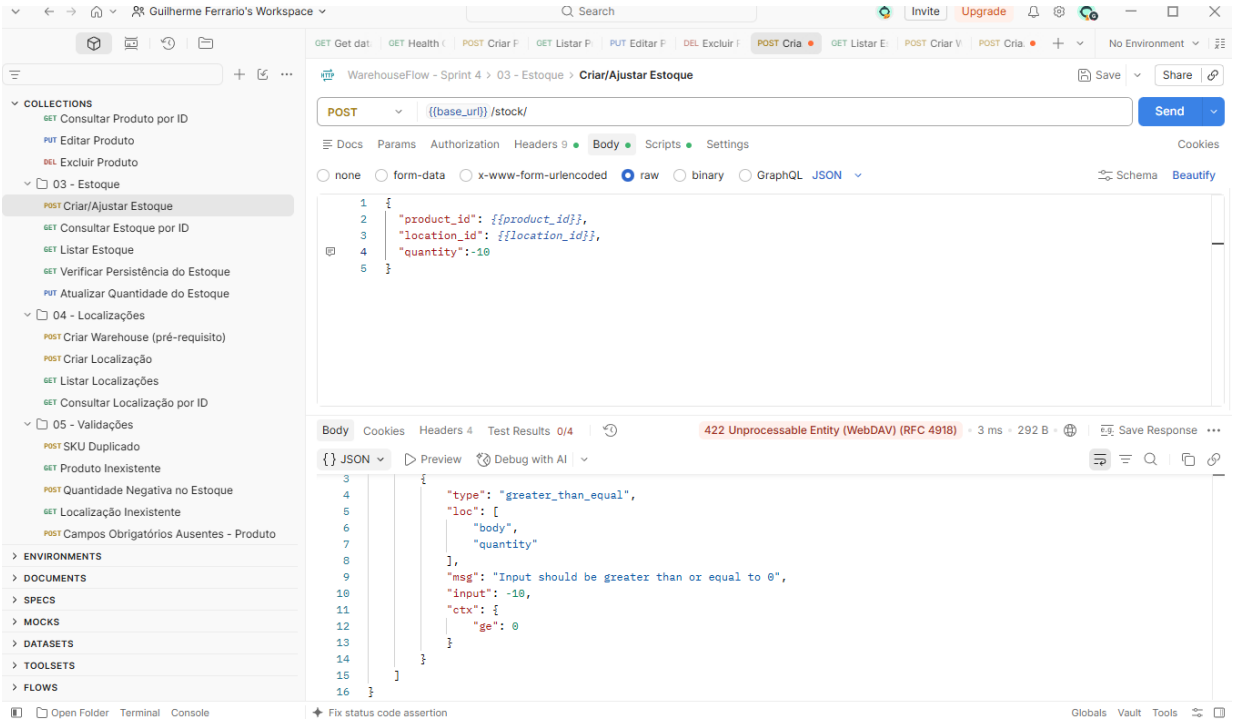
CT 03 — Exclusão de produto. Retorno 204 No Content.



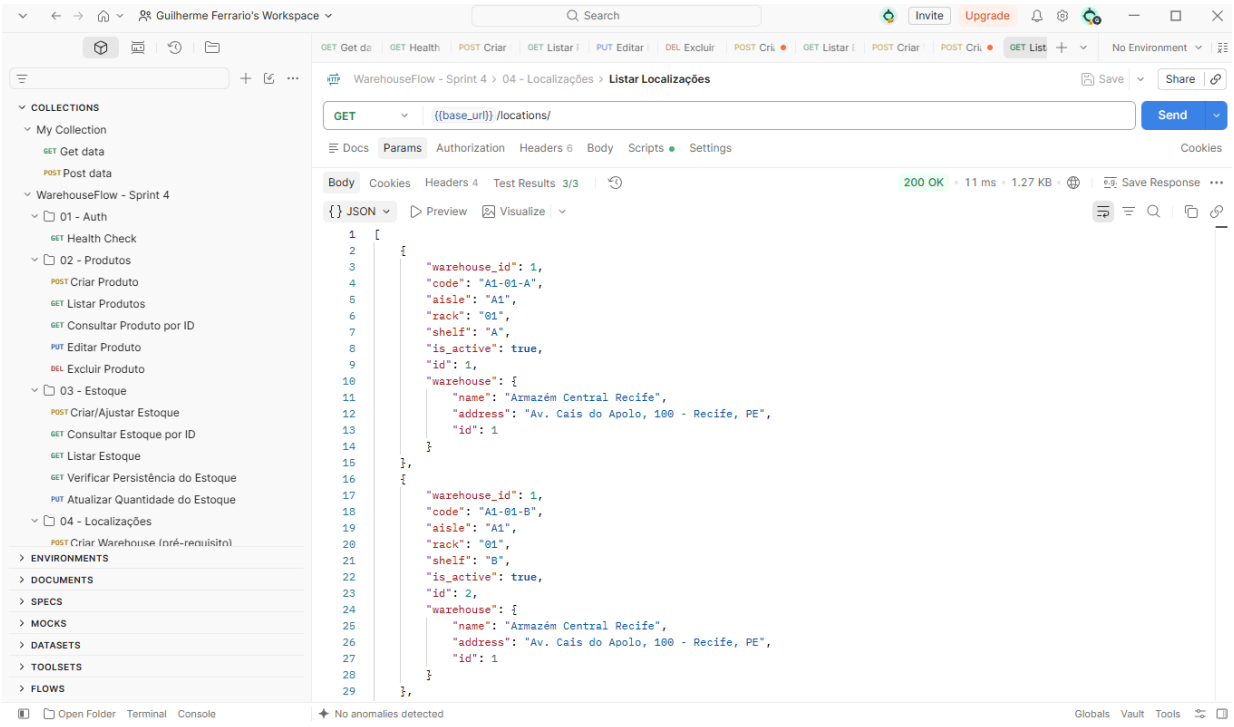
CT 04 — Criação/ajuste de estoque. Retorno 201 Created.



CT 05 — Criação de localização. Retorno 201 Created.



CT 06 — Estoque negativo rejeitado. Retorno 422 Unprocessable Entity.



CT 07 — Listagem de localizações. Retorno 200 OK.

7. Análise dos resultados

As evidências demonstram o funcionamento das operações centrais do módulo. O cadastro, a edição e a exclusão de produtos retornaram os códigos esperados. Também foi validada a criação de estoque relacionando produto, localização e quantidade, bem como o cadastro e a listagem de localizações.

No cenário de quantidade negativa, a API retornou 422 Unprocessable Entity e informou que o campo quantity deve ser maior ou igual a 0. Dessa forma, a validação de entrada foi efetivamente acionada.

8. Cenários previstos para complementação

Os seguintes pontos faziam parte do roteiro geral do Sprint 04 e não possuíam captura correspondente na primeira consolidação desta etapa. Commits, validação de erro no login, navegação entre os módulos e a persistência após reinicialização dos containers foram evidenciados na seção 10; os demais permanecem em aberto:

- Login com credenciais válidas e controle de acesso.
- SKU duplicado.
- Produto ou localização inexistente.
- Campos obrigatórios ausentes.

9. Ferramentas utilizadas

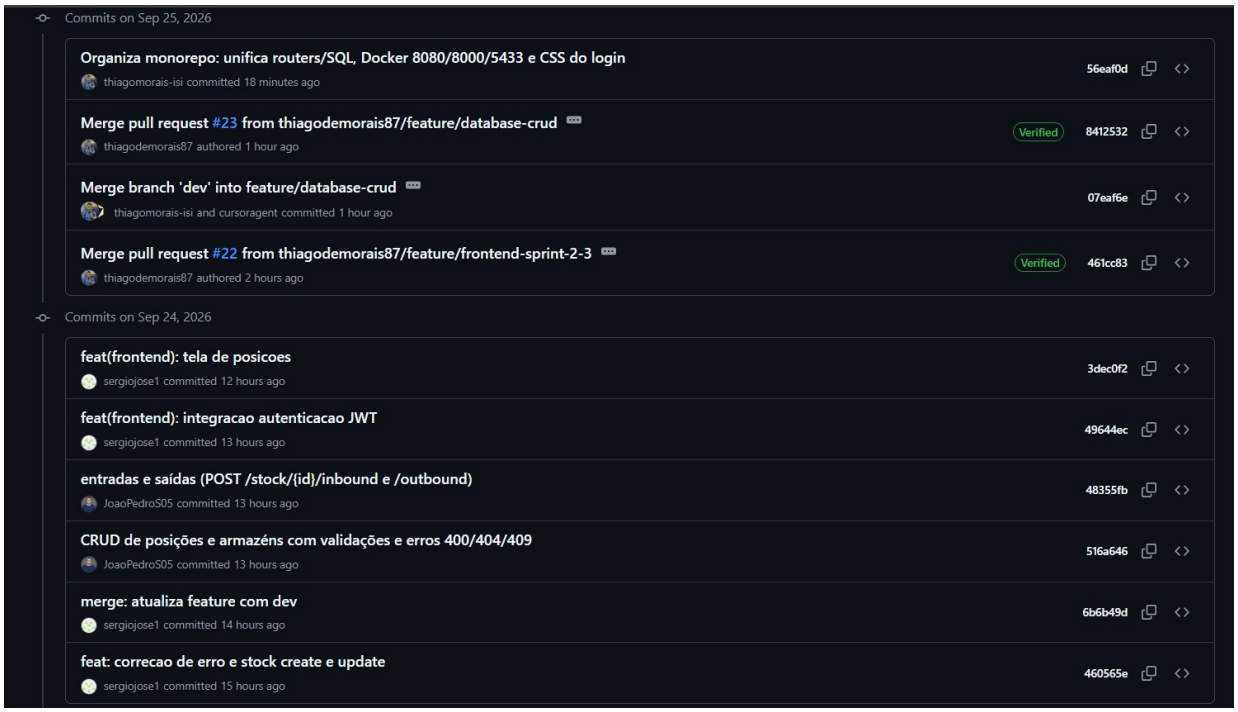
- Postman — execução dos testes da API.
- Docker / Docker Compose — execução dos serviços do projeto.
- API REST do WarehouseFlow — operações de produtos, estoque e localizações.
- PostgreSQL — persistência do sistema, conforme a arquitetura do projeto.

10. Evidências complementares da Sprint 04

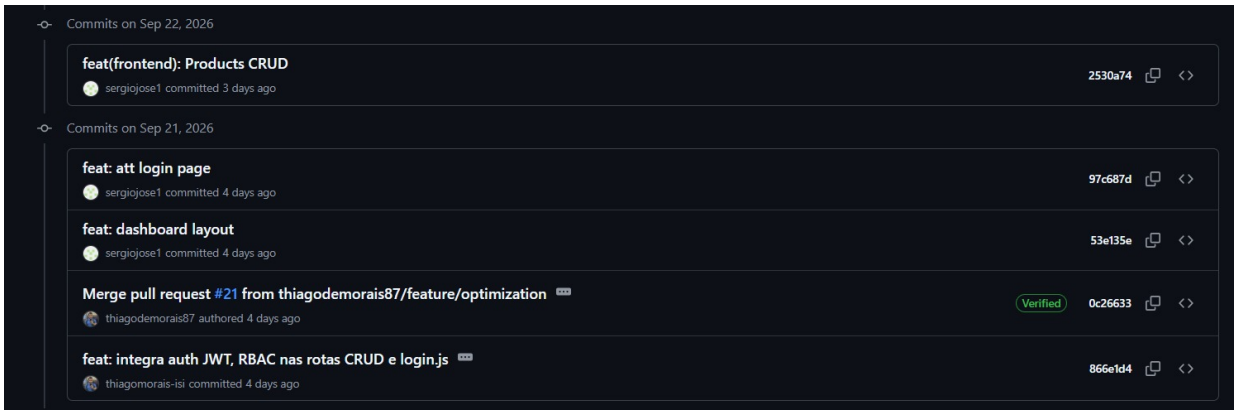
Esta seção reúne evidências adicionais coletadas após a primeira consolidação da Sprint 04, complementando os cenários de teste da API (seção 6) com capturas da aplicação em execução, do repositório no GitHub e da infraestrutura de containers, incluindo uma verificação explícita de persistência de dados após a reinicialização do banco.

10.1 Histórico de commits no GitHub

O repositório do projeto reúne o histórico de commits da equipe, evidenciando a evolução do código ao longo da Sprint 04: integração de autenticação, CRUD de produtos e estoque, cadastro de posições e organização do monorepo.



Evidência 02 — Histórico de commits no repositório GitHub (commits de 25/09/2026).



Evidência 03 — Histórico de commits no repositório GitHub (commits de 21 a 22/09/2026).

10.2 Autenticação — validação de erro

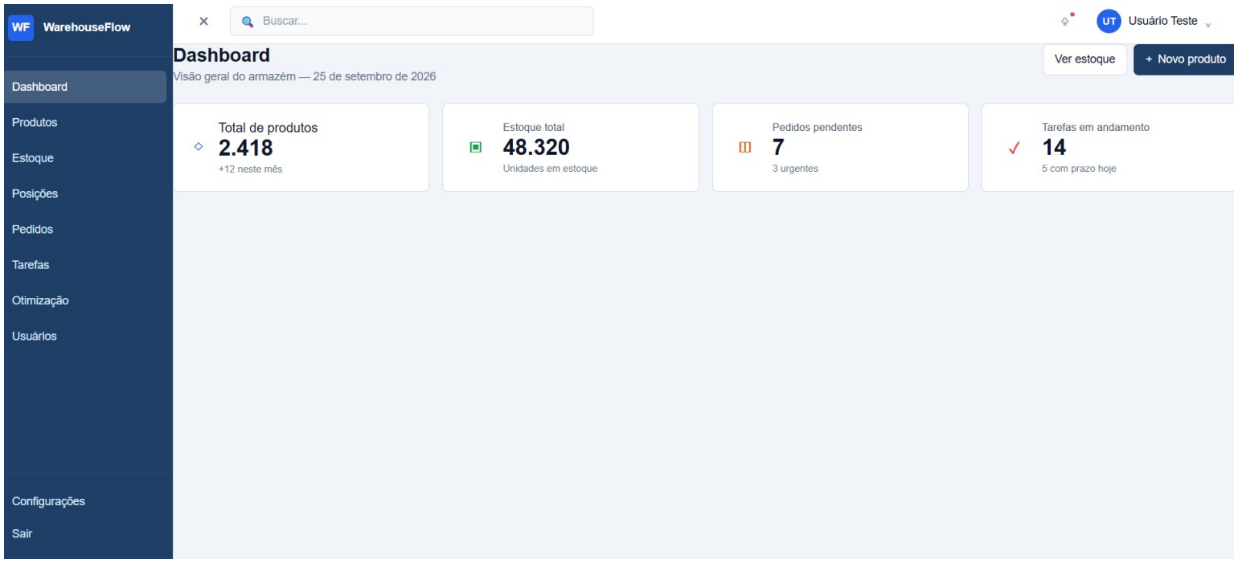
Além dos cenários de sucesso, a tela de login foi testada com credenciais inválidas, confirmando que a aplicação impede o acesso e exibe uma mensagem de erro apropriada ao usuário.



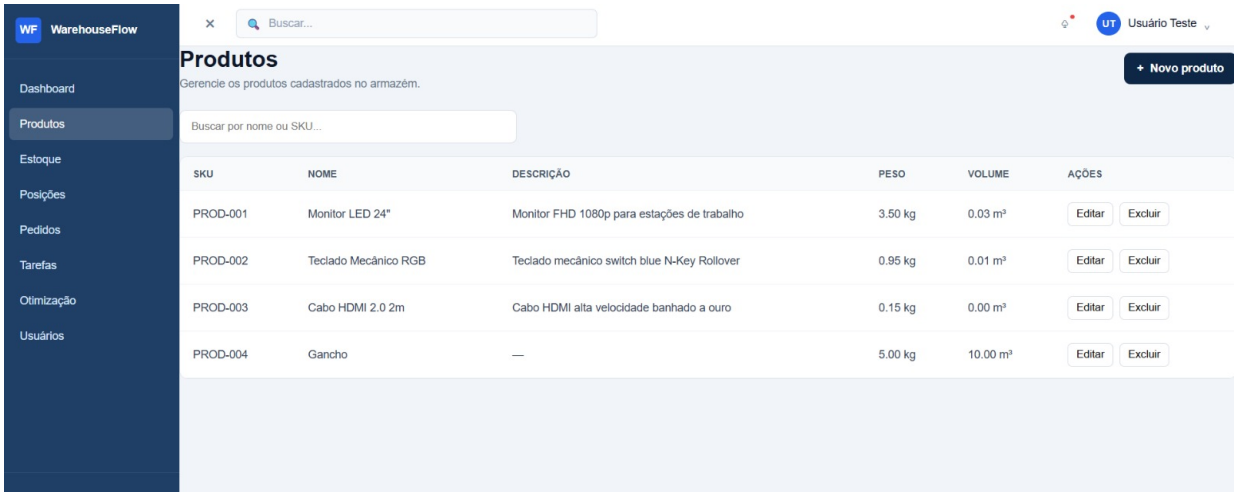
Evidência 04 — Tentativa de login com credenciais inválidas, exibindo a mensagem "E-mail ou senha incorretos."

10.3 Navegação entre os módulos do sistema

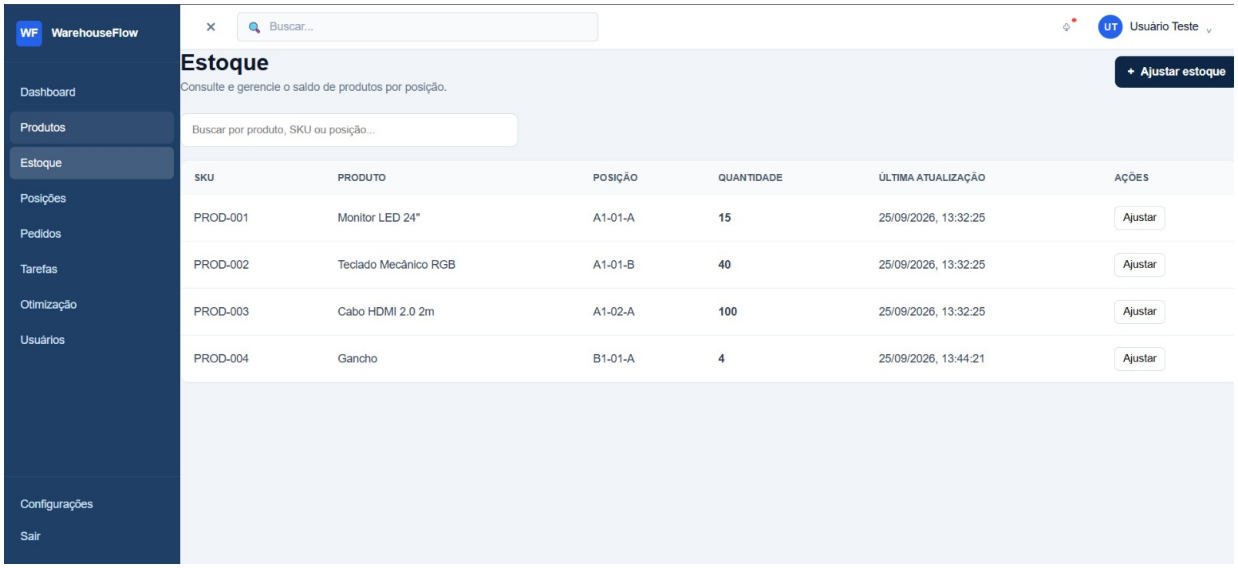
Após autenticado, o usuário navega pelos módulos consolidados nesta Sprint: Dashboard, Produtos, Estoque e Posições, todos consultando dados reais persistidos no PostgreSQL.



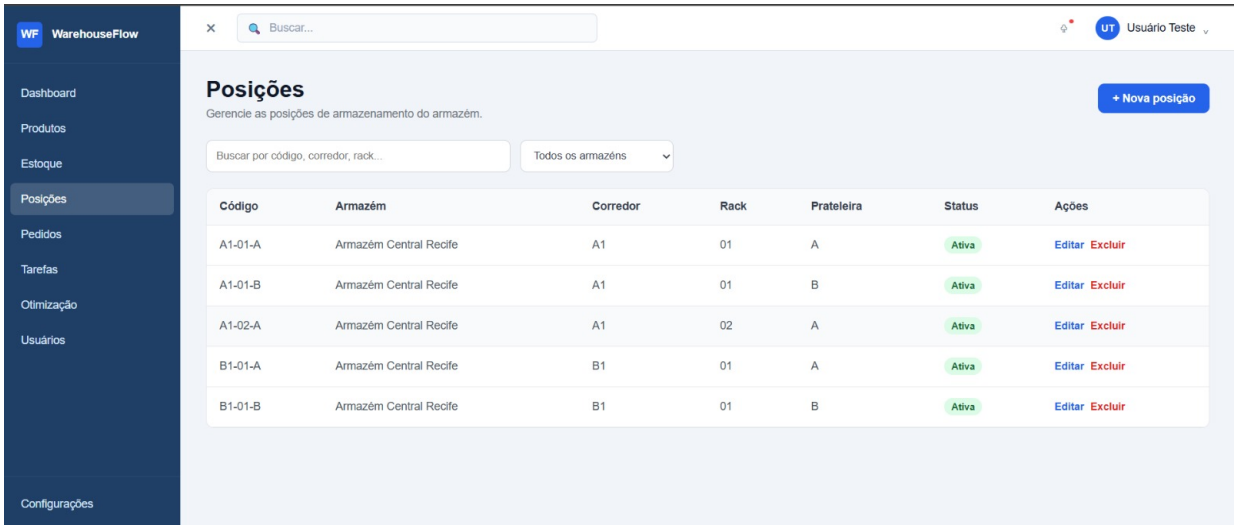
Evidência 05 — Dashboard: visão geral do armazém.



Evidência 06 — Módulo Produtos: listagem cadastrada no sistema.



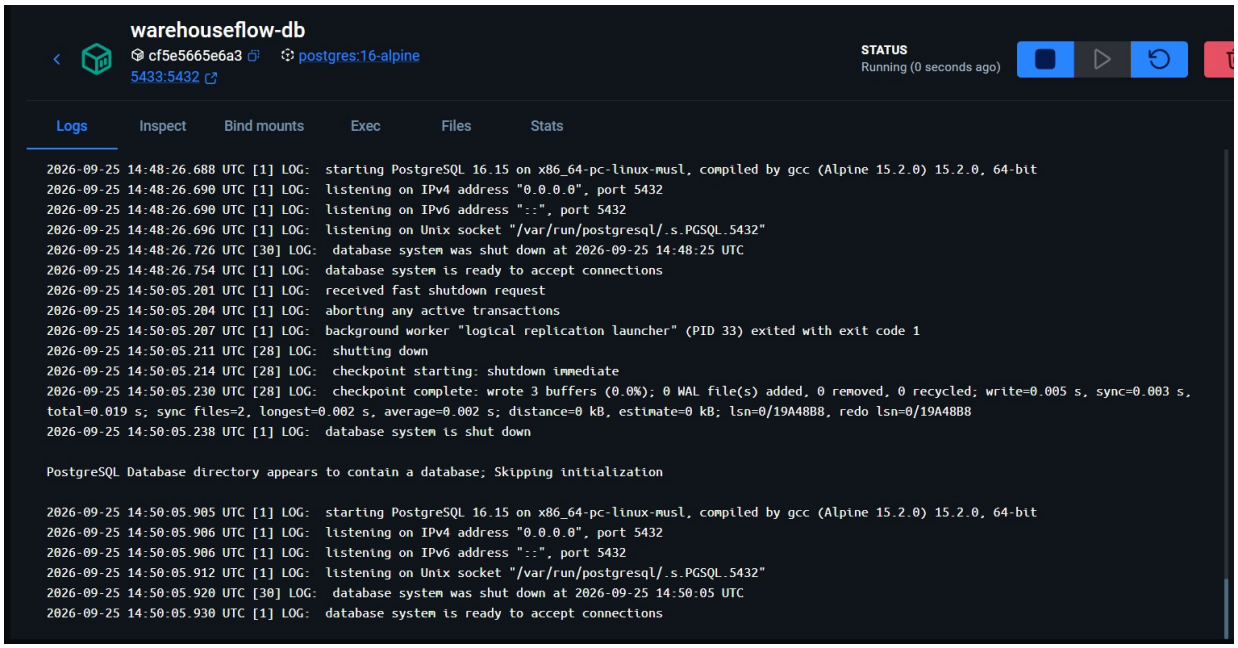
Evidência 07 — Módulo Estoque: saldo por posição.



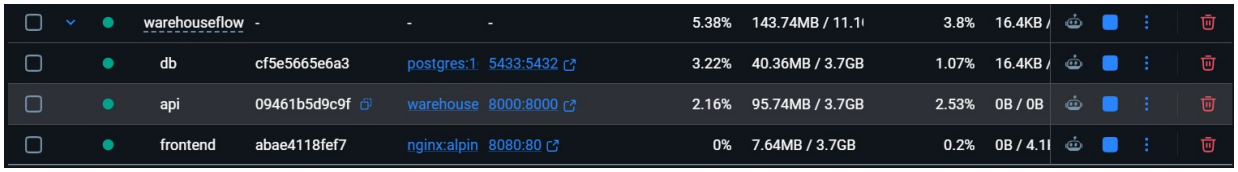
Evidência 08 — Módulo Posições: localizações físicas do armazém.

10.4 Persistência de dados após reinicialização dos containers

Para fechar o cenário de persistência previsto na seção 8, o container do banco de dados (warehouseflow-db) foi reiniciado e os dados foram conferidos em seguida. O log abaixo mostra o desligamento (fast shutdown) e a nova inicialização do PostgreSQL, seguido da confirmação de que os quatro serviços do projeto (warehouseflow, db, api e frontend) permanecem em execução.

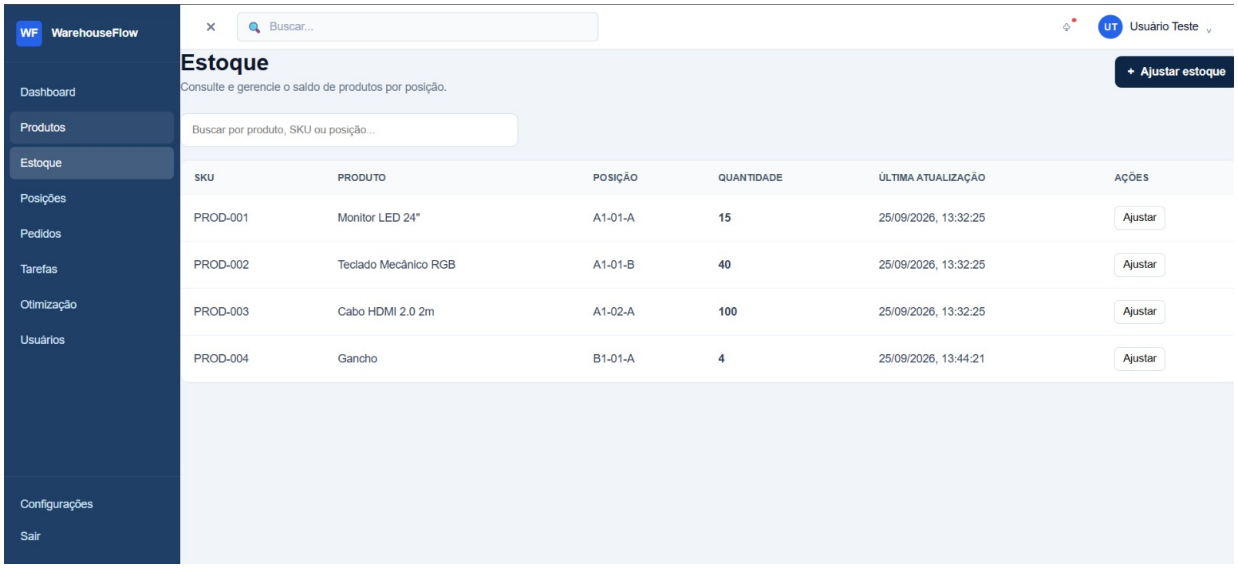


Evidência 09 — Log do container warehouseflow-db: shutdown e reinicialização do PostgreSQL.



Evidência 10 — Containers do WarehouseFlow (warehouseflow, db, api, frontend) em execução após o restart do banco.

Em seguida, o módulo Estoque foi consultado novamente: os saldos e os horários de "última atualização" (13:32:25 e 13:44:21) permaneceram idênticos aos registrados antes da reinicialização (log às 14:50), confirmando que os dados persistiram corretamente.



Evidência 11 — Módulo Estoque consultado após a reinicialização do container do banco, com saldos e timestamps preservados — persistência confirmada.

11. Repositório e protótipo

GitHub: https://github.com/thiagodemorais87/WarehouseFlow

Figma: https://www.figma.com/design/uGceagehdqafcFckYSlWZY/Sem-título?node-id=5-569&t=LlfUx9e9ylwhR2CV-0

12. Dificuldades encontradas

Foi necessário atualizar o WSL para permitir a execução adequada do Docker no ambiente local. Após a atualização, os containers do projeto puderam ser executados e os testes da API foram realizados pelo Postman.

13. Próximos passos

- Executar e registrar os cenários ainda não evidenciados (SKU duplicado, produto/localização inexistente, campos obrigatórios ausentes), caso sejam exigidos.
- Consolidar os Sprints 01 a 04 em um único PDF para a entrega.
- Prosseguir para as funcionalidades dos próximos Sprints.

14. Considerações finais

O Sprint 04 apresentou evidências funcionais das operações de Produtos, Estoque e Localizações. Os resultados registrados no Postman demonstram respostas HTTP compatíveis com as operações executadas e uma validação explícita para impedir quantidade negativa no estoque. As evidências complementares desta revisão — commits no GitHub, validação de erro no login, navegação entre os módulos consolidados e a verificação de persistência após a reinicialização dos containers — fecham os principais pontos em aberto identificados anteriormente.

WAREHOUSEFLOW

DOCUMENTAÇÃO TÉCNICA — SPRINT 05

Módulo 2 — Pedidos, Tarefas, Usuários e Integração

Grupo 26 • Turma 8MB • Ciência da Computação
Prazo da Sprint: 03/10/2026

Pessoa 5 — Integração, Testes e Documentação

1. Identificação da Sprint 05

Item	Informação
Projeto	WarehouseFlow — sistema de gerenciamento e otimização de estoque
Grupo	26
Turma	8MB
Curso	Ciência da Computação
Sprint	05
Responsabilidade	Pessoa 5 — Integração, Testes e Documentação
Prazo	03/10/2026

2. Objetivo da Sprint 05

A Sprint 05 tem como objetivo consolidar o Módulo 2 do WarehouseFlow, ampliando o sistema para os fluxos de Pedidos e Tarefas e fortalecendo a autenticação, o controle de perfis e a integração entre interface, API e PostgreSQL. A responsabilidade da Pessoa 5 é validar o funcionamento integrado, executar os cenários de teste, verificar persistência após reinicialização, registrar falhas e correções e organizar as evidências da entrega.

O fluxo principal definido para a validação é: Login → Pedido → Itens → Tarefa → Operador → Status → Persistência. A documentação adota como princípio que uma funcionalidade somente será declarada comprovada quando houver teste ou evidência correspondente.

3. Escopo da Sprint 05

- CRUD completo de Pedidos e Tarefas, incluindo regras de negócio e validações.
- Login com JWT, proteção de rotas e perfis ADMIN, GESTOR e OPERADOR.
- Listagem, criação, inclusão de itens, visualização e alteração de status de pedidos pela interface.
- Listagem, criação, vínculo com pedido, atribuição de operador e alteração de status de tarefas pela interface.
- Integração Frontend → API → PostgreSQL.
- Persistência dos dados após reinicialização dos serviços.
- Manutenção do endpoint POST /optimization/route e documentação do contrato para a Sprint 06.
- Organização de branches, commits, pull requests, testes, correções e documentação.

4. Responsabilidade da Pessoa 5

Guilherme Branco Ferrario atua como Pessoa 5 — Integração, Testes e Documentação. Nesta Sprint, sua atuação está concentrada em verificar o fluxo integrado da aplicação, confrontar o comportamento observado com os requisitos da equipe, testar cenários positivos e negativos, conferir a persistência no PostgreSQL, registrar bugs e correções e consolidar as evidências acadêmicas.

5. Evidências já disponíveis e reaproveitadas da Sprint 04

Antes da execução específica da Sprint 05, a equipe já possuía evidências reais produzidas na Sprint 04. Elas são reaproveitadas neste documento como comprovação da base funcional sobre a qual o Módulo 2 será integrado. Essas imagens não são apresentadas como testes novos da Sprint 05; elas registram funcionalidades anteriores que permanecem relevantes para a integração.

5.1 Operações de Produtos, Estoque e Posições

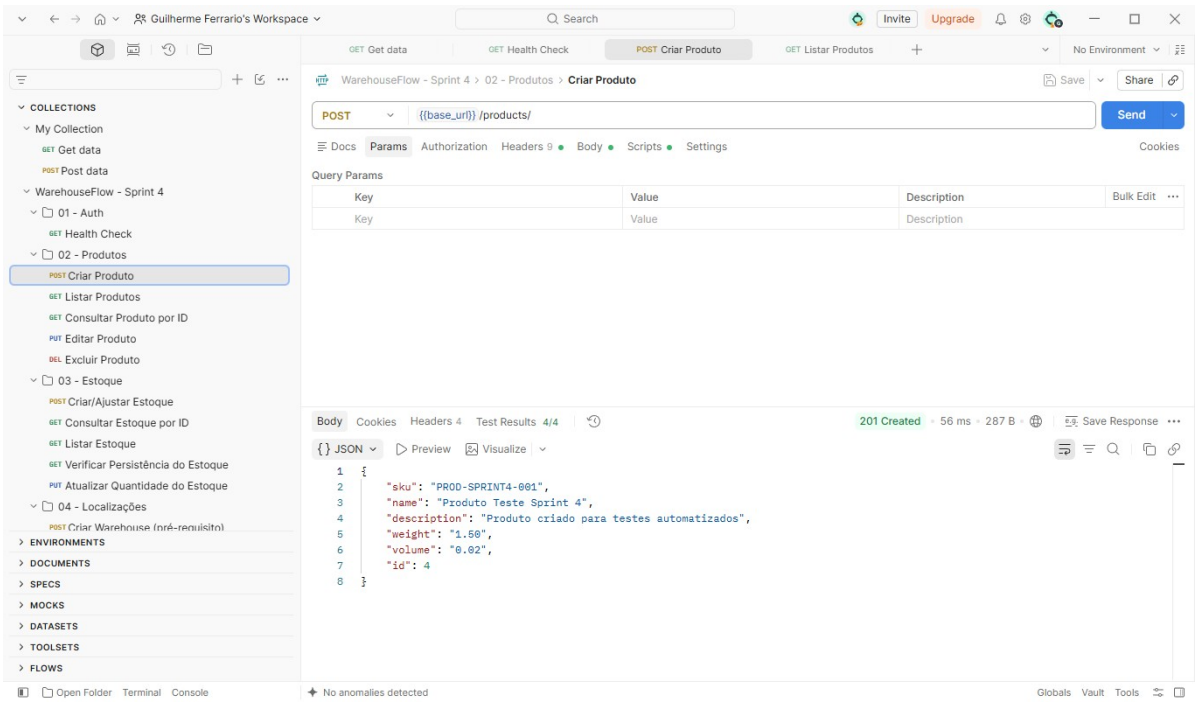


Figura 1 — Cadastro de produto válido. Retorno HTTP 201 Created.

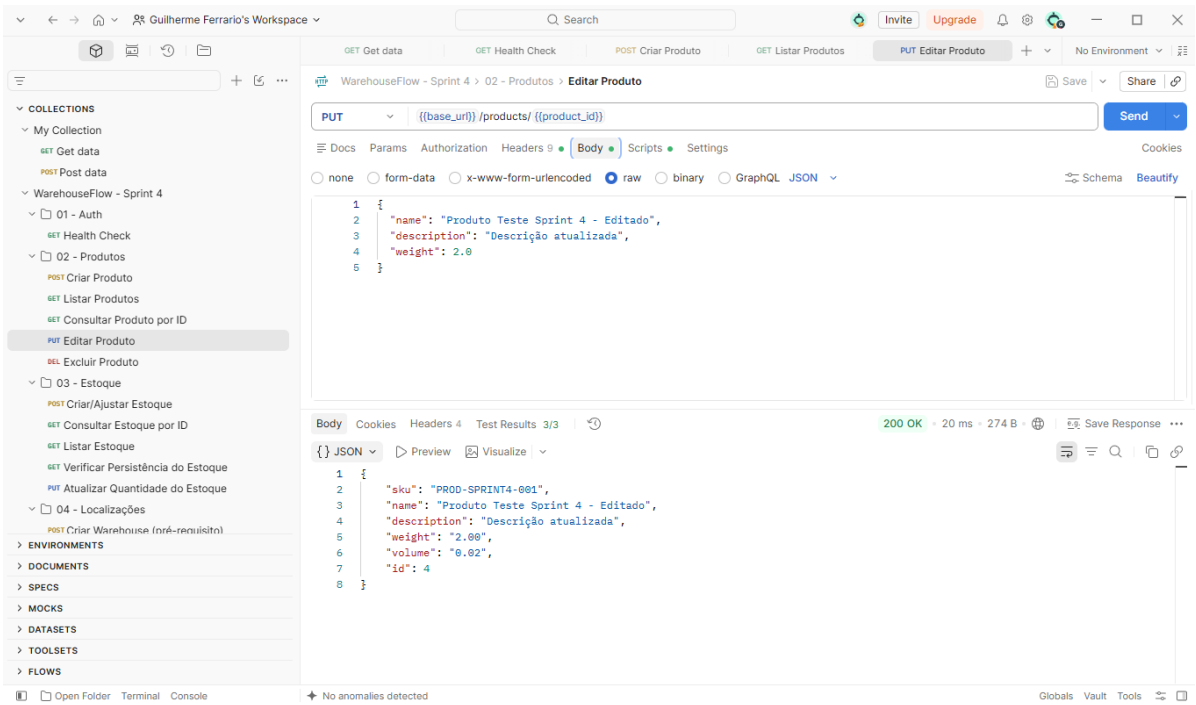


Figura 2 — Edição de produto. Retorno HTTP 200 OK.

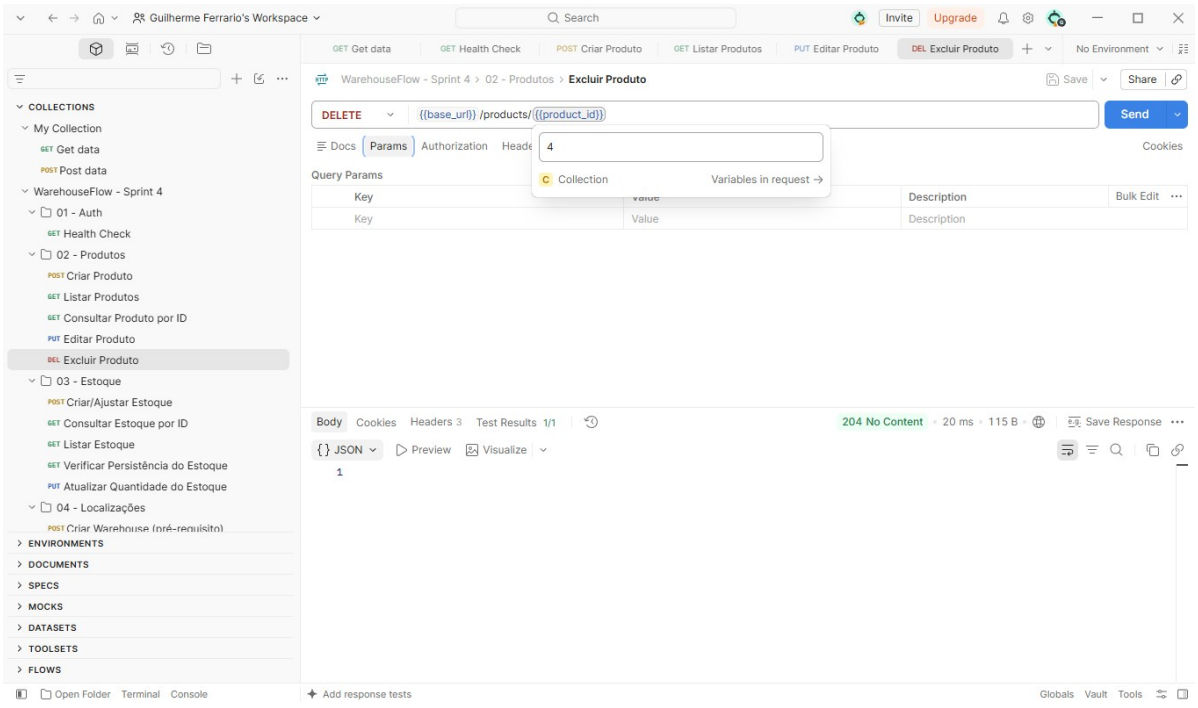


Figura 3 — Exclusão de produto. Retorno HTTP 204 No Content.

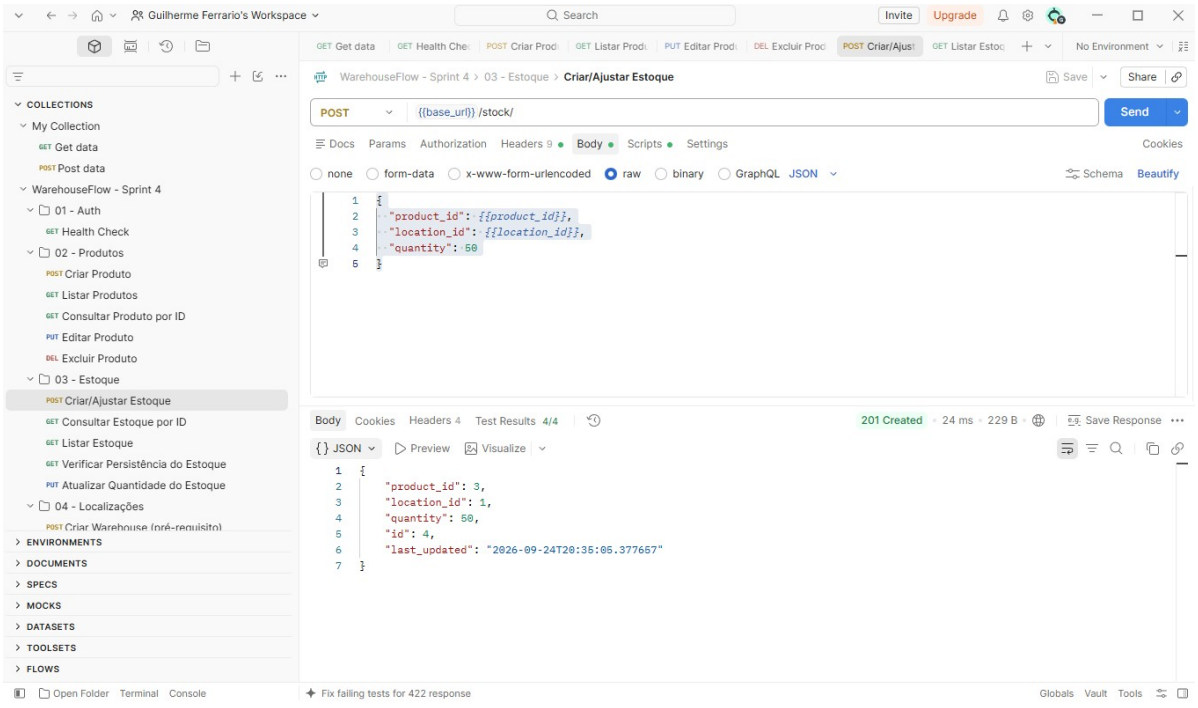


Figura 4 — Criação/ajuste de estoque associado ao produto. Retorno HTTP 201 Created.

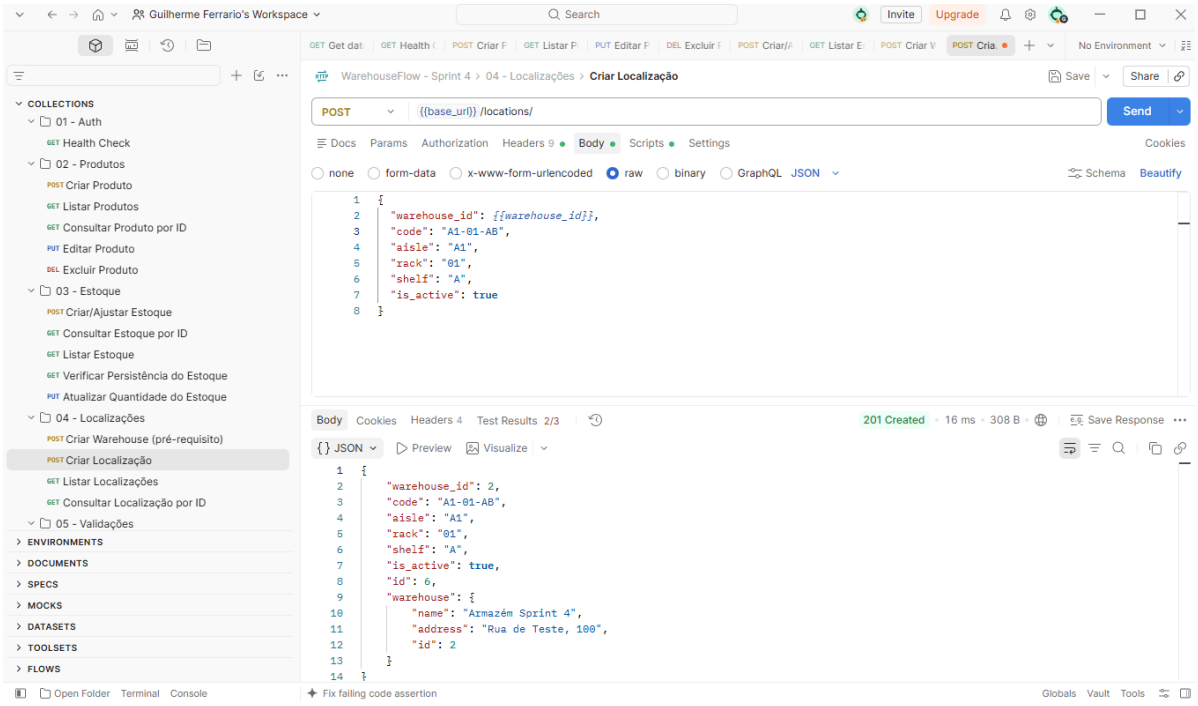


Figura 5 — Criação de localização. Retorno HTTP 201 Created.

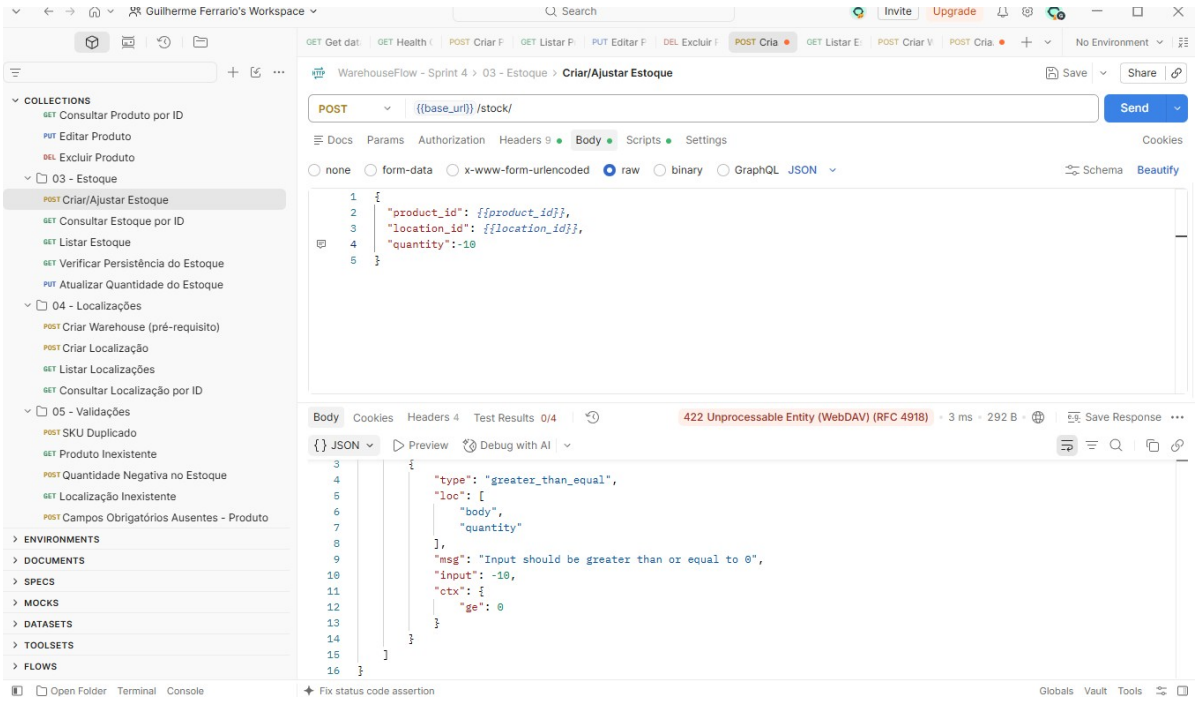


Figura 6 — Validação de quantidade negativa no estoque. Retorno HTTP 422 Unprocessable Entity.

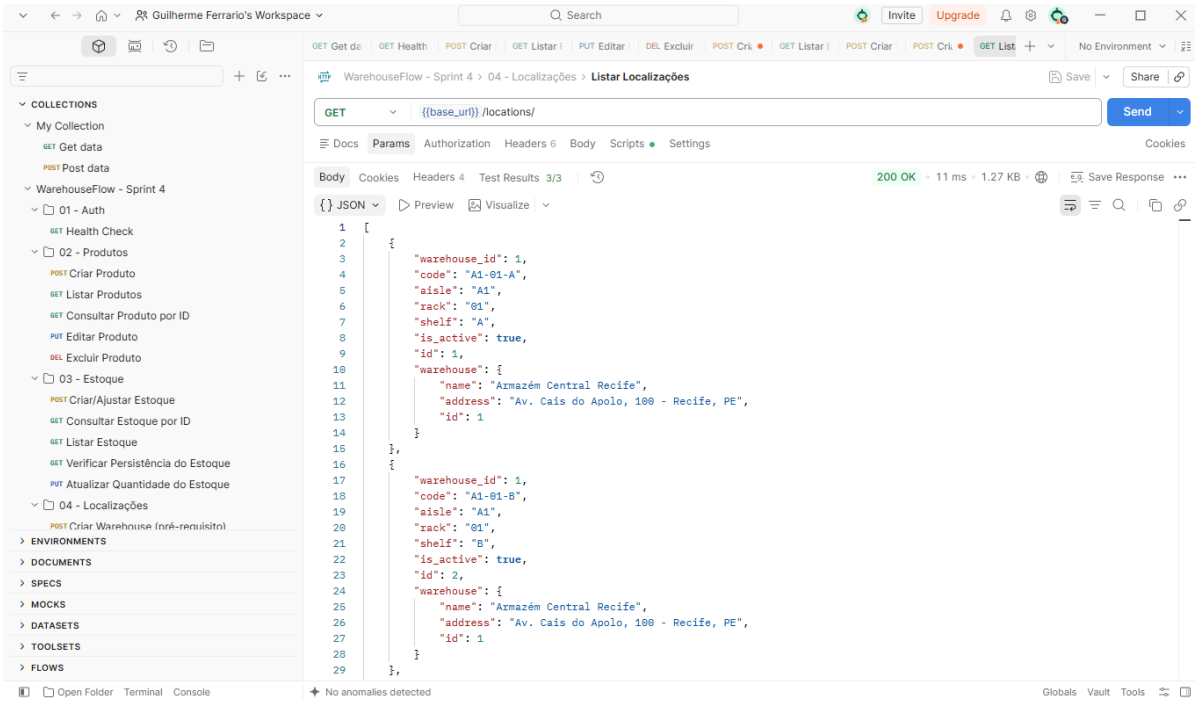


Figura 7 — Listagem de localizações. Retorno HTTP 200 OK.

5.2 Evidências de interface, autenticação, ambiente e versionamento já disponíveis

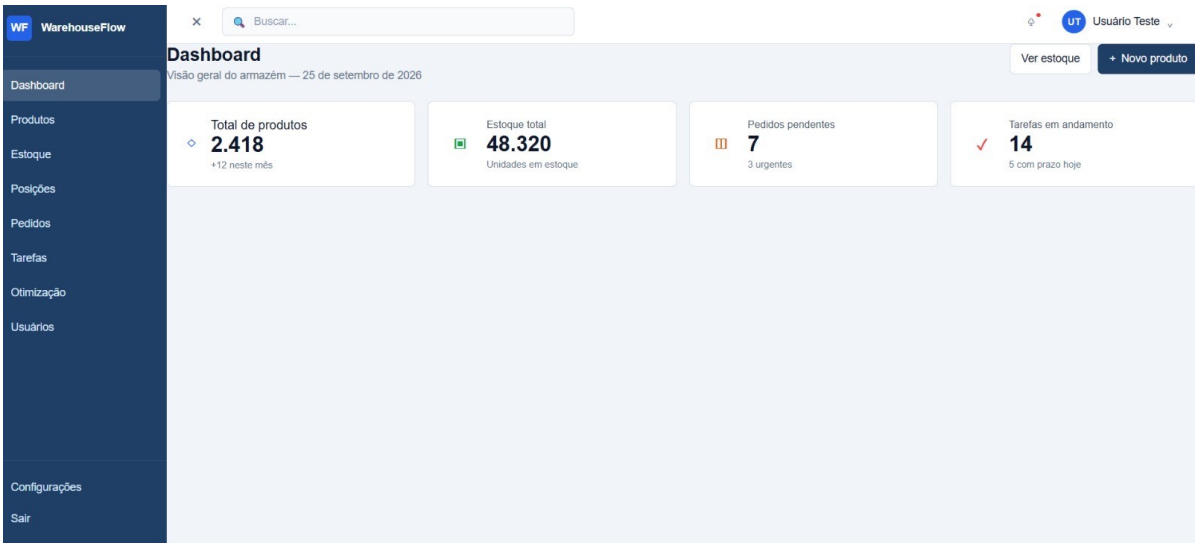


Figura 9 — Dashboard do WarehouseFlow após acesso à aplicação.

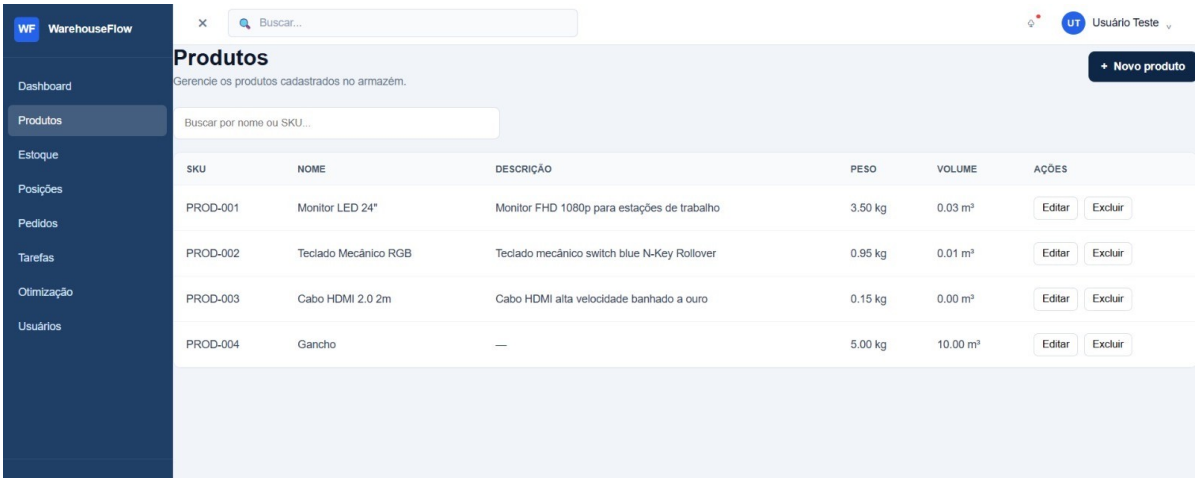


Figura 10 — Tela de Produtos pela interface do sistema.

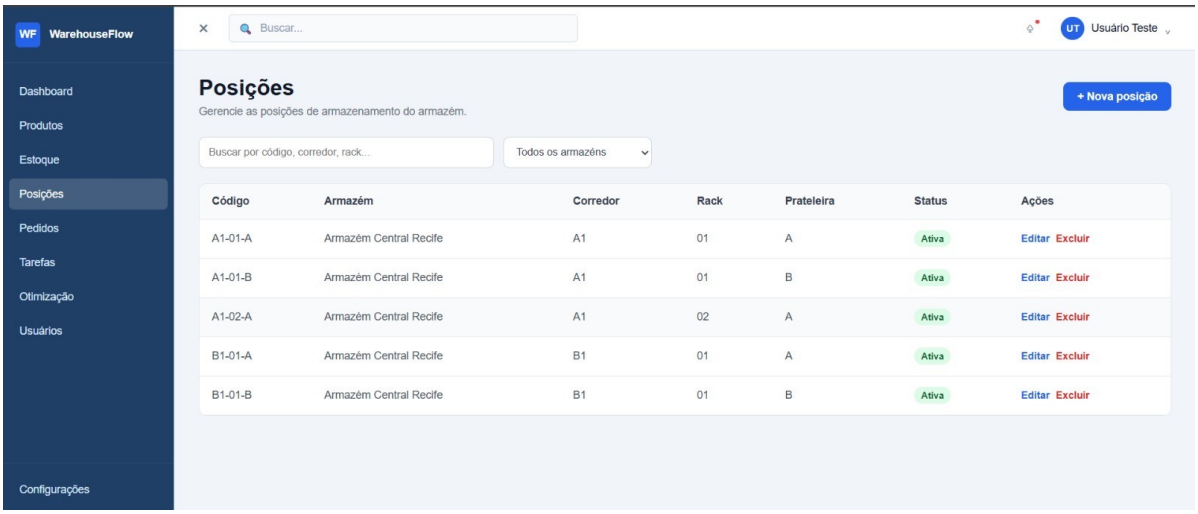


Figura 11 — Tela de Posições/Localizações pela interface.

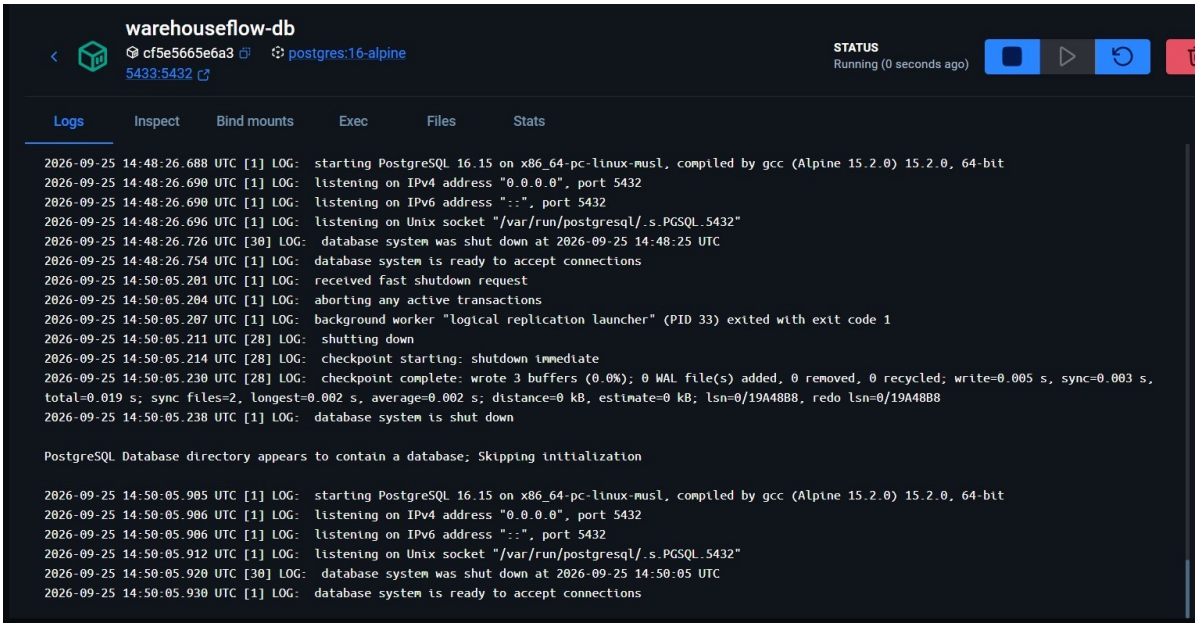


Figura 12 — Terminal com execução dos serviços/ambiente Docker e PostgreSQL.

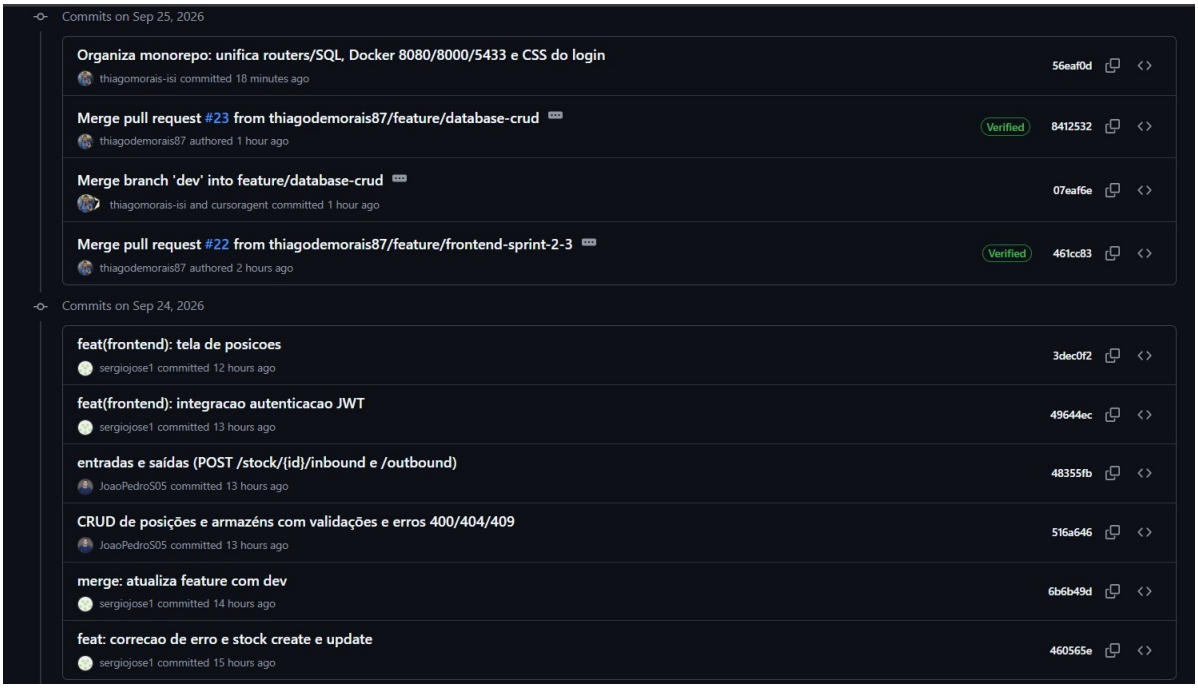


Figura 13 — Histórico de commits do repositório no GitHub.



Figura 14 — Evidência complementar do histórico de commits no GitHub.

6. Relação das evidências existentes com a Sprint 05

Evidência existente	Utilização na Sprint 05
Produtos / Estoque / Posições	Base funcional para os pedidos, itens e futura relação entre produtos e posições.
Login com erro	Evidência anterior de tratamento de autenticação inválida; a Sprint 05 deverá complementar com login válido e perfis.
Dashboard / Produtos / Posições	Demonstração da navegação da aplicação e da existência da interface integrada.
Docker / PostgreSQL	Evidência do ambiente de execução; a Sprint 05 ainda deverá comprovar a persistência após restart.
Commits GitHub	Evidência de versionamento e organização do trabalho; deverão ser complementados com os commits específicos da Sprint 05.

7. Matriz de validação da Sprint 05

ID	Cenário	Critério de aceite	Evidência
CT01	Login válido	Usuário autenticado e acesso ao sistema	Evidência nova da Sprint 05
CT02	Login inválido	Mensagem de erro amigável	Figura 8 já disponível; complementar se houver correção/novo comportamento
CT03	Perfil ADMIN	Permissões compatíveis com ADMIN	Evidência nova
CT04	Perfil GESTOR	Permissões compatíveis com GESTOR	Evidência nova
CT05	Perfil OPERADOR	Permissões compatíveis com OPERADOR	Evidência nova
CT06	Criar pedido válido	Pedido criado e exibido na interface	Evidência nova
CT07	Pedido sem item	Sistema rejeita conforme regra definida	Evidência nova
CT08	Adicionar item válido	Item associado ao pedido	Evidência nova
CT09	Produto inexistente	Operação rejeitada com mensagem adequada	Evidência nova
CT10	Quantidade ≤ 0	Operação rejeitada com mensagem adequada	Evidência nova
CT11	Visualizar detalhes do pedido	Dados e itens apresentados corretamente	Evidência nova
CT12	Alterar status do pedido	Status atualizado conforme	Evidência nova

		regras	
CT13	Criar tarefa	Tarefa criada e exibida	Evidência nova
CT14	Vincular tarefa a pedido	Relação pedido–tarefa preservada	Evidência nova
CT15	Atribuir operador	Operador associado à tarefa	Evidência nova
CT16	Alterar status da tarefa	Status atualizado conforme regras	Evidência nova
CT17	Fluxo completo	Login → Pedido → Item → Tarefa → Operador → Status	Evidência nova, preferencialmente pela interface
CT18	Persistência após restart	Dados continuam disponíveis após reinicialização	Evidência nova antes/depois do restart
CT19	Testes automatizados	Pytest executado sem falhas incompatíveis com a entrega	Print do terminal
CT20	GitHub Sprint 05	Commits/PRs relevantes identificáveis	Prints novos do GitHub

8. Estratégia de integração

A validação da Sprint 05 deve priorizar a execução pela interface, conforme orientação da equipe, sem deixar de utilizar API e banco como camadas de conferência. O Frontend envia as requisições à API; a API valida regras, executa a lógica e persiste os dados no PostgreSQL; a interface apresenta o resultado ao usuário.

- Login → autenticação e autorização
- Criação do pedido → registro do pedido
- Inclusão de itens → associação produto/pedido
- Criação da tarefa → vínculo com pedido
- Atribuição → associação do operador
- Alteração de status → atualização do fluxo operacional
- Restart → confirmação de persistência

9. Persistência e reinicialização

A documentação da Sprint 05 considera persistência como um critério específico de integração. A validação deve registrar um pedido e uma tarefa, apresentar os dados antes da reinicialização, reiniciar os serviços por Docker Compose e consultar novamente os registros. A permanência dos mesmos dados após o restart será a evidência de que a gravação está ocorrendo no PostgreSQL persistente, e não apenas em memória.

10. Consistência entre Banco, Models, Schemas, MER e Modelo Relacional

A Sprint 05 também prevê a conferência de consistência entre a estrutura do PostgreSQL e as representações utilizadas pela aplicação e pela documentação. A revisão deve considerar especialmente roles, usuários/perfis, pedidos, itens de pedido, tarefas e optimization_results. O objetivo é evitar divergências de nomes, campos, tipos, chaves e relacionamentos entre código, banco e documentação.

11. Motor de otimização e contrato para a Sprint 06

O motor de otimização existente deve ser preservado sem alterações que quebrem o funcionamento atual. O contrato de integração previsto para a Sprint 06 é:

Pedido → Itens → Produtos → Posições → Rota otimizada

A documentação técnica deve manter o registro do uso de Nearest Neighbor + 2-opt e do endpoint POST /optimization/route. A Sprint 05 não prevê a criação de uma nova tela do motor de otimização.

12. Organização do trabalho e versionamento

O repositório oficial do projeto permanece no GitHub. Os commits e pull requests devem registrar de forma objetiva as funcionalidades, correções, testes e documentação da Sprint 05. A evidência já existente mostra o histórico do repositório; os registros específicos desta Sprint serão adicionados ao conjunto final de evidências.

Repositório: github.com/thiagodemorais87/WarehouseFlow

13. Situação das evidências na presente versão

Esta versão da documentação já incorpora as evidências que estavam disponíveis antes da execução da Sprint 05: operações de Produtos, Estoque e Posições pela API; tela de login com erro; Dashboard; Produtos; Posições; ambiente Docker/PostgreSQL; e histórico de commits do GitHub. Essas evidências são identificadas como registros anteriores e não são utilizadas para afirmar que os novos fluxos de Pedidos e Tarefas já foram validados.

As evidências específicas de Pedidos, Tarefas, perfis de acesso, fluxo completo, persistência após restart e execução dos testes automatizados serão incorporadas após a execução da Sprint 05.

14. Critérios para encerramento da Sprint 05

- Fluxo principal demonstrado pela interface.
- Pedidos e Tarefas validados em cenários positivos e negativos.
- Perfis ADMIN, GESTOR e OPERADOR verificados.
- Integração Frontend → API → PostgreSQL confirmada.
- Persistência após reinicialização comprovada.
- Testes automatizados executados e registrados.
- Bugs e correções relevantes documentados.
- Commits/PRs da Sprint organizados.
- Documentação atualizada somente com afirmações respaldadas por evidências reais.

15. Equipe

Integrante	Matrícula
Gabriel George de Araújo Figueiredo	01605236
Guilherme Branco Ferrario	01596391
João Pedro Silva de Araujo	01606470
Sérgio José de Araújo Júnior	01590694
Thiago de Morais Gonçalves	01609695

16. Referências

- Repositório oficial — WarehouseFlow: github.com/thiagodemorais87/WarehouseFlow

- Documentação consolidada das Sprints 01 a 04 — Grupo 26.
- Documentação técnica da Sprint 04 — Grupo 26.
- Orientações da Sprint 05 fornecidas pela equipe.

WAREHOUSEFLOW
DOCUMENTAÇÃO DA SPRINT 06

Módulo 3 — Integração do motor de otimização

Grupo	26
Turma	8MB
Curso	Ciência da Computação
Projeto	Sistema de Gestão de Armazém e Estoque — WarehouseFlow
Responsabilidade	Integração, testes e documentação

Repositório: github.com/thiagodemorais87/WarehouseFlow

Recife — PE | 2026

1. Objetivo da Sprint 06

A Sprint 06 tem como objetivo integrar os módulos do WarehouseFlow, com foco no terceiro módulo de otimização. O trabalho contempla a revisão das APIs e da estrutura do banco de dados, a integração do motor de otimização ao backend, os ajustes da interface e da navegação e a validação dos fluxos operacionais.

Busca-se conectar os dados de produtos, estoque, posições e pedidos ao processamento de rotas, disponibilizando os resultados de forma organizada na interface.

2. Divisão das responsabilidades

Responsável	Atividades
Pessoa 1 — Banco de Dados + Backend	Revisar banco e APIs; implementar endpoints necessários; integrar serviços e verificar gravação e recuperação dos dados.
Pessoa 2 — Login + Usuários + Perfis	Revisar login/logout, usuários, perfis e permissões; proteger páginas e corrigir problemas de acesso e navegação.
Pessoa 3 — Motor de Otimização	Revisar o algoritmo, integrá-lo ao backend, utilizar dados do sistema e retornar resultados para diferentes cenários.
Pessoa 4 — Frontend	Ajustar a interface do terceiro módulo, integrar telas às APIs, padronizar componentes e melhorar mensagens, carregamentos e navegação.
Pessoa 5 — Integração + Testes + Documentação	Validar integração, organizar evidências, registrar correções, acompanhar ajustes da Pré-Banca, atualizar documentação e conferir o repositório.

3. Integração entre os módulos

O fluxo integrado conecta autenticação e controle de acesso às funcionalidades operacionais. Produtos, estoque, posições e pedidos fornecem os dados para a otimização. O backend encaminha os dados ao motor, recebe a rota calculada e disponibiliza a resposta para a interface.

Etapa	Descrição
1. Autenticação	Acesso ao sistema por credenciais e uso conforme o perfil.
2. Dados operacionais	Produtos, estoque e posições/localizações compõem as informações da operação.
3. Pedido e itens	Os itens do pedido indicam os produtos a considerar no fluxo operacional.
4. Otimização	O backend envia os dados ao motor e recebe a sequência calculada.
5. Resultado	A interface apresenta a rota e as informações retornadas pelo serviço.

4. Motor de otimização

O WarehouseFlow utiliza **Nearest Neighbor** para construir uma rota inicial com base na proximidade e **2-opt** para avaliar trocas entre segmentos e buscar reduzir o percurso.

Os endpoints documentados nas requisições são **POST /optimization/route** (otimização por coordenadas), **GET /optimization/orders/{orderId}/pick-locations** (consulta das posições de picking de um pedido) e **POST /optimization/route/by-order** (otimização por pedido). Também há um cenário de rejeição de IDs duplicados.

As respostas incluem rota original, rota Nearest Neighbor e rota 2-opt, além de métricas de distância. A validação de IDs duplicados retorna HTTP 400 com mensagem indicando que cada posição deve aparecer apenas uma vez na rota.

5. Evidências dos testes de integração

As capturas do Postman apresentam requisições e respostas de autenticação, cálculo de rota por coordenadas, consulta de posições de picking, otimização por pedido e tratamento de IDs duplicados.

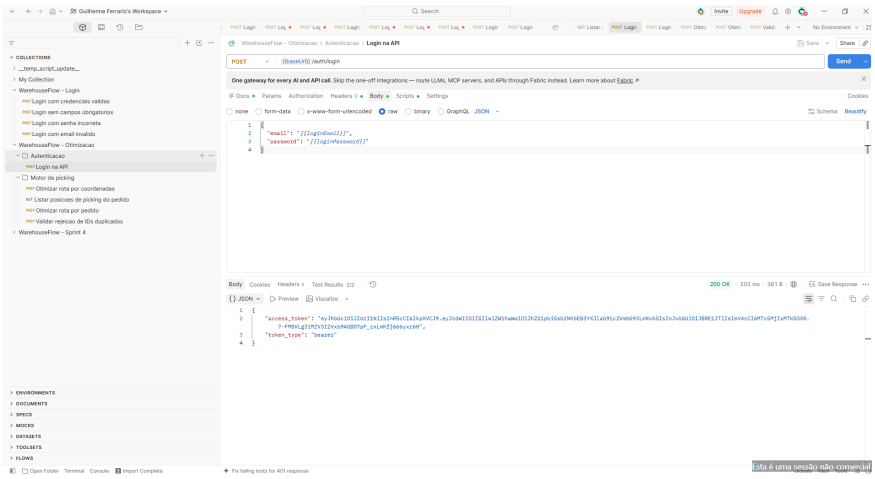


Figura 1 — Autenticação pela API: resposta HTTP 200 OK com token Bearer.

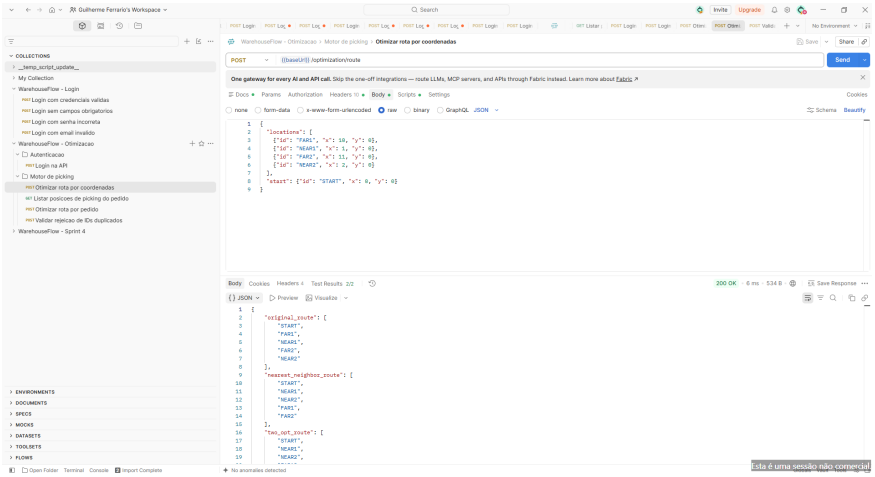


Figura 2 — Otimização por coordenadas: resposta HTTP 200 OK com as rotas original, Nearest Neighbor e 2-opt.

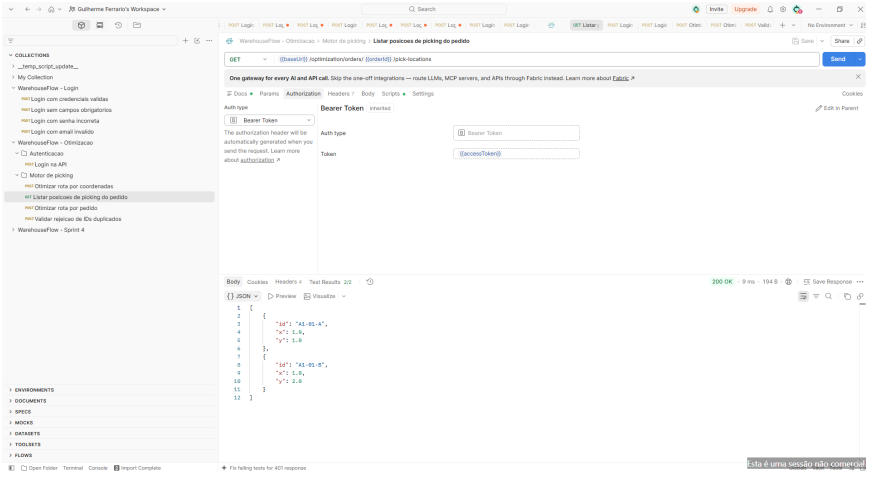


Figura 3 — Consulta de posições de picking de um pedido: resposta HTTP 200 OK.

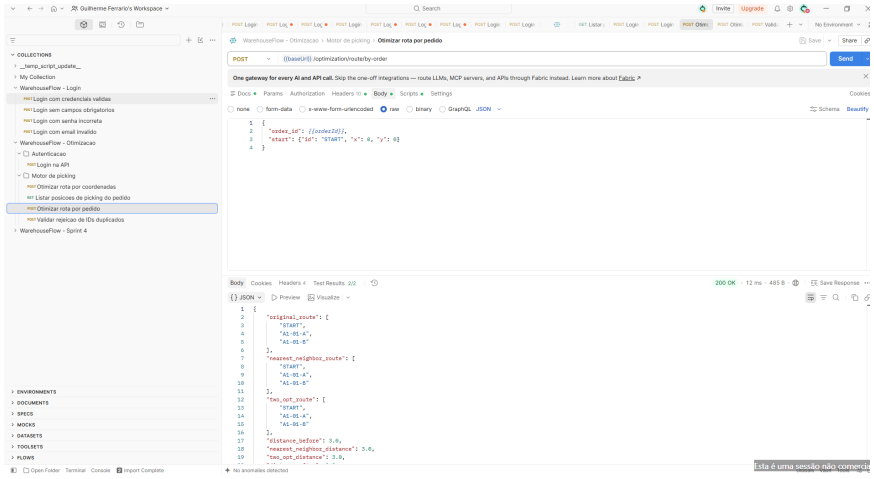


Figura 4 — Otimização por pedido: resposta HTTP 200 OK com rotas e métricas.

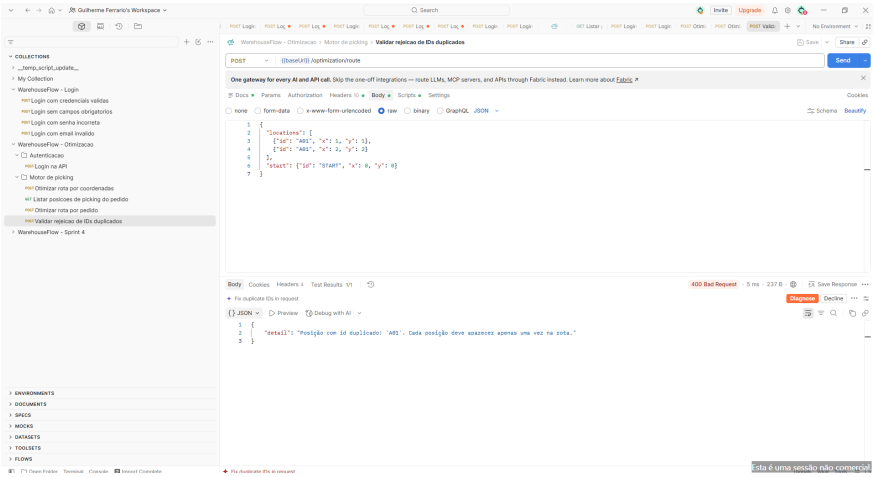


Figura 5 — Rejeição de IDs duplicados: resposta HTTP 400 Bad Request.

6. Resumo dos cenários demonstrados

Funcionalidade	Endpoint	Resposta
Autenticação	POST /auth/login	200 OK
Otimização por coordenadas	POST /optimization/route	200 OK
Consulta de posições de picking	GET /optimization/orders/{orderId}/pick-locations	200 OK
Otimização por pedido	POST /optimization/route/by-order	200 OK
Rejeição de IDs duplicados	POST /optimization/route	400 Bad Request

Esses cenários complementam os registros funcionais anteriores do projeto relacionados a produtos, estoque e localizações.

7. Próximos passos

Como continuidade, recomenda-se conferir o fluxo completo pela interface, garantindo que os dados do pedido sejam encaminhados corretamente ao motor e que a rota retornada seja apresentada ao usuário. Também é importante manter as correções da Pré-Banca documentadas e organizar os commits e evidências no repositório.

8. Conclusão

A Sprint 06 reúne a integração do motor de otimização às operações do WarehouseFlow. As capturas do Postman mostram os fluxos de API relacionados à autenticação, consulta das posições de picking, cálculo de rotas e validação de entradas duplicadas, apoiando a continuidade da integração com o frontend.

9. Referência

Repositório do projeto WarehouseFlow: https://github.com/thiagodemorais87/WarehouseFlow

WAREHOUSEFLOW

DOCUMENTAÇÃO DA SPRINT 07

Sistema quase completo: consolidação, consultas e acompanhamento

Disciplina	Fábrica de Software
Curso / Turma	Ciência da Computação • 8MB
Grupo	26
Data da entrega	24 de outubro de 2026
Repositório	github.com/thiagodemorais87/WarehouseFlow

1. Apresentação

A Sprint 07 tem como foco aproximar o WarehouseFlow da conclusão funcional, consolidando os módulos já desenvolvidos e ampliando os recursos de consulta, análise e rastreabilidade. O sistema reúne funcionalidades de dashboard, produtos, estoque, posições, pedidos, tarefas, usuários e otimização de rotas.

2. Objetivos da Sprint

• Consolidar indicadores do armazém em um dashboard.
• Disponibilizar consultas, pesquisas e filtros integrados aos registros da aplicação.
• Apresentar informações de picking e otimização de rotas.
• Registrar operações relevantes para permitir acompanhamento e rastreabilidade.
• Verificar autorização por perfil e preparar a documentação integrada das sprints.

3. Dashboard

O dashboard apresenta indicadores gerais do armazém, incluindo total de produtos cadastrados, quantidade total em estoque, pedidos pendentes e tarefas em andamento. Também apresenta a distribuição dos pedidos por status e uma tabela de pedidos recentes, facilitando a leitura rápida da situação operacional.

A captura de tela incluída nas evidências mostra quatro produtos cadastrados, 155 unidades em estoque, um pedido pendente e nenhuma tarefa em andamento. Esses valores correspondem ao conjunto de dados visível na tela capturada.

4. Consultas, pesquisa e filtros

A área de pedidos apresenta busca por número e filtros por tipo e status, permitindo restringir a listagem de acordo com os critérios selecionados. A Sprint também contempla pesquisa de usuários por e-mail. Durante a verificação compartilhada, a consulta por e-mail não diferenciou o endereço informado: os resultados apresentados foram iguais para e-mail existente e inexistente, indicando um ponto de correção na lógica de pesquisa e na integração da interface.

5. Relatório de picking e otimização

A tela de otimização permite informar um ponto de partida e posições a visitar. O resultado é apresentado em etapas, comparando a rota original, a sequência calculada pelo algoritmo Nearest Neighbor e a rota após a melhoria 2-opt, além de exibir distâncias, economia percentual, quantidade de localizações processadas e tempo de execução.

Na captura utilizada como exemplo, a distância original e a distância otimizada aparecem como 12 unidades, com redução de 0%. O resultado ilustra a apresentação comparativa; o indicador agregado de redução média deve ser calculado a partir dos resultados persistidos no sistema.

6. Exportação de dados

A exportação CSV do relatório de picking é pertinente ao projeto porque permite consultar os resultados fora da aplicação e reutilizar os dados em planilhas. A evidência fornecida para esta etapa indica que a listagem de pedidos ainda não apresenta um recurso de exportação visível. A implementação esperada é uma ação de exportação para o relatório de picking, incluindo rotas, distâncias e tempo de execução.

7. Histórico de operações e autenticação

O histórico deve registrar, no mínimo, logins bem-sucedidos e malsucedidos, alterações de usuários e otimizações realizadas. Para cada otimização, o registro deve permitir identificar o responsável, a data/hora e o resultado produzido. A verificação compartilhada da falha de login não apresentou registro de histórico, portanto esse fluxo

precisa ser integrado ao mecanismo de auditoria para atender ao requisito da Sprint.

8. Controle de acesso por perfil

A regra definida para o sistema é: ADMIN tem acesso administrativo completo; GESTOR consulta indicadores e operações conforme suas permissões; OPERADOR acessa apenas os dados e as tarefas que lhe pertencem ou que estejam autorizados para seu perfil.

Na execução de autorização registrada em 23/10/2026, ADMIN obteve os resultados esperados ao listar e criar usuários, enquanto GESTOR e OPERADOR foram bloqueados na criação de usuários. Entretanto, também foram identificados problemas de isolamento de tarefas: o OPERADOR recebeu tarefas de outro usuário e conseguiu consultar ou alterar a atribuição de tarefa, quando o esperado era bloquear essas ações. Esses casos devem ser corrigidos antes da entrega final.

9. Integração com os módulos anteriores

As evidências mostram a navegação integrada entre dashboard, pedidos, usuários e otimização. O fluxo funcional previsto é: autenticação → consulta de dados do armazém → pesquisa/filtro → execução da otimização → consulta do relatório → exportação → registro no histórico. A verificação final deve assegurar que cada etapa utilize dados da aplicação e preserve as permissões do usuário autenticado.

10. Resumo das verificações e pontos de correção

Área	Resultado observado	Próxima ação
Dashboard	Indicadores e pedidos recentes exibidos.	Conectar/confirmar indicadores agregados, incluindo redução média salva.
Pedidos e filtros	Listagem e filtros por tipo/status visíveis.	Validar combinações de filtros e busca por número.
Pesquisa de usuários	E-mail existente e inexistente retornaram a mesma quantidade.	Implementar filtro real por e-mail e validar resultados.
Relatório de picking	Tela de otimização apresenta rotas, distâncias e tempo.	Disponibilizar relatório consultável e exportável.
CSV	Exportação não aparece na tela evidenciada.	Implementar e testar geração e conteúdo do arquivo.
Histórico	Falha de login sem registro visível.	Registrar sucessos/falhas, mudanças de usuário e otimizações.
Permissões	Criação de usuário bloqueada para GESTOR/OPERADOR; isolamento de tarefas falhou.	Corrigir autorização e repetir os testes de acesso.

11. Dificuldades encontradas

As principais dificuldades observadas foram a pesquisa de usuários por e-mail ainda sem filtragem efetiva, a ausência de registro de tentativas de login no histórico, a necessidade de disponibilizar a exportação CSV e falhas de autorização no isolamento das tarefas do perfil OPERADOR. Também é necessário garantir que os indicadores agregados utilizem resultados de otimização salvos, em vez de valores fixos ou calculados apenas na interface.

12. Próximos passos

1. Corrigir a pesquisa de usuários por e-mail.
2. Implementar a exportação CSV do relatório de picking e validar o conteúdo gerado.
3. Registrar logins bem-sucedidos e malsucedidos, alterações de usuários e resultados de otimização.
4. Corrigir o isolamento de tarefas e revisar as permissões de ADMIN, GESTOR e OPERADOR.
5. Calcular a redução média de distância com base nos resultados persistidos.
6. Executar novamente os testes dos endpoints de dashboard, filtros, exportação e histórico; guardar as respostas e capturas atualizadas.
7. Atualizar o repositório e consolidar este relatório com as documentações das Sprints 01 a 06.

13. Repositório

GitHub: https://github.com/thiagodemorais87/WarehouseFlow

14. Conclusão

O WarehouseFlow já apresenta telas operacionais para consulta do estado do armazém, listagem e filtragem de pedidos e otimização de rotas. As verificações da Sprint 07 também identificaram ajustes importantes para concluir a entrega: pesquisa efetiva por e-mail, exportação CSV, persistência do histórico e isolamento correto dos dados por perfil. A finalização desses pontos, seguida da repetição dos testes e atualização do repositório, permitirá uma demonstração mais consistente do sistema integrado.

ANEXO A — Evidências visuais da Sprint 07

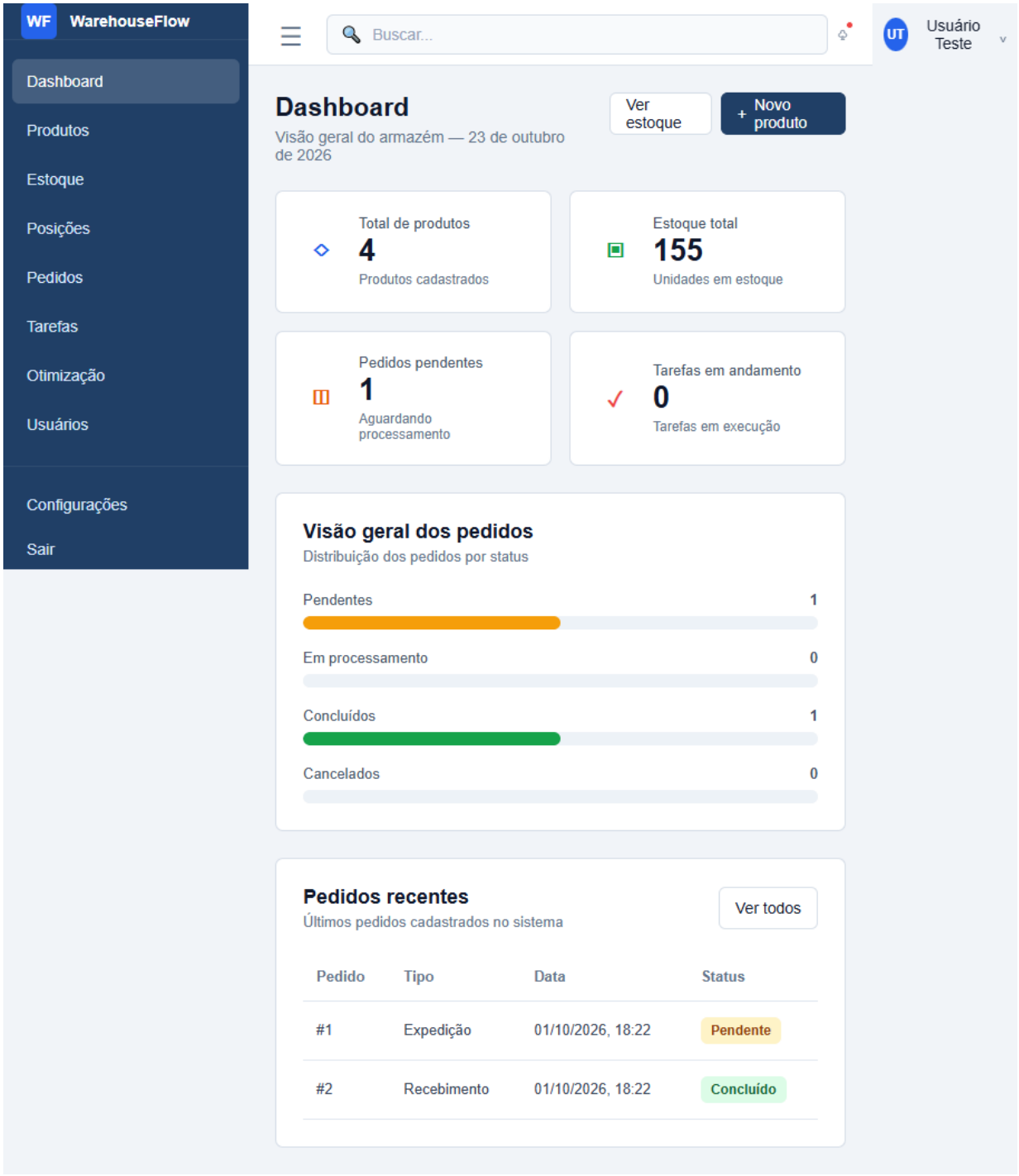


Figura 1 — Dashboard com indicadores de produtos, estoque, pedidos e tarefas.



Figura 2 — Tela de pedidos com filtro por tipo de pedido.



Figura 3 — Listagem de pedidos e filtros disponíveis.

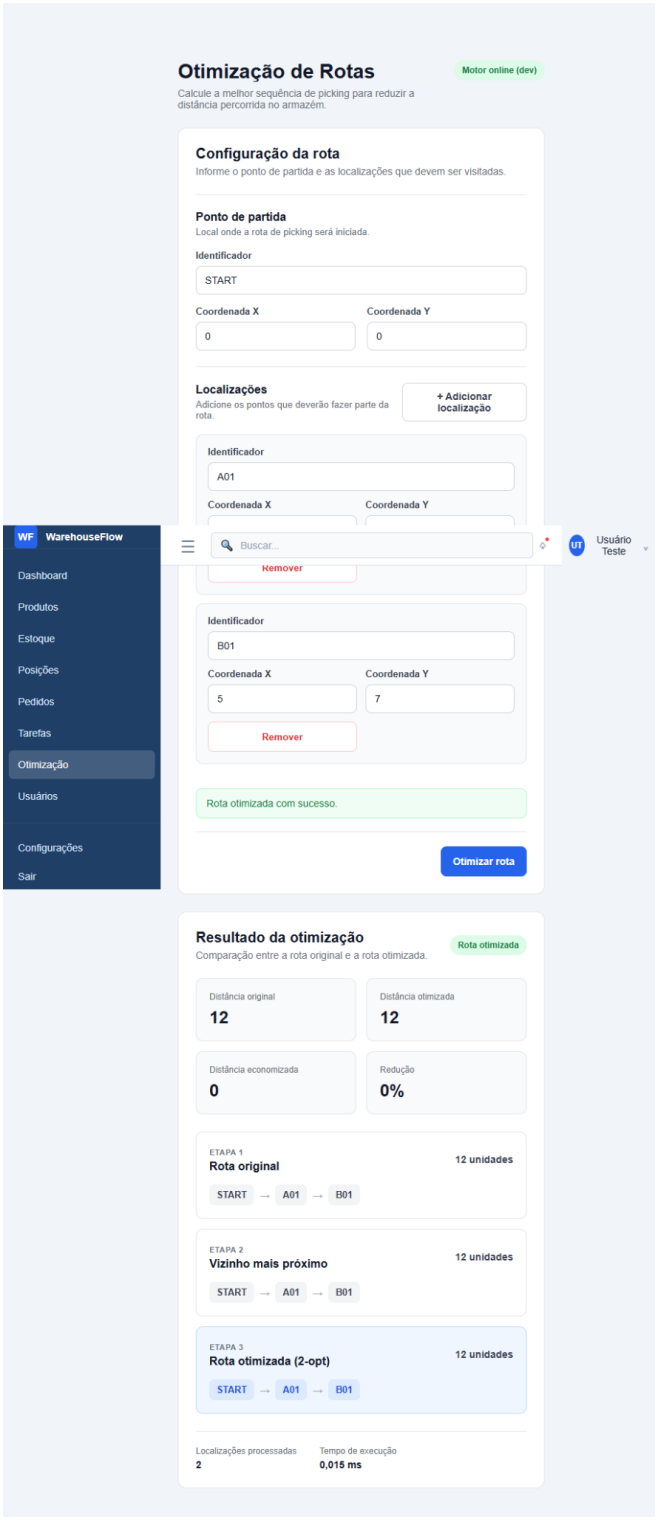


Figura 4 — Tela de otimização de rotas e comparação entre rota original, vizinho mais próximo e 2-opt.

WarehouseFlow · Evidência de testes de autorização

Execução automatizada via API da aplicação, 23/10/2026. Os usuários e a tarefa de QA foram removidos após os testes.

Perfil	Listar usuários	Criar usuário
ADMIN	**200 — esperado**	**201 — esperado**
GESTOR	**200 — permitido**	**403 — bloqueado**
OPERADOR	**200 — permitido**	**403 — bloqueado**

Falhas encontradas · isolamento de tarefas do OPERADOR

Verificação	Resultado real	Esperado	Status
Lista de tarefas (2 existentes)	OPERADOR recebeu 2 tarefas, incluindo tarefa alheia	Somente tarefa atribuída ao operador	**FALHOU**
Consultar tarefa de outro usuário	HTTP 200	HTTP 403	**FALHOU**
Alterar responsável por tarefa	HTTP 200	HTTP 403	**FALHOU**

Busca de usuário por e-mail · API

Consulta para e-mail existente: 8 registros. Para e-mail inexistente: 8 registros. A query de e-mail é ignorada; não há busca por e-mail implementada.

Figura 5 — Resultados registrados para autorização por perfil e pesquisa de usuários por e-mail.



Figura 6 — Resposta da consulta ao relatório de picking durante a verificação.

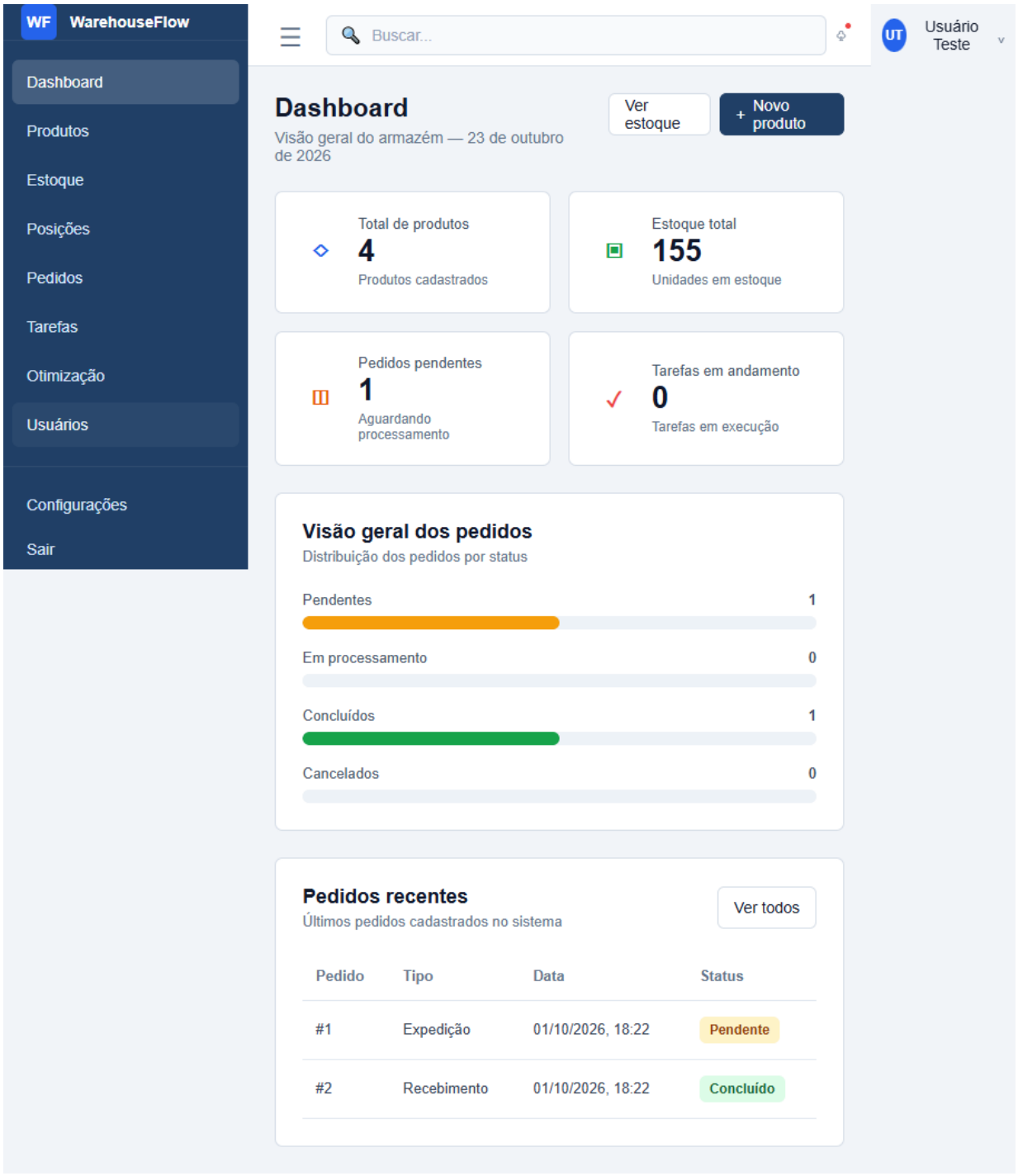


Figura 7 — Tela de usuários observada durante a verificação da funcionalidade de pesquisa.



Figura 8 — Verificação de login com falha e consulta do histórico de operações.

GRUPO 26 — WAREHOUSEFLOW

Documentação complementar — Sprint 08

Execução da aplicação, cenários de teste e evidências visuais fornecidas

Turma 8MB • Ciência da Computação

Repositório: https://github.com/thiagodemorais87/WarehouseFlow

1. Objetivo e escopo

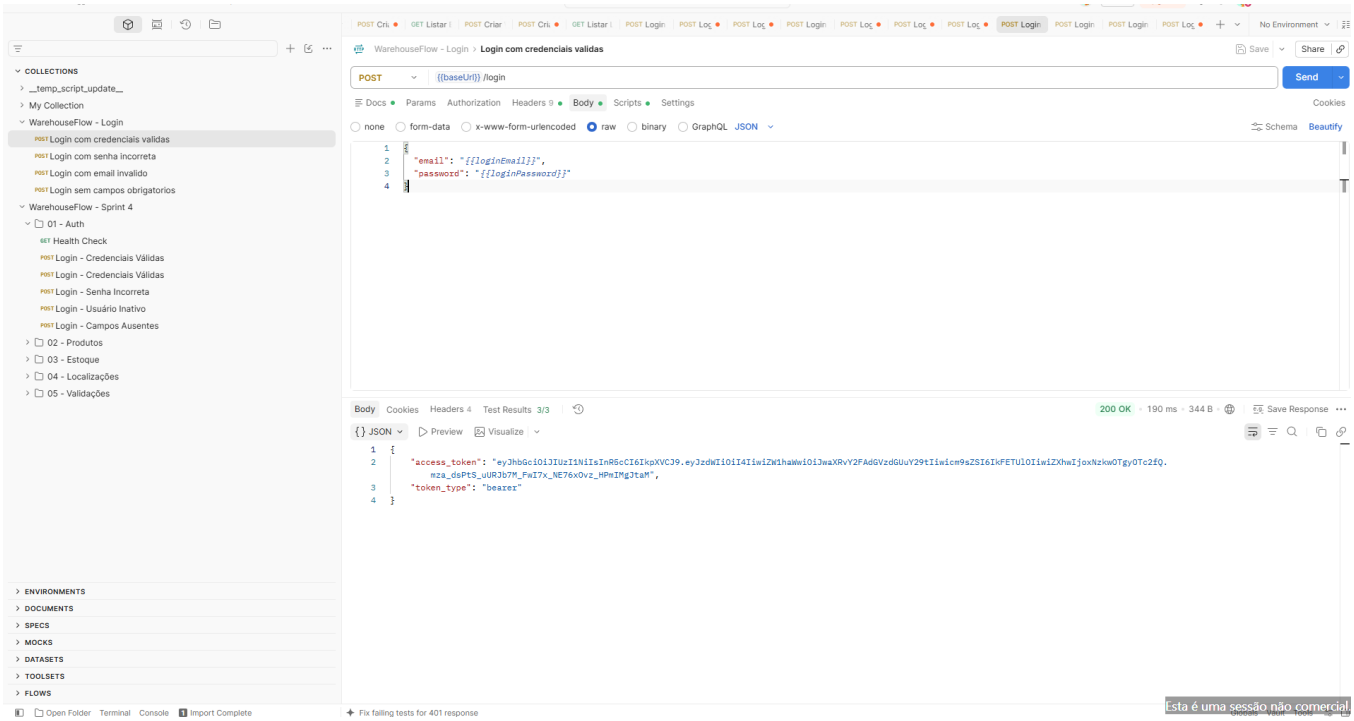
Esta seção complementa a documentação das Sprints 01 a 07 com as evidências disponibilizadas para a Sprint 08. O material recebido contém capturas de requisições e respostas no cliente de API para autenticação, operações de produtos, estoque e localizações. A documentação descreve apenas o que pode ser identificado nas evidências; não presume que funcionalidades sem captura tenham sido testadas.

2. Cenários e evidências recebidas

As imagens abaixo são reproduzidas como recebidas, mantendo sua proporção. Os resultados devem ser interpretados em conjunto com o status e a resposta visíveis em cada captura.

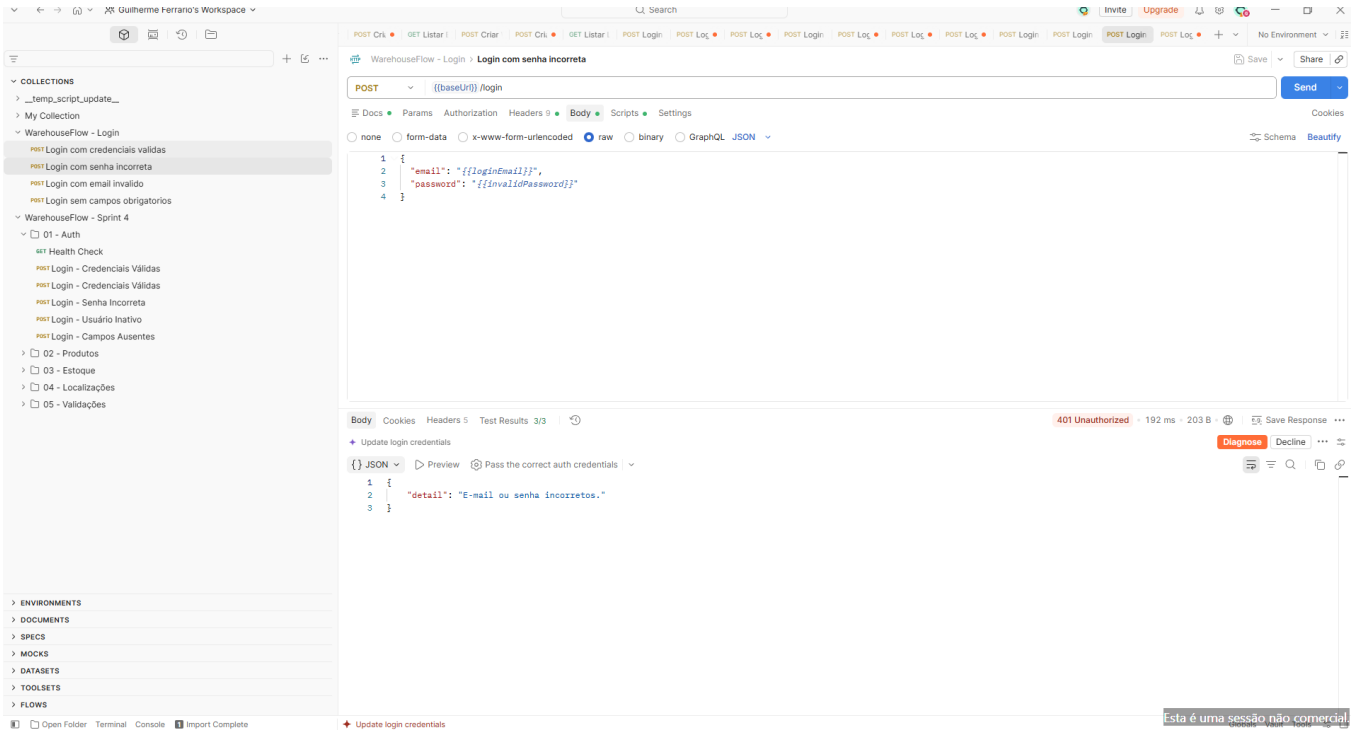
2.1. Login — autenticação válida

Evidência de requisição de login com credenciais válidas. Conferir na imagem o código HTTP e o conteúdo da resposta.



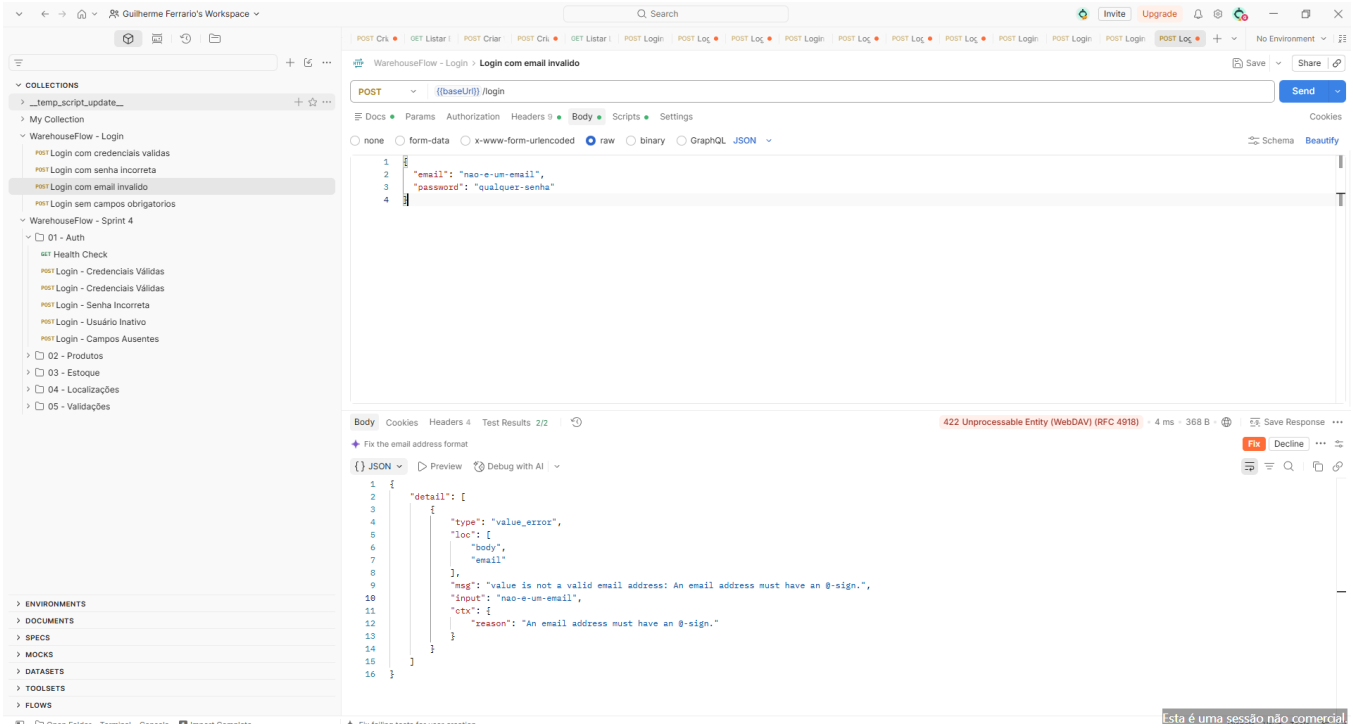
2.2. Login — senha incorreta

Cenário negativo de autenticação com senha incorreta.



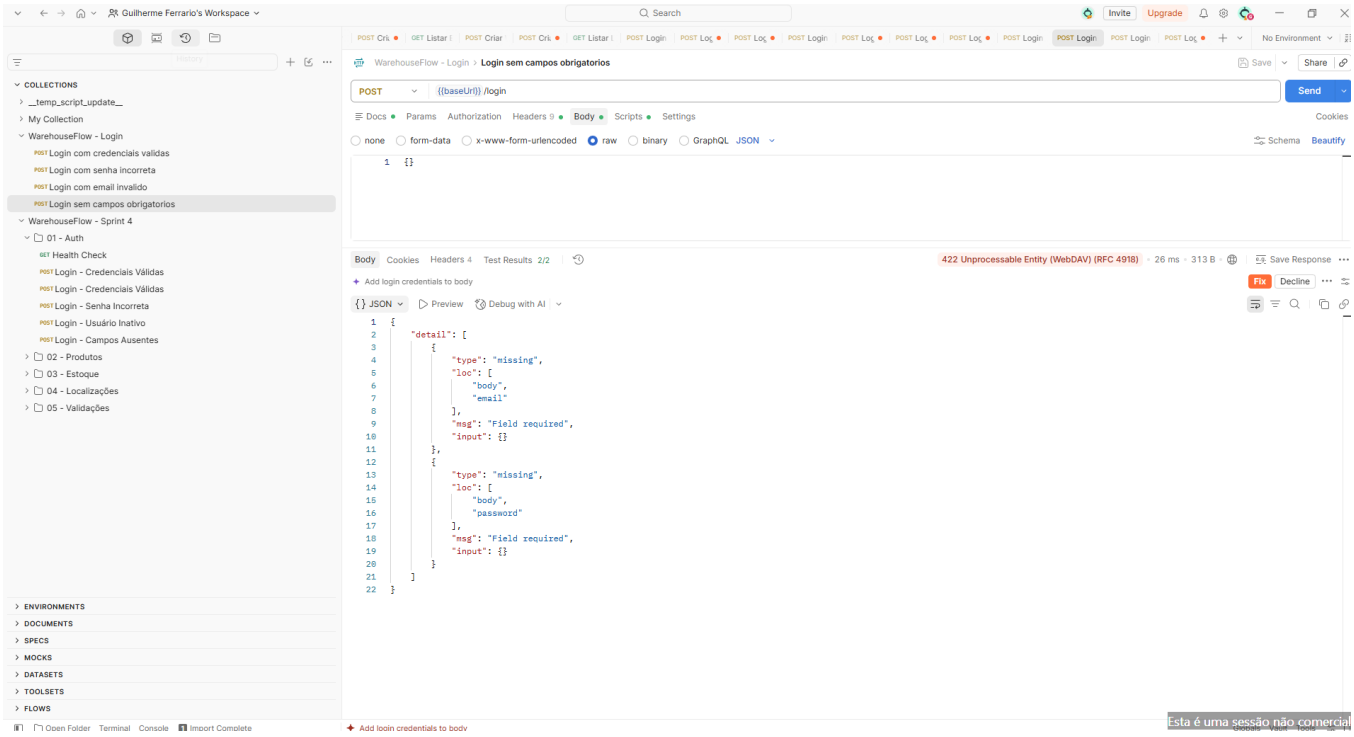
2.3. Login — formato de e-mail inválido

Cenário de validação com endereço de e-mail em formato inválido.



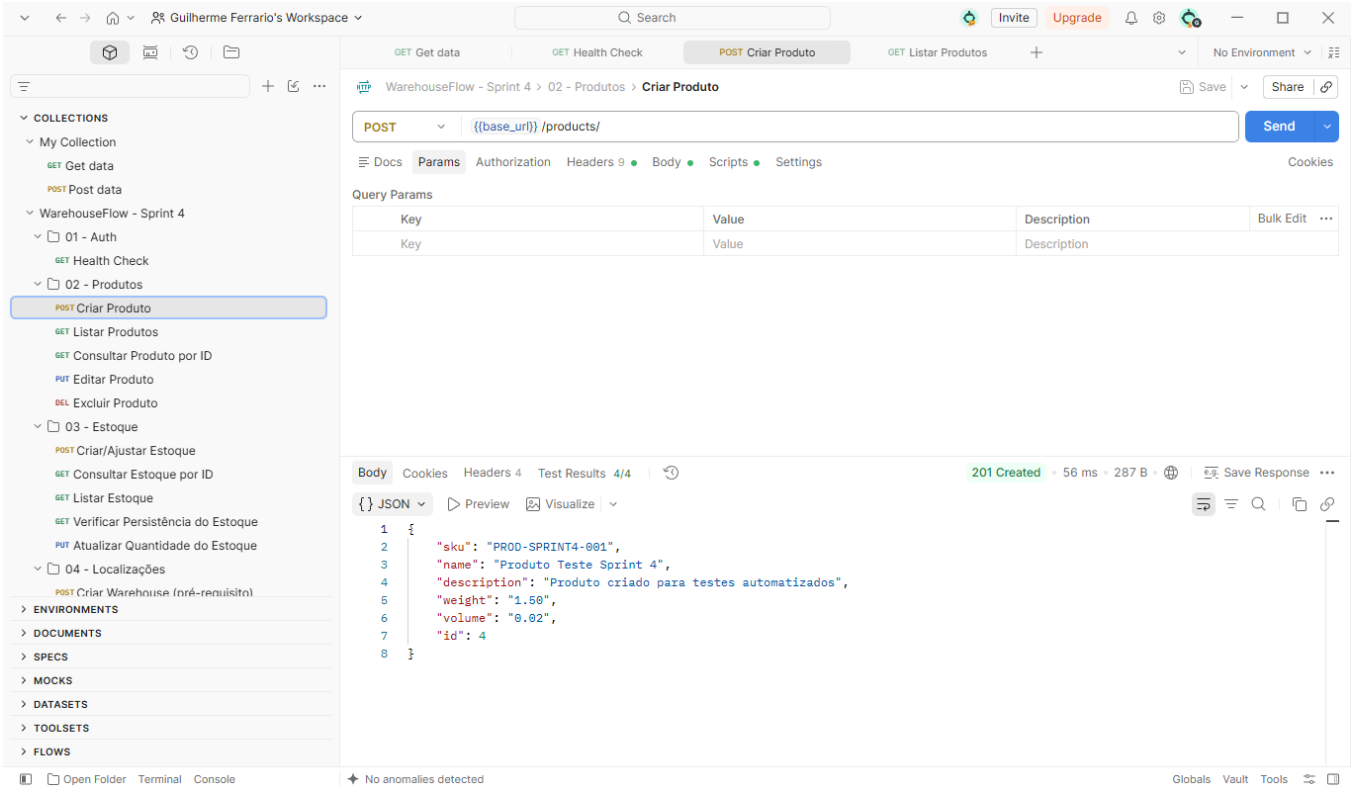
2.4. Login — campos obrigatórios ausentes

Cenário de envio sem preencher os campos obrigatórios.



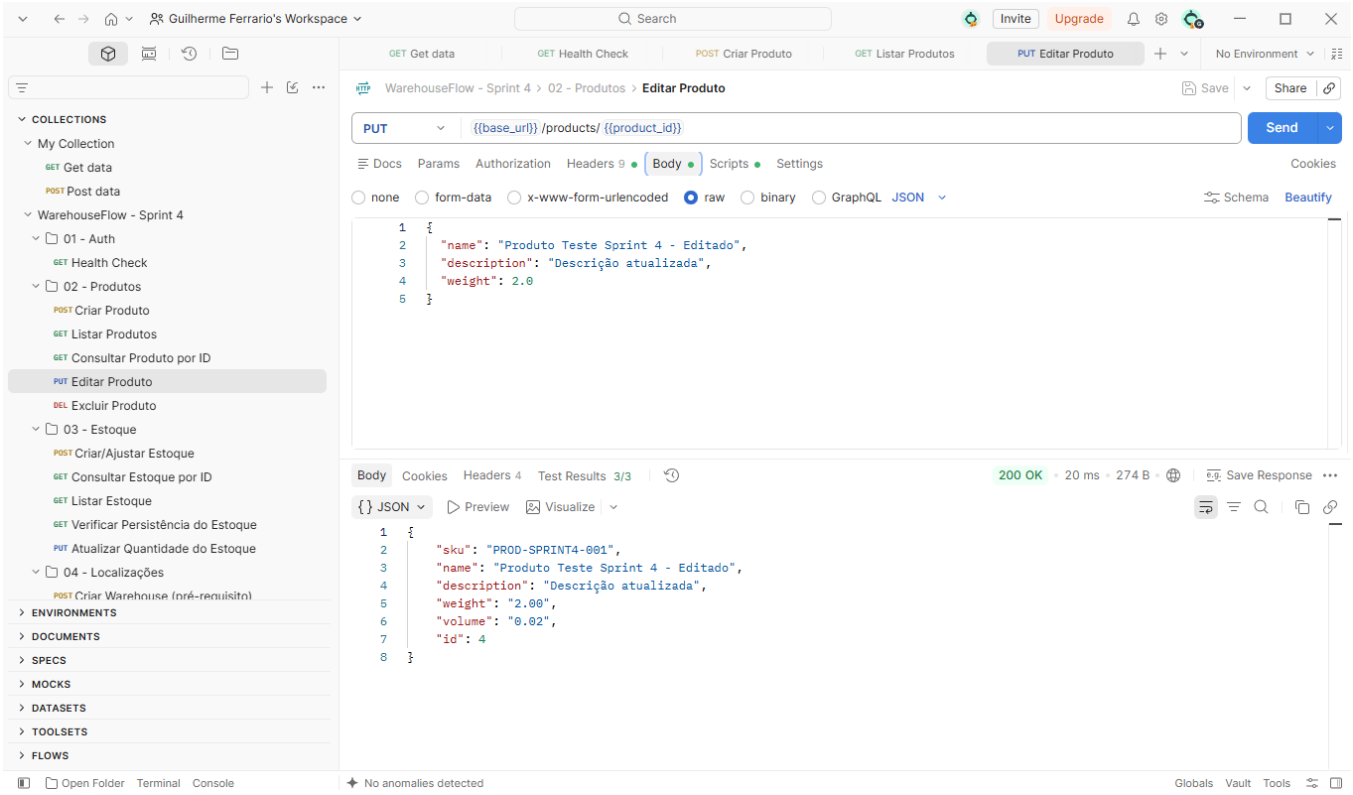
2.5. Produtos — cadastro

Requisição de criação de produto com dados válidos.



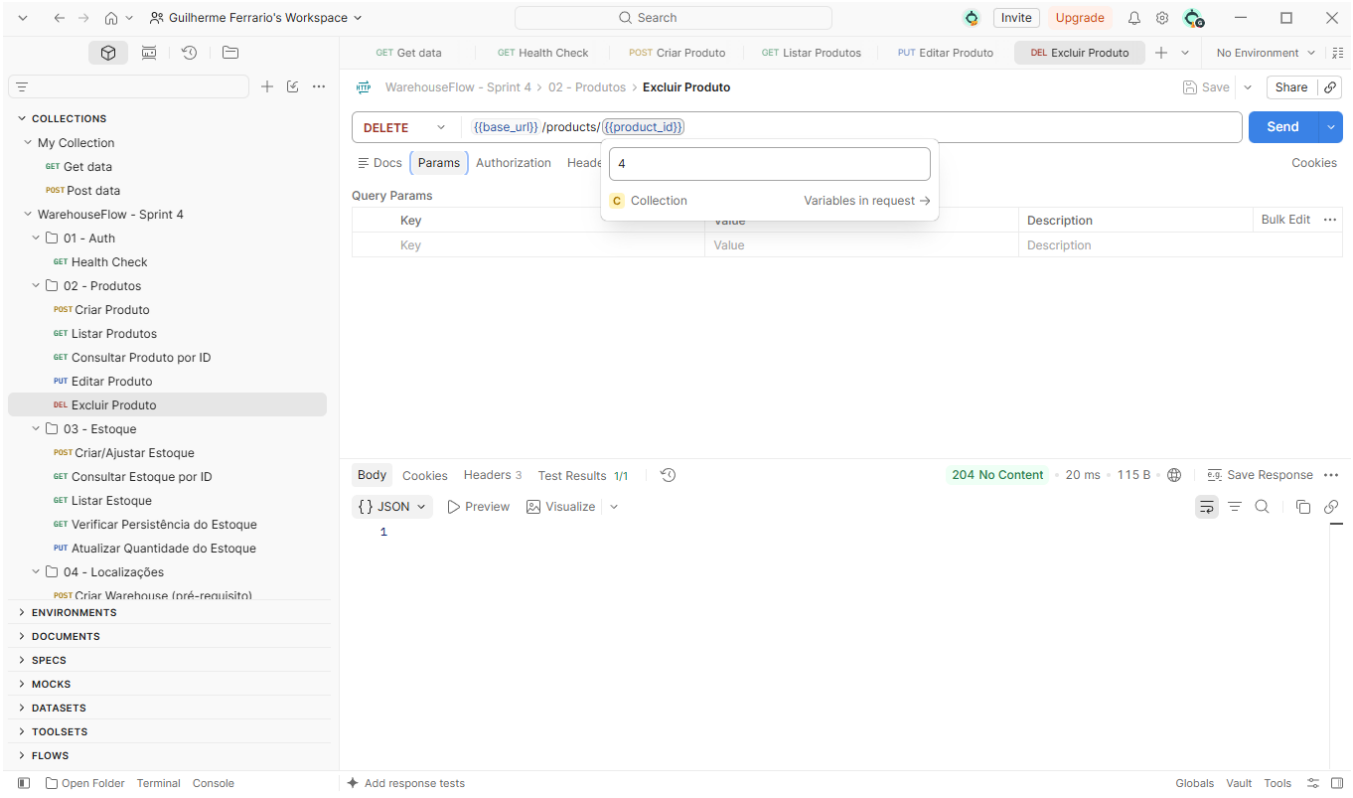
2.6. Produtos — edição

Requisição de edição de produto.



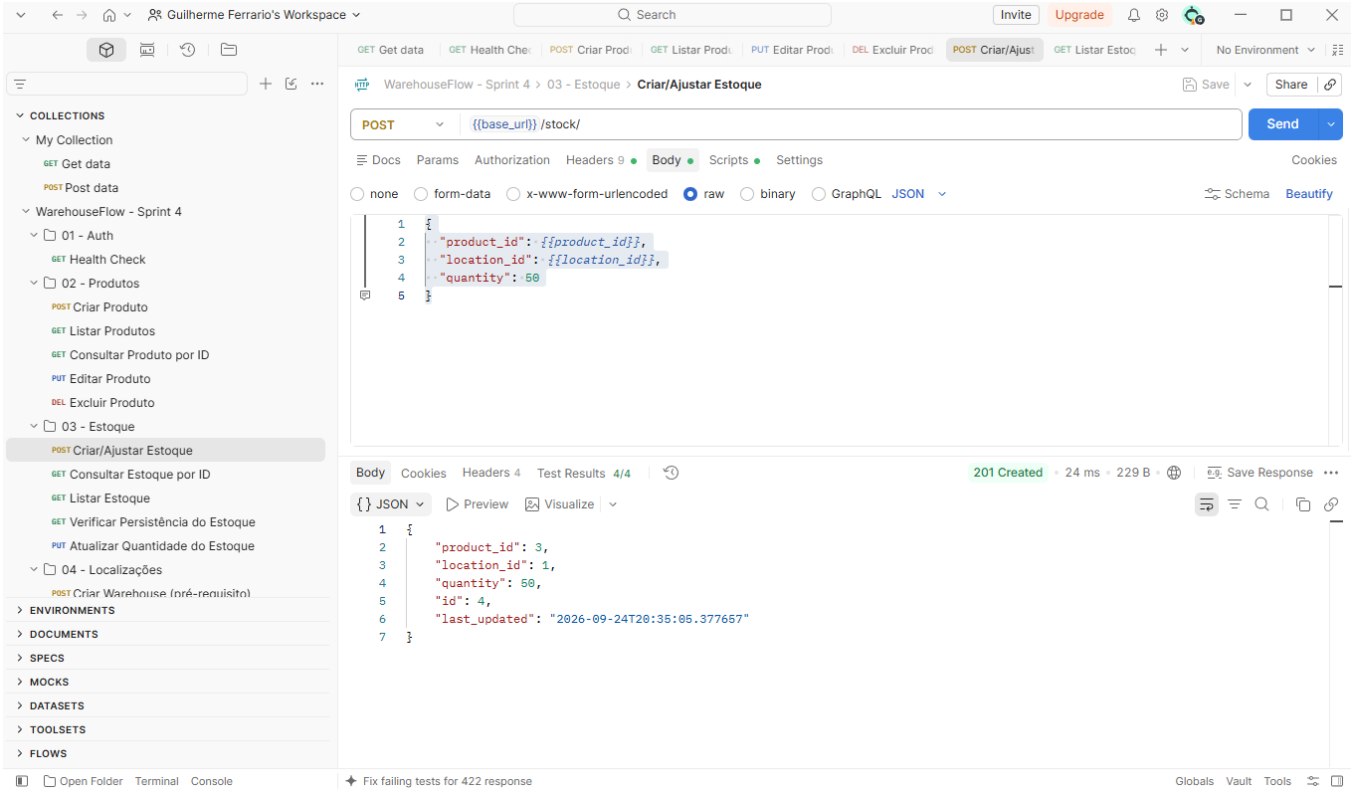
2.7. Produtos — exclusão

Requisição de exclusão de produto.



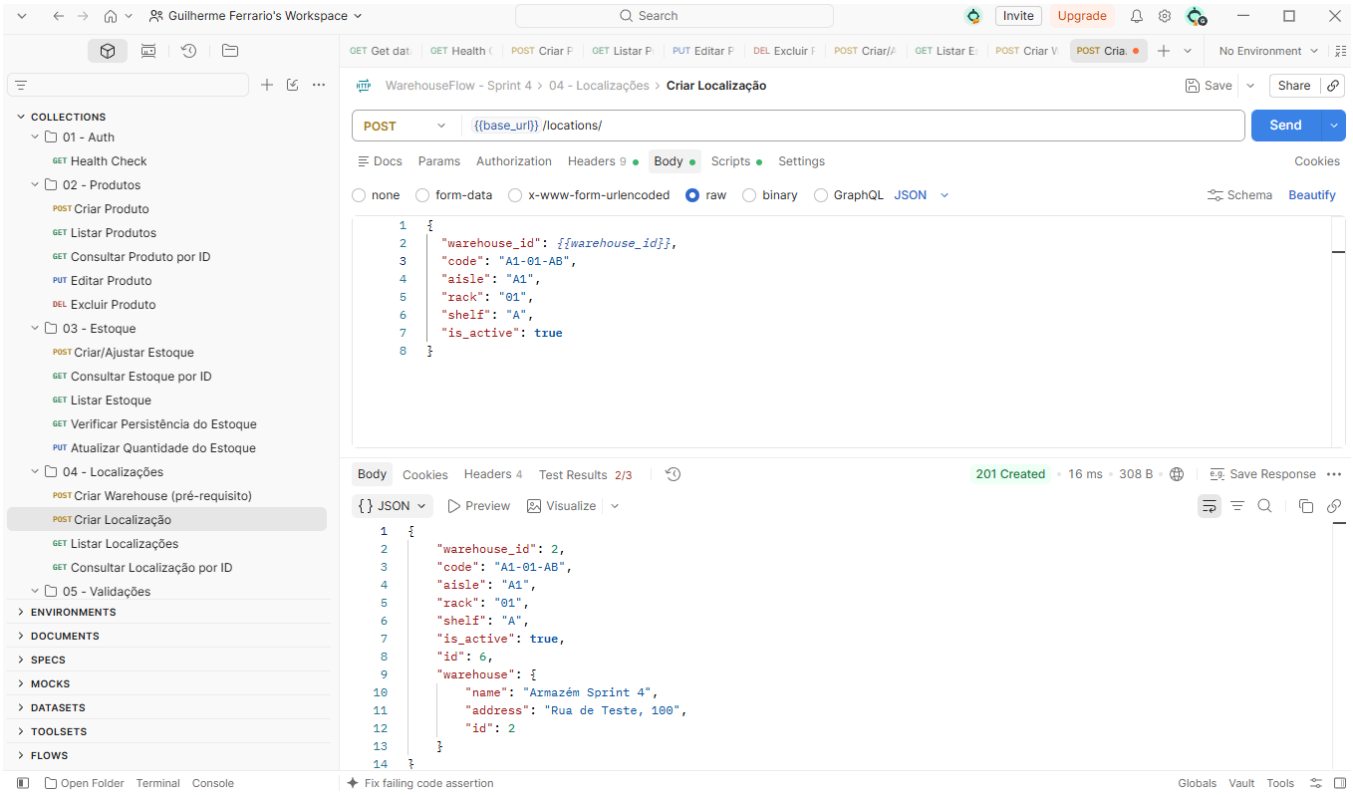
2.8. Estoque — criação de estoque

Requisição de criação de registro de estoque para produto.



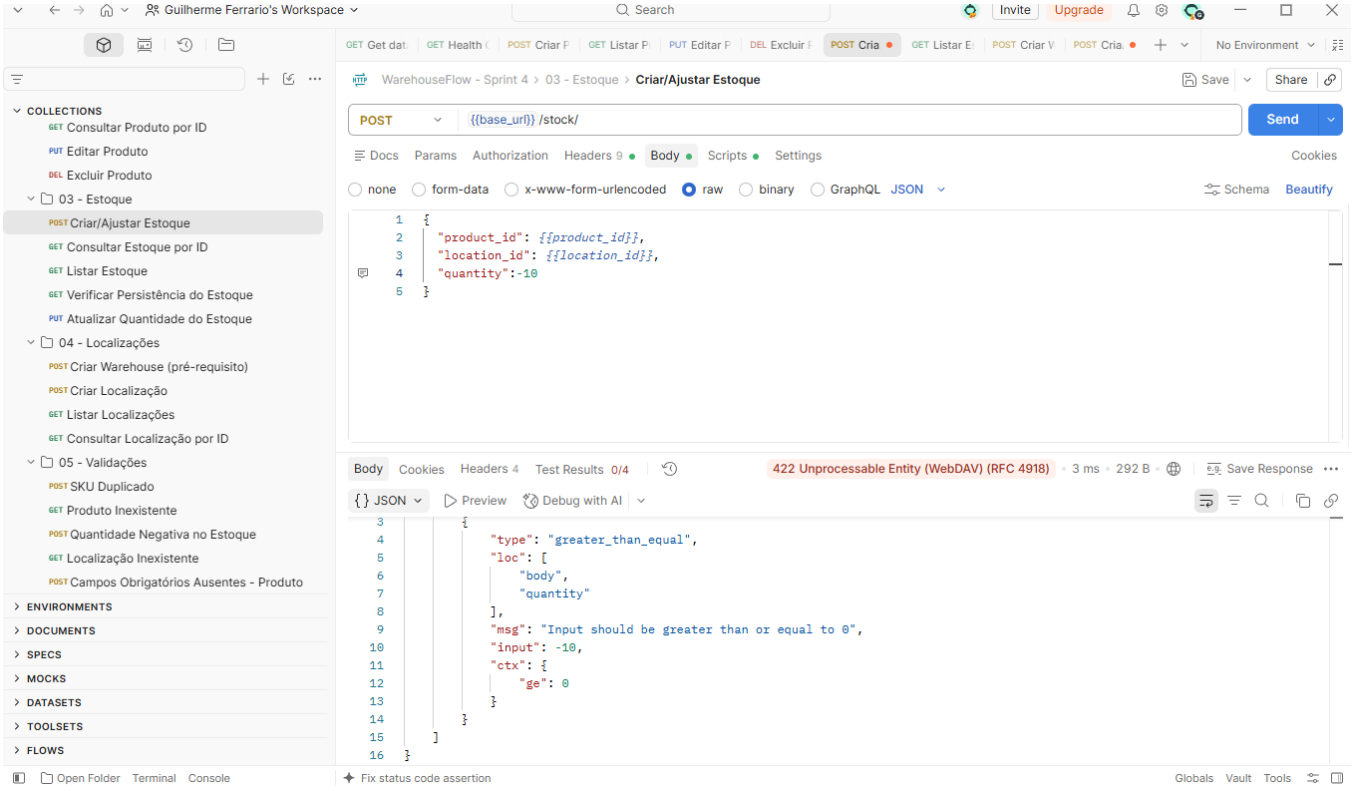
2.9. Localizações — criação

Requisição de criação de localização.



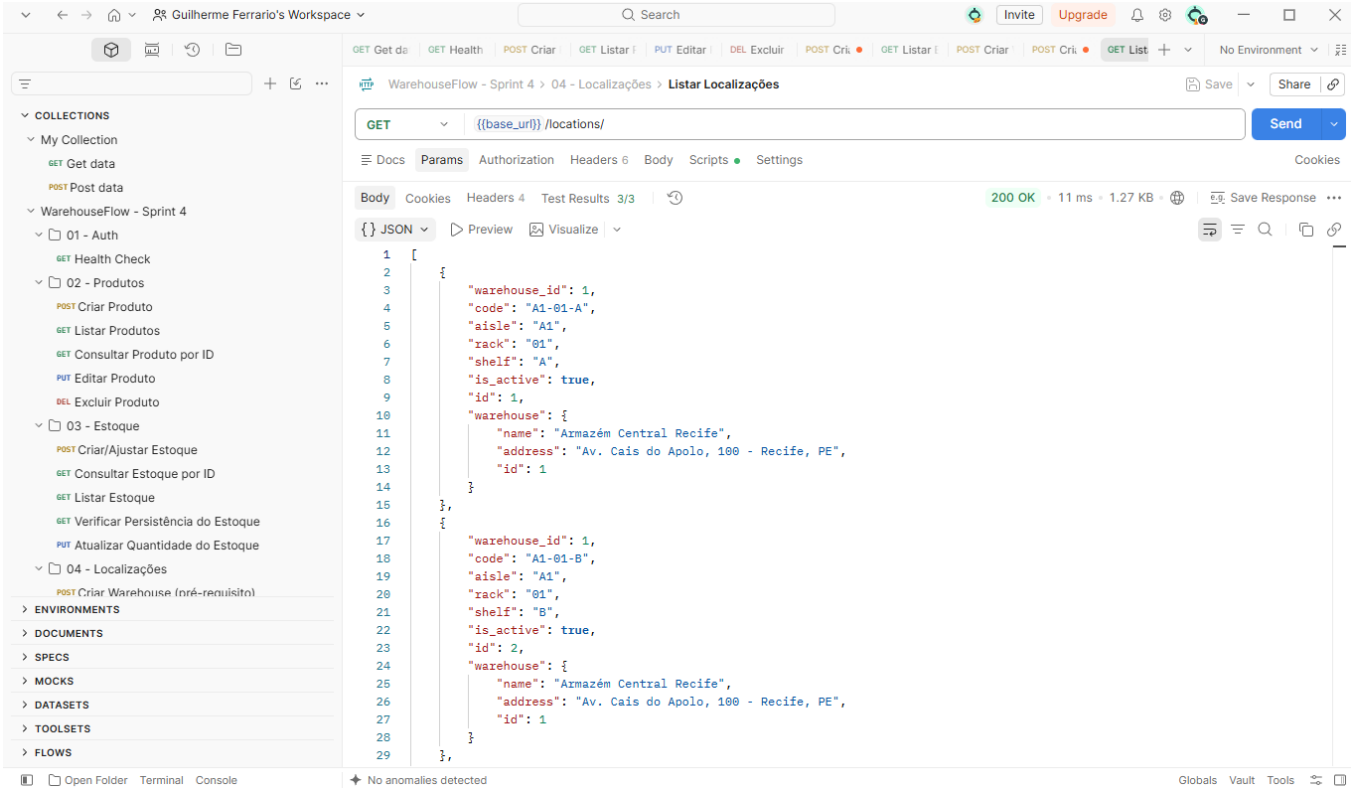
2.10. Estoque — quantidade negativa

Cenário negativo relacionado a estoque com quantidade negativa.



2.11. Localizações — listagem

Requisição de consulta/listagem de localizações.



3. Síntese e limitações das evidências

As capturas fornecidas cobrem cenários de autenticação, operações de cadastro/edição/exclusão de produtos, estoque e localizações. A confirmação do resultado de cada teste depende da leitura do status HTTP e do corpo da resposta presente na captura; não se atribui aprovação automática a uma operação sem essa verificação.

Não foi possível confirmar, apenas a partir do conjunto de imagens nomeadas acima, evidência específica de execução da otimização de picking ou de uma tentativa de acesso negado por perfil. Esses cenários devem ser acrescentados somente se houver capturas ou registros reais correspondentes. A evidência de quantidade negativa registra um teste negativo de estoque, mas não substitui o teste de controle de acesso.

4. Próximos passos recomendados

• Conferir os códigos HTTP e as respostas de cada captura e registrar o status final de cada teste.
• Acrescentar evidências de otimização de picking e de permissão negada caso tenham sido executadas e estejam disponíveis.
• Validar a aplicação e as funcionalidades em ambiente local antes da entrega final.
• Integrar esta seção ao documento consolidado e conferir a presença das Sprints 01 a 08, sem duplicações desnecessárias.

Repositório do projeto: https://github.com/thiagodemorais87/WarehouseFlow

SPRINT 09 — EXECUÇÃO DE TESTES

Grupo 26 • WarehouseFlow • Turma 8MB

1. Escopo e evidências recebidas

A documentação desta sprint foi elaborada a partir do pacote report.rar enviado, contendo screenshots, logs, código alterado, testes automatizados e arquivos de regressão. Os cenários identificados vão de S9-T01 a S9-T20.

2. Resultado dos testes automatizados

Resultado reportado pelo pytest: 92 passed, 1 skipped, 1 warning, em 14,20 segundos. O aviso registrado é de depreciação do uso de httpx com Starlette TestClient; não foi contabilizado como falha.

3. Regressão da exclusão de produto com dependências (S9-T18)

Antes da correção: 1 teste falhou (test_excluir_produto_com_dependencias_devolve_409), com IntegrityError ao tentar definir product_id como NULL em order_items. Após a correção: 1 teste passou. Os arquivos de regressão antes/depois foram fornecidos como evidência.

4. Validações de API (S9-T20)

• GET /products/ sem credencial: HTTP 401.
• POST /orders/ com quantity=0: HTTP 400, com mensagem de validação indicando que quantity deve ser maior que zero.
• POST /optimization/route com IDs de localização duplicados: HTTP 400, informando que cada posição deve aparecer apenas uma vez.

5. Cenários documentados

S9-T01 health/login; T02 e-mail inválido; T03 credenciais inválidas; T04 login válido/dashboard; T05 campos obrigatórios de produto; T06 peso negativo; T07 produto criado; T08 SKU duplicado; T09 produto atualizado e persistido; T10 quantidade zero em pedido; T11 pedido outbound criado; T12 filtro de pedidos/expedição; T13 tarefa de picking criada; T14 otimização por pedido; T15 histórico de picking; T16 navegação do relatório de pedidos; T17 logout/redirecionamento; T18 exclusão com dependências (409) e regressão; T19 exclusão sem vínculos; T20 validações de API.

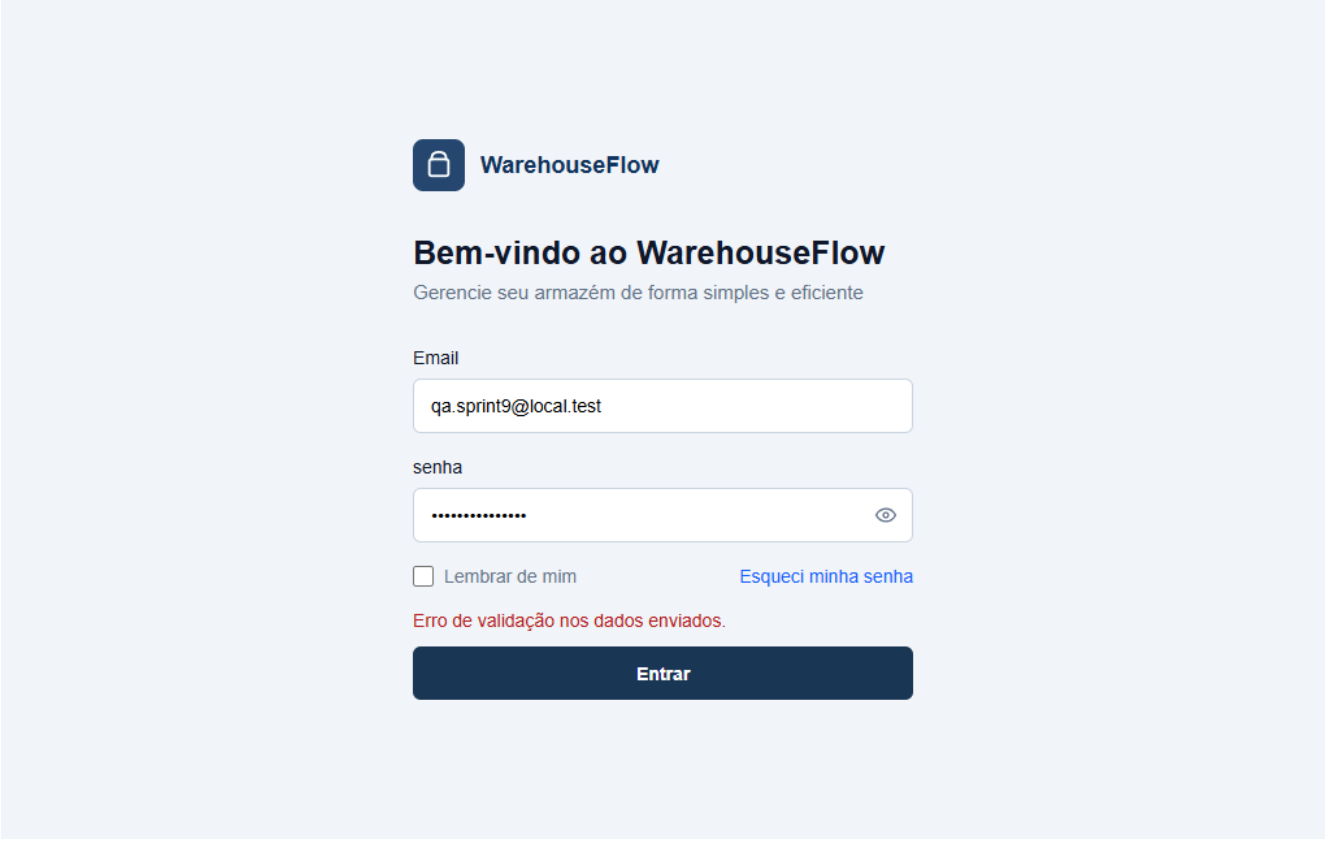
6. Evidências visuais

As capturas abaixo foram anexadas ao pacote de execução. Cada imagem mantém o nome original correspondente ao cenário.

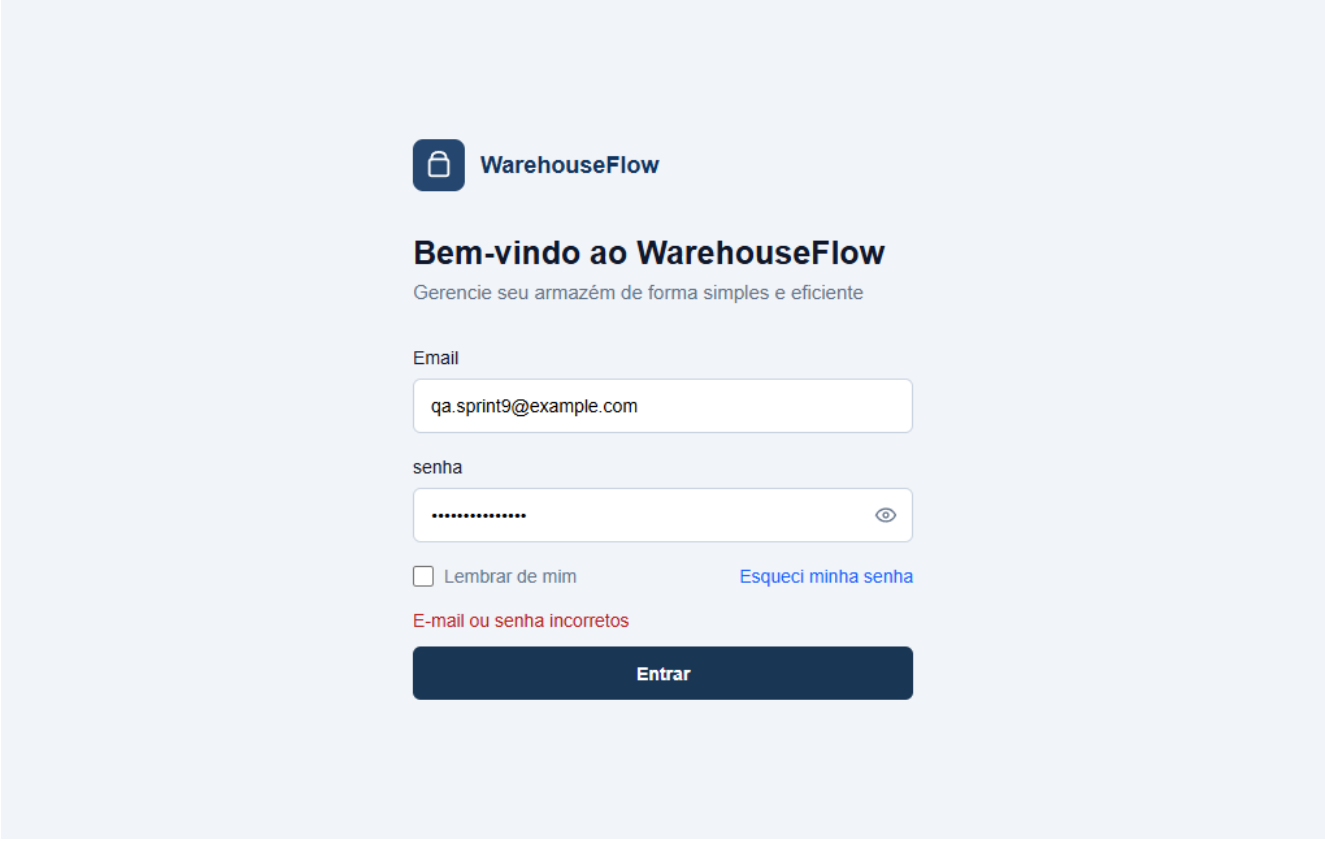
S9-T01-health-e-login.png



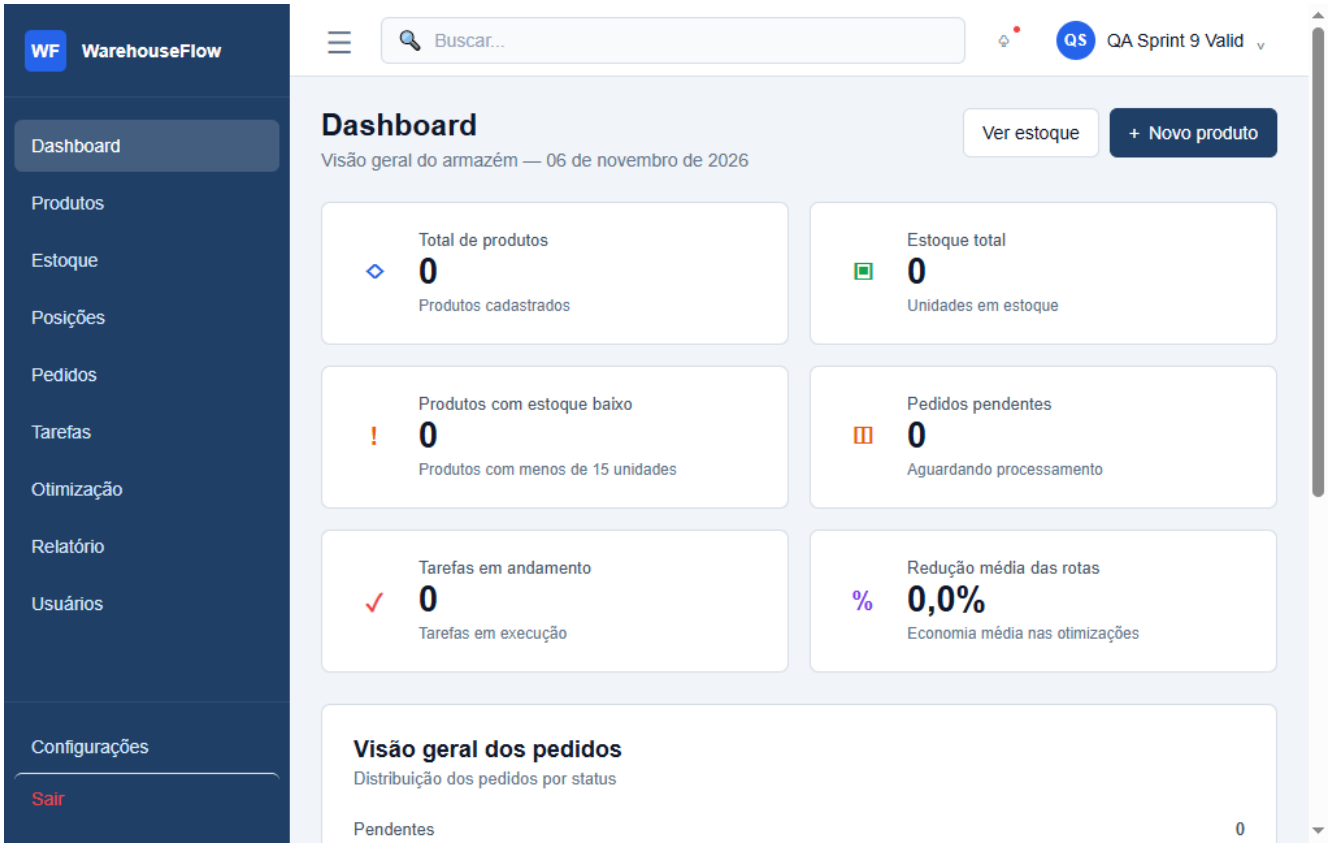
S9-T02-email-invalido.png



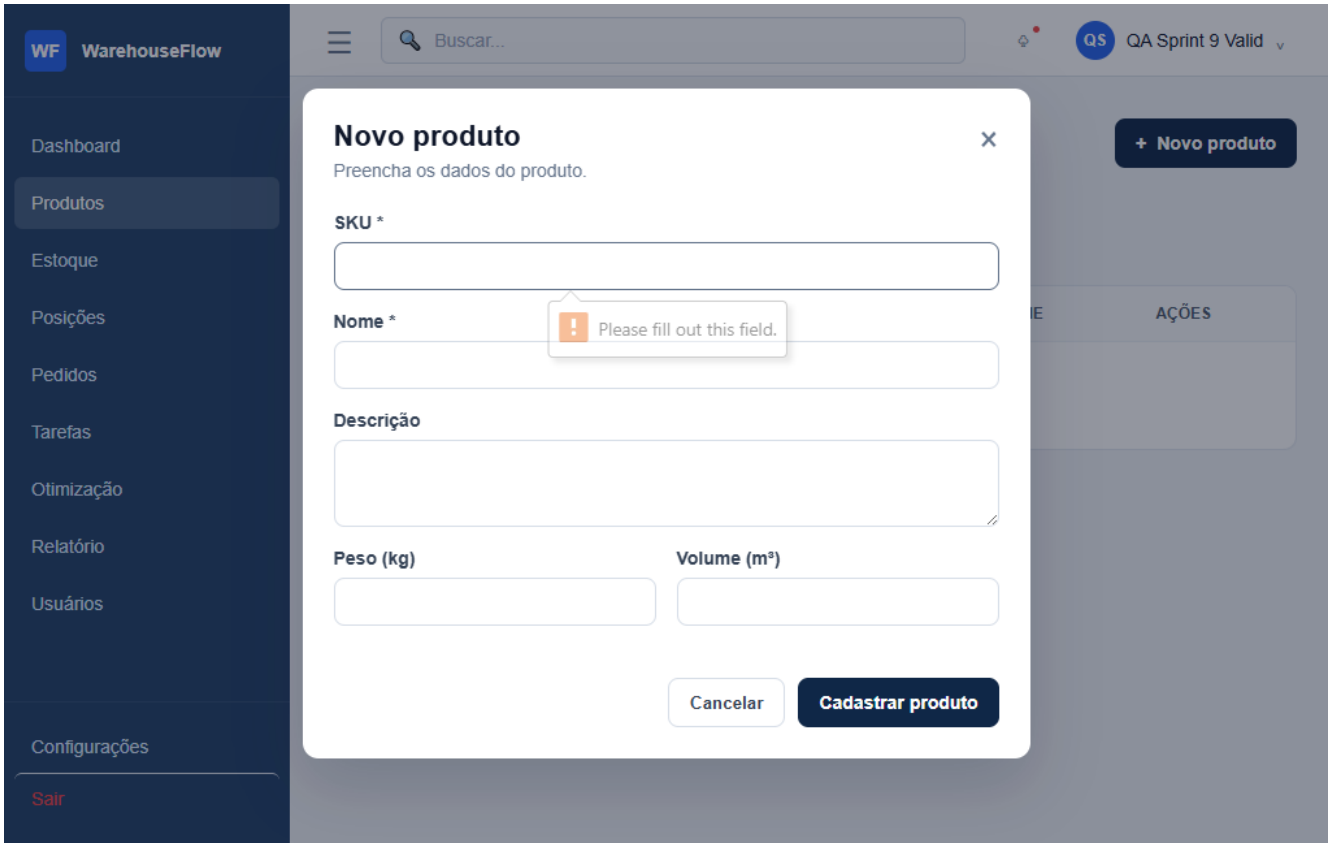
S9-T03-login-credenciais-invalidas.png



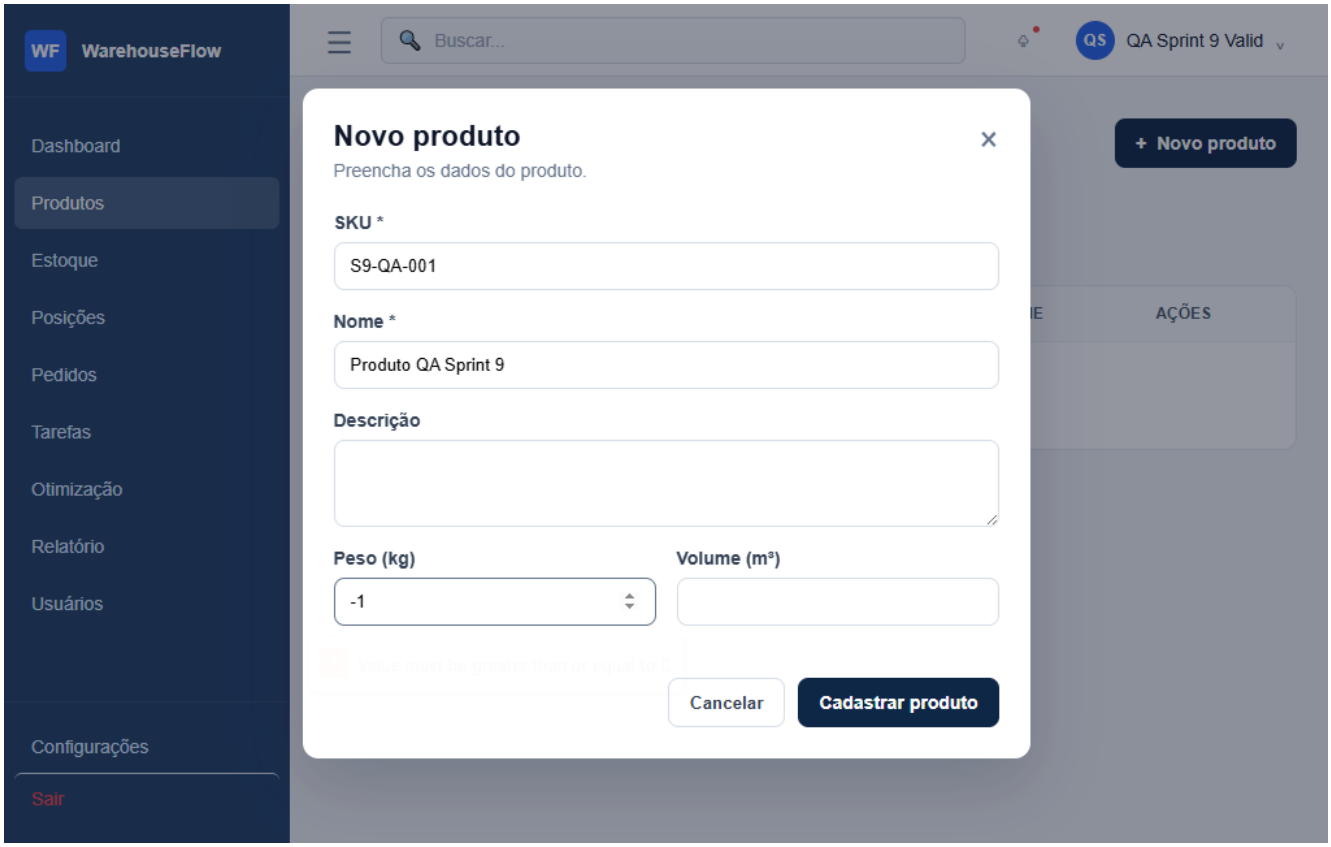
S9-T04-login-sucesso-dashboard.png



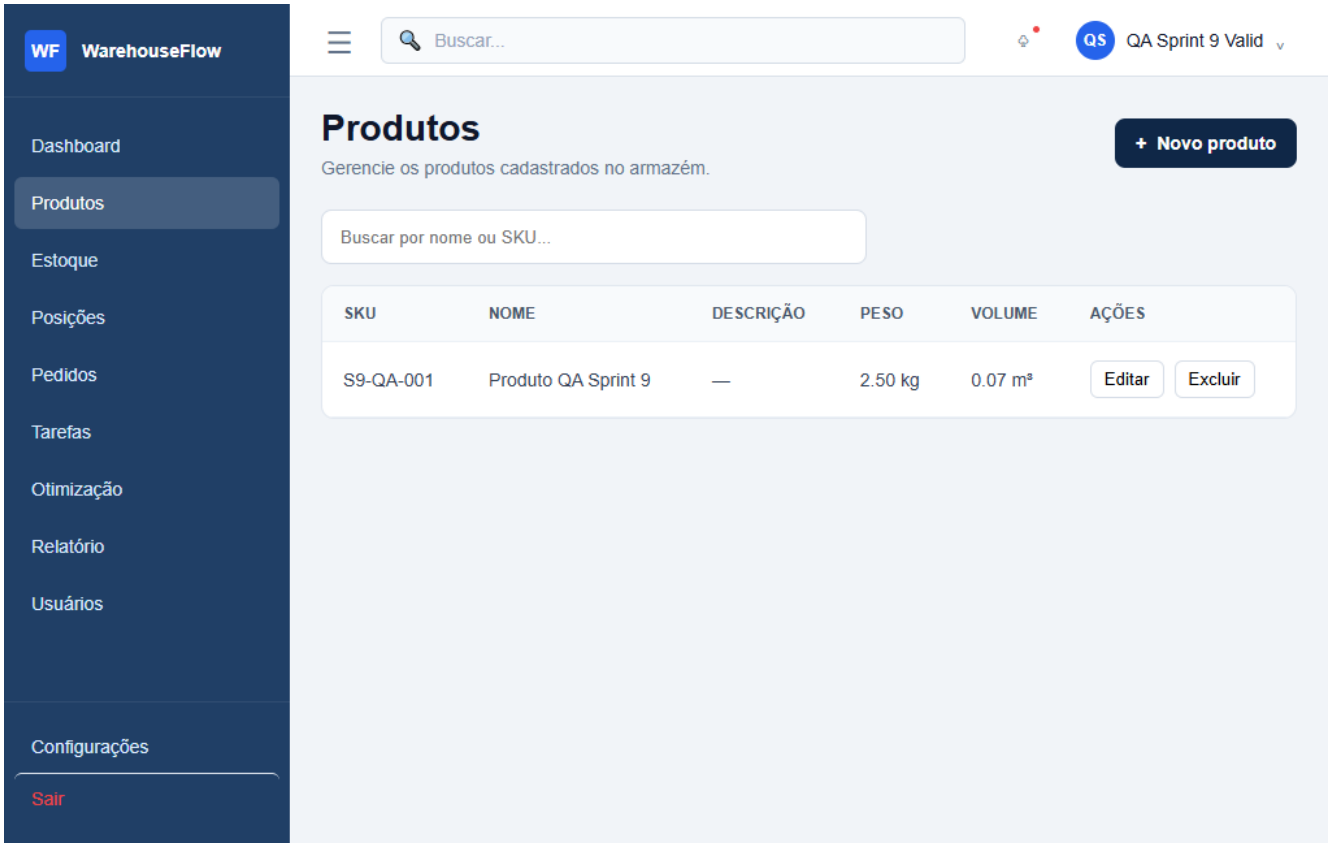
S9-T05-produto-campos-obrigatorios.png



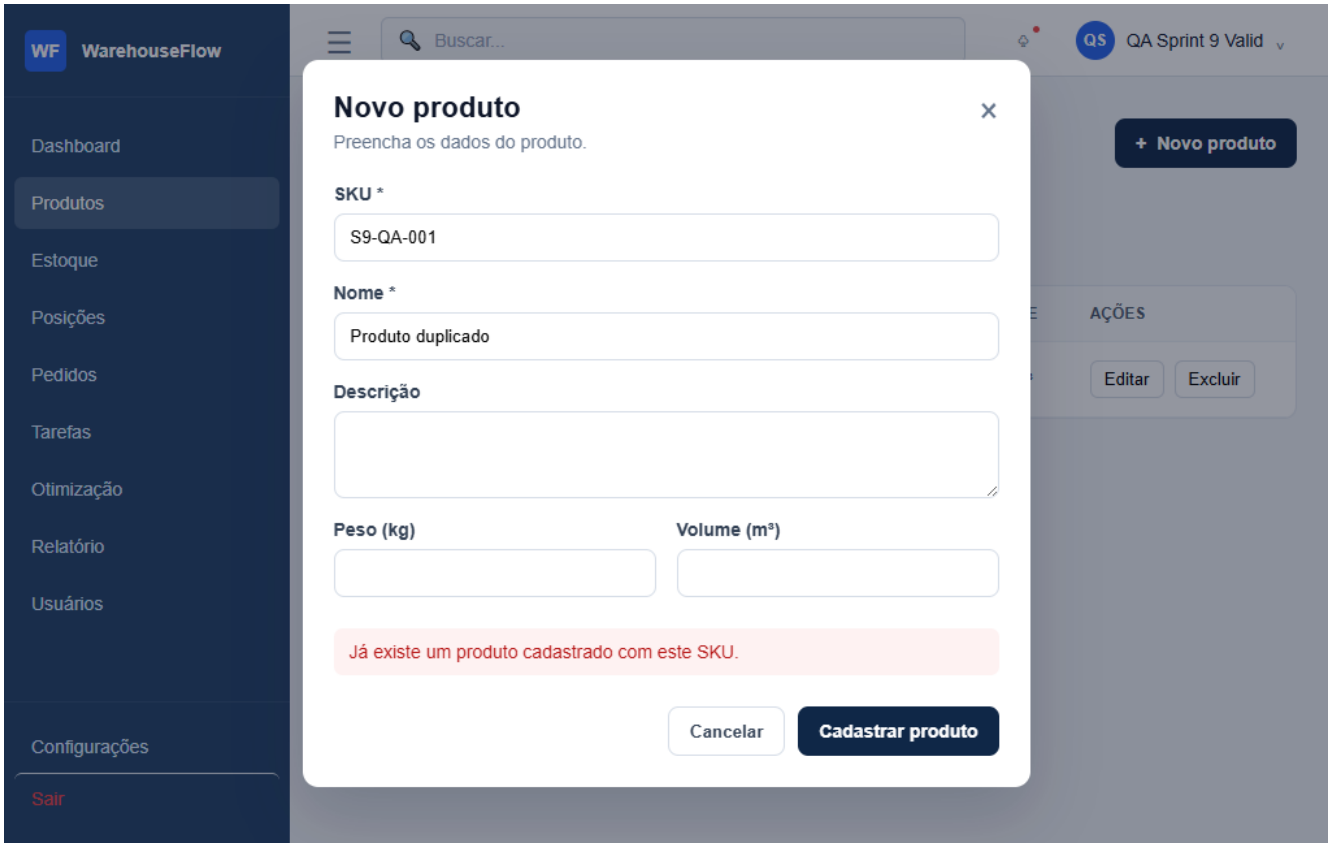
S9-T06-peso-negativo.png



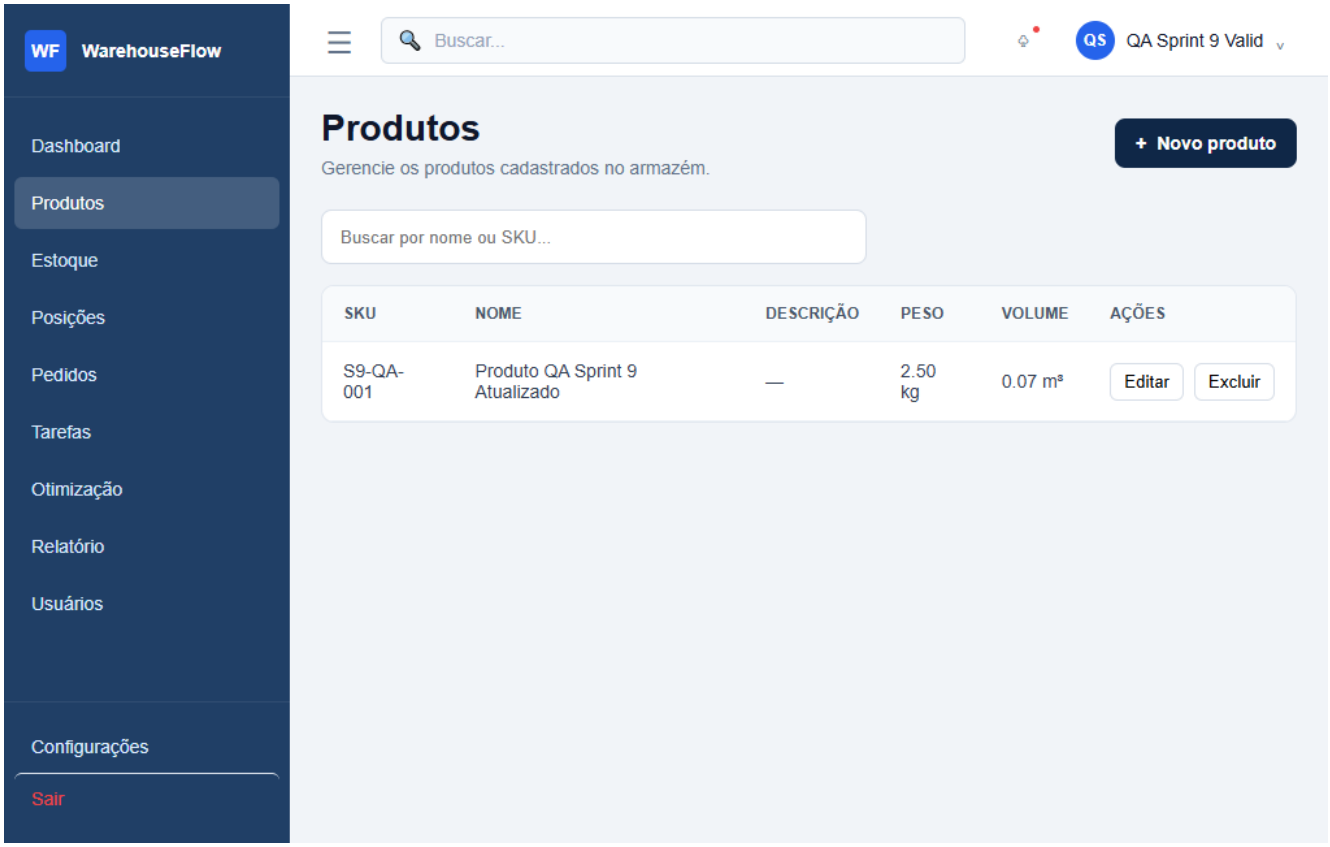
S9-T07-produto-criado.png



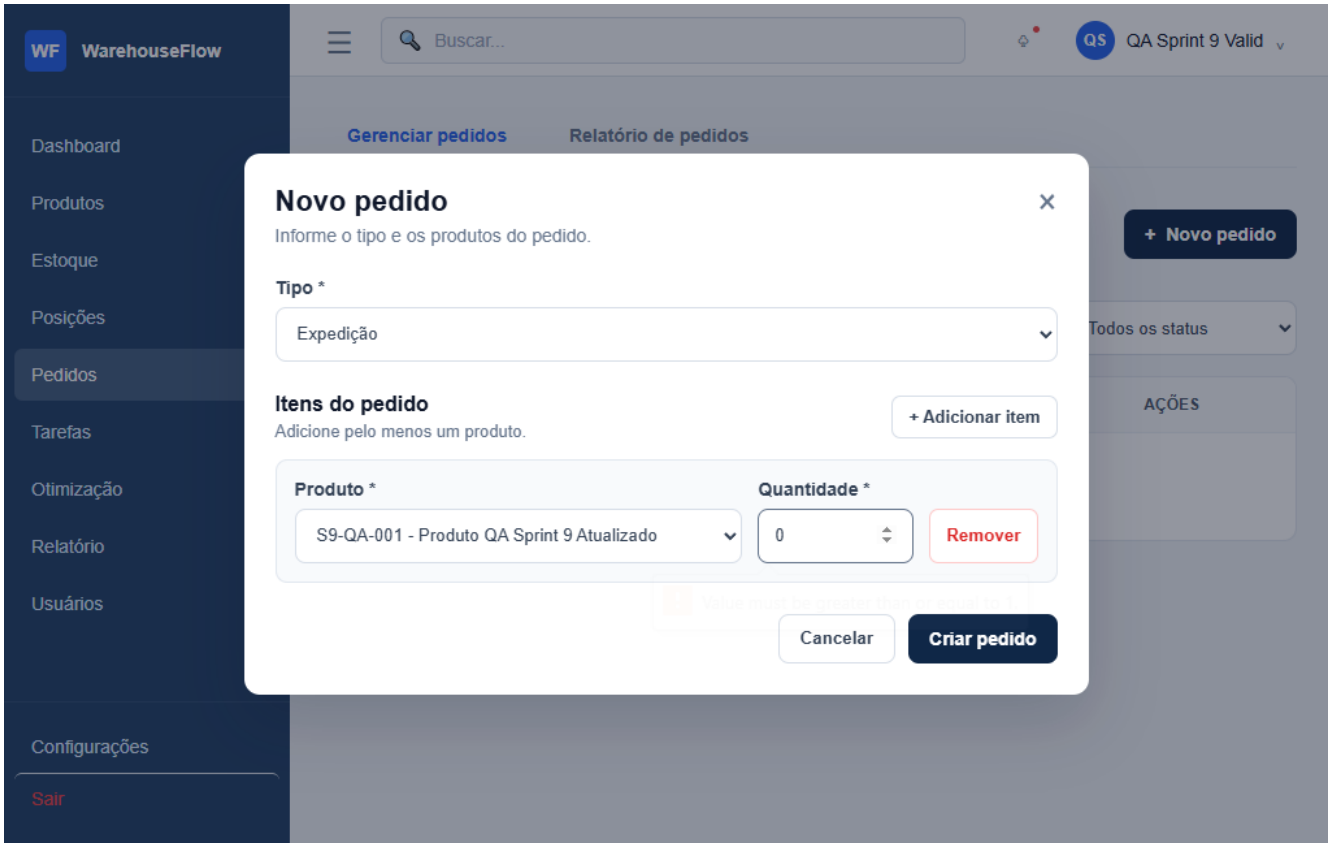
S9-T08-sku-duplicado.png



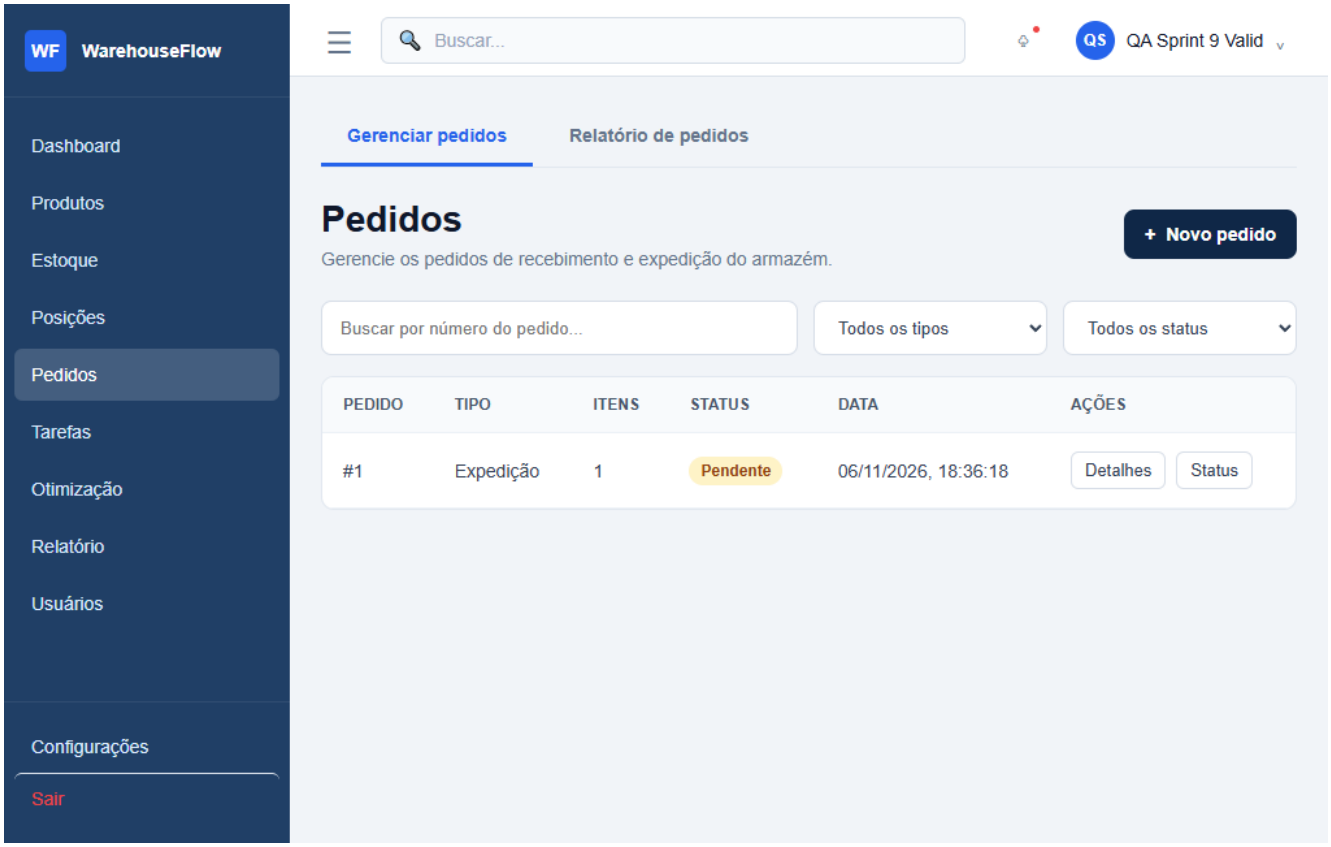
S9-T09-produto-atualizado-persistido.png



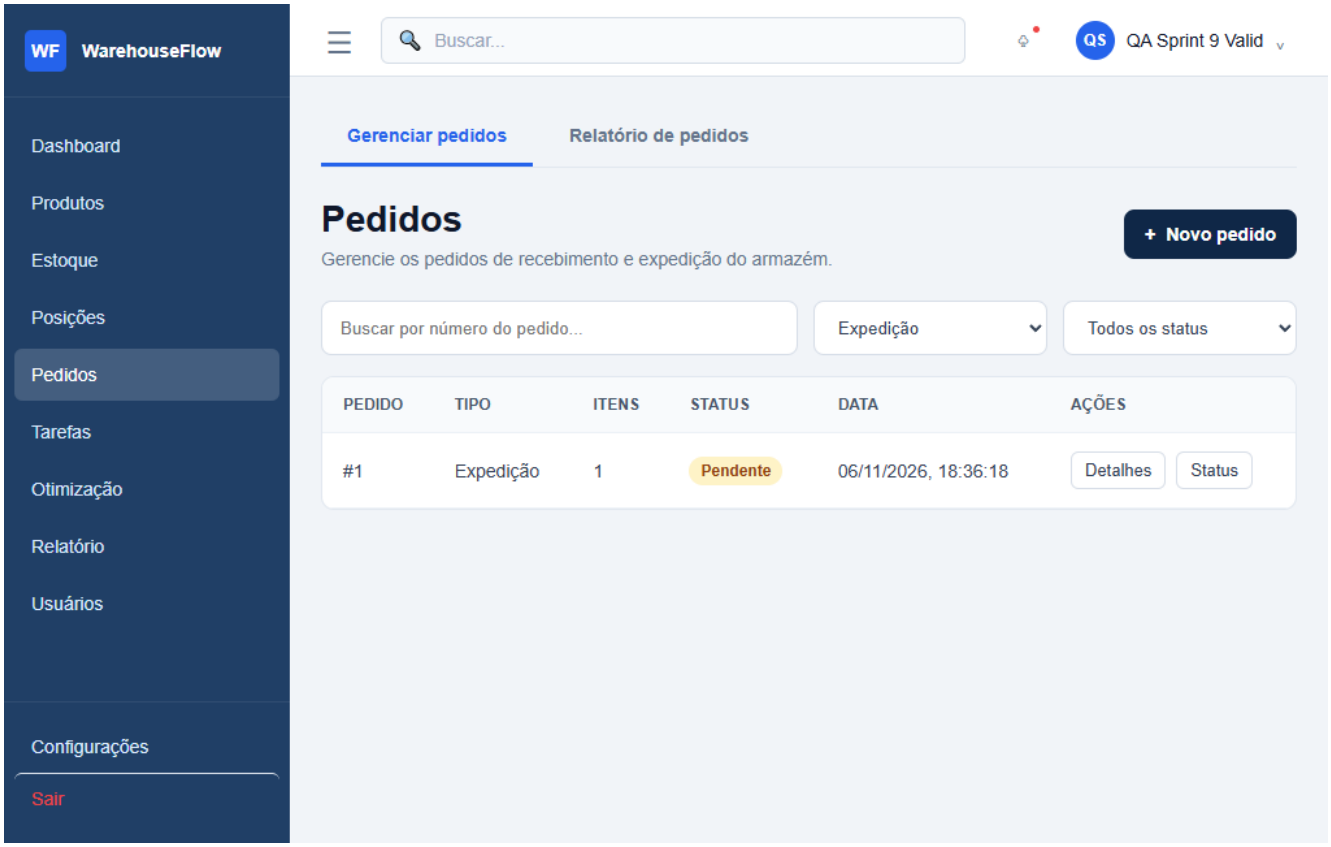
S9-T10-quantidade-zero-pedido.png



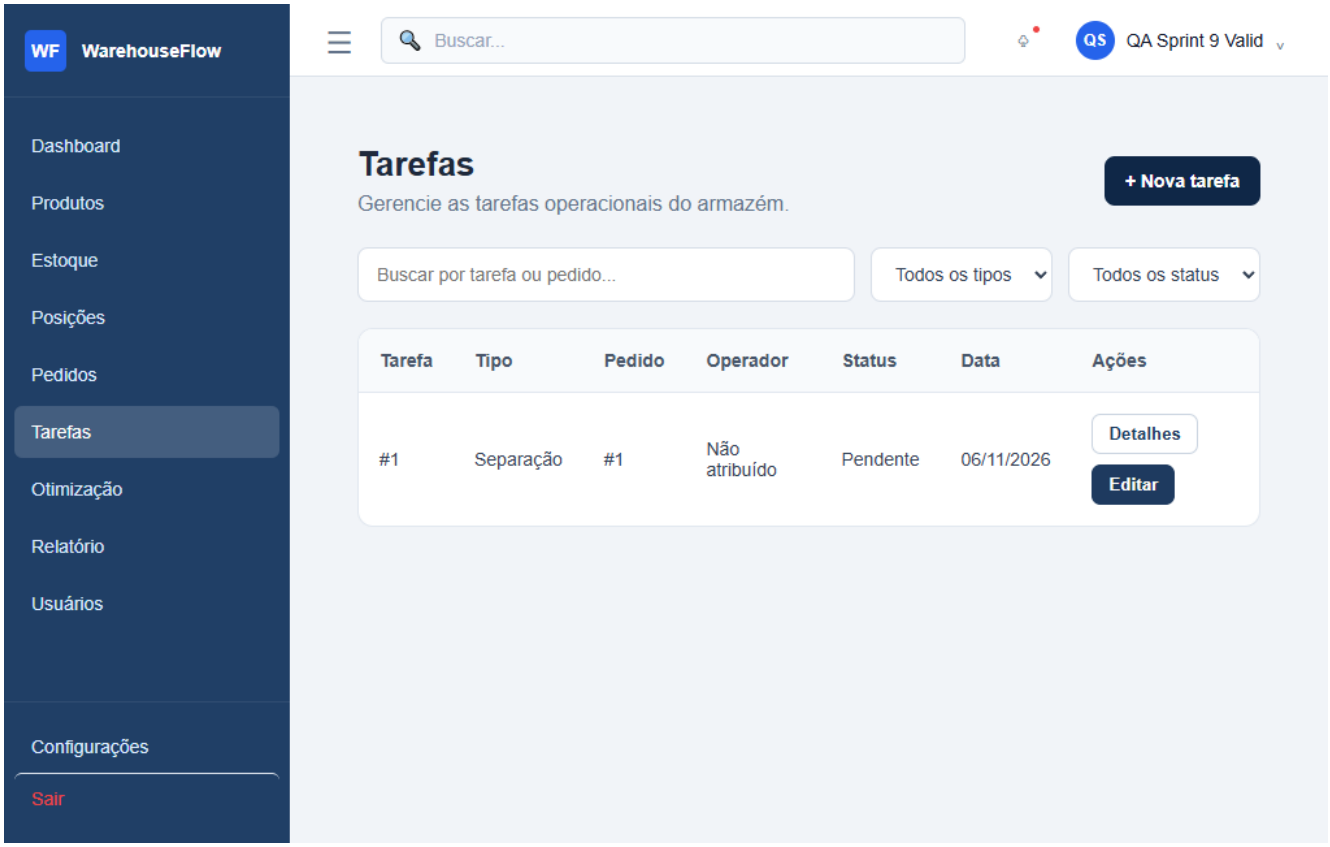
S9-T11-pedido-outbound-criado.png



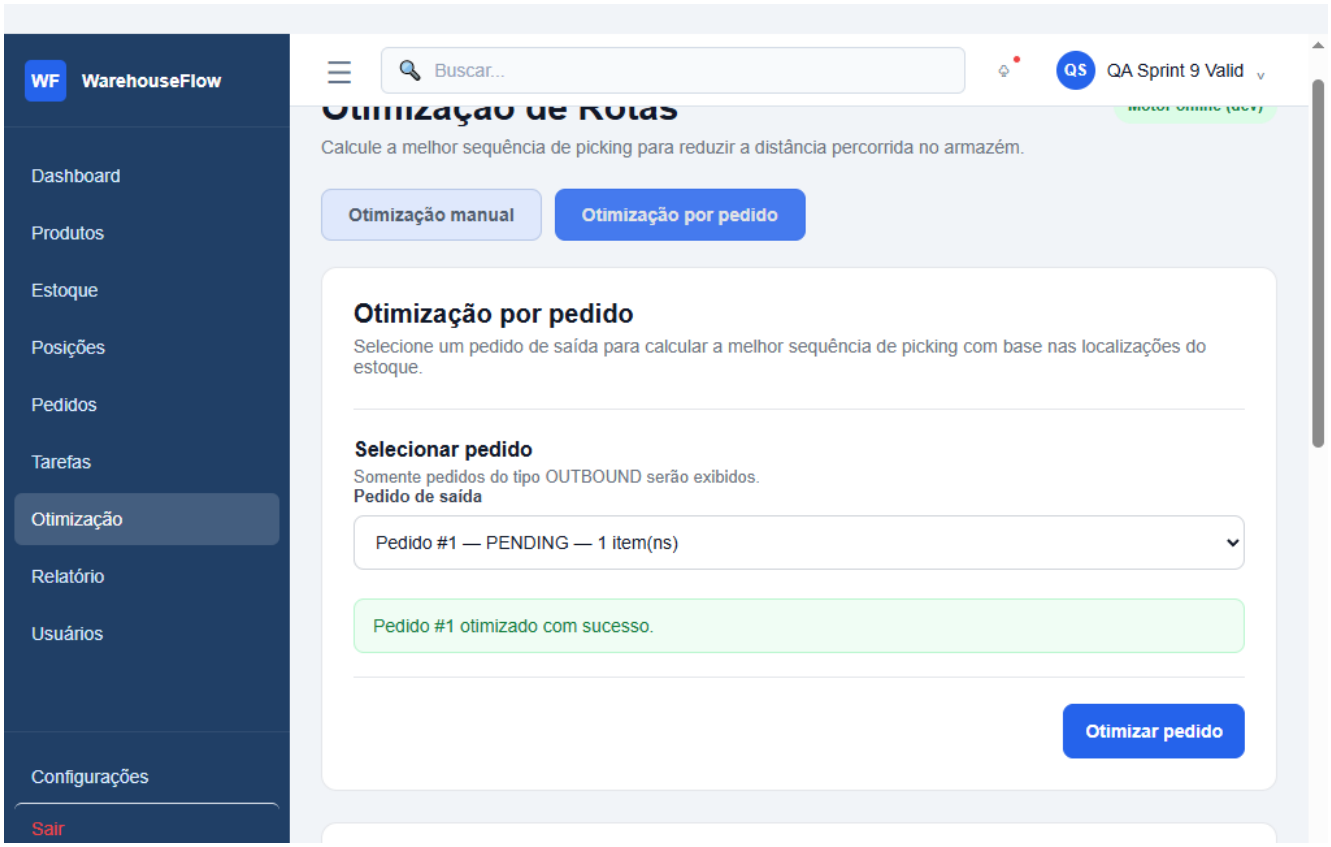
S9-T12-filtro-pedidos-expedicao.png



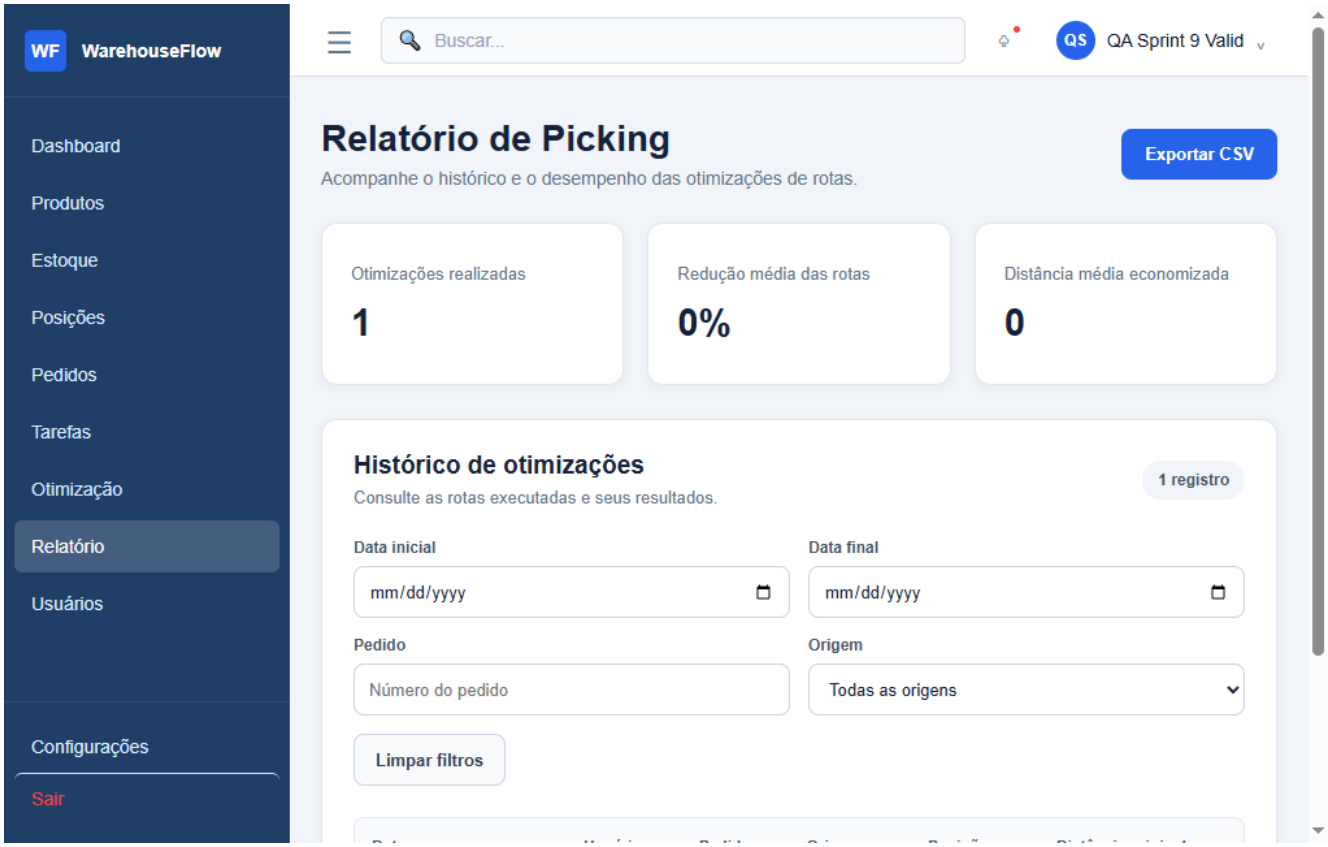
S9-T13-tarefa-picking-criada.png



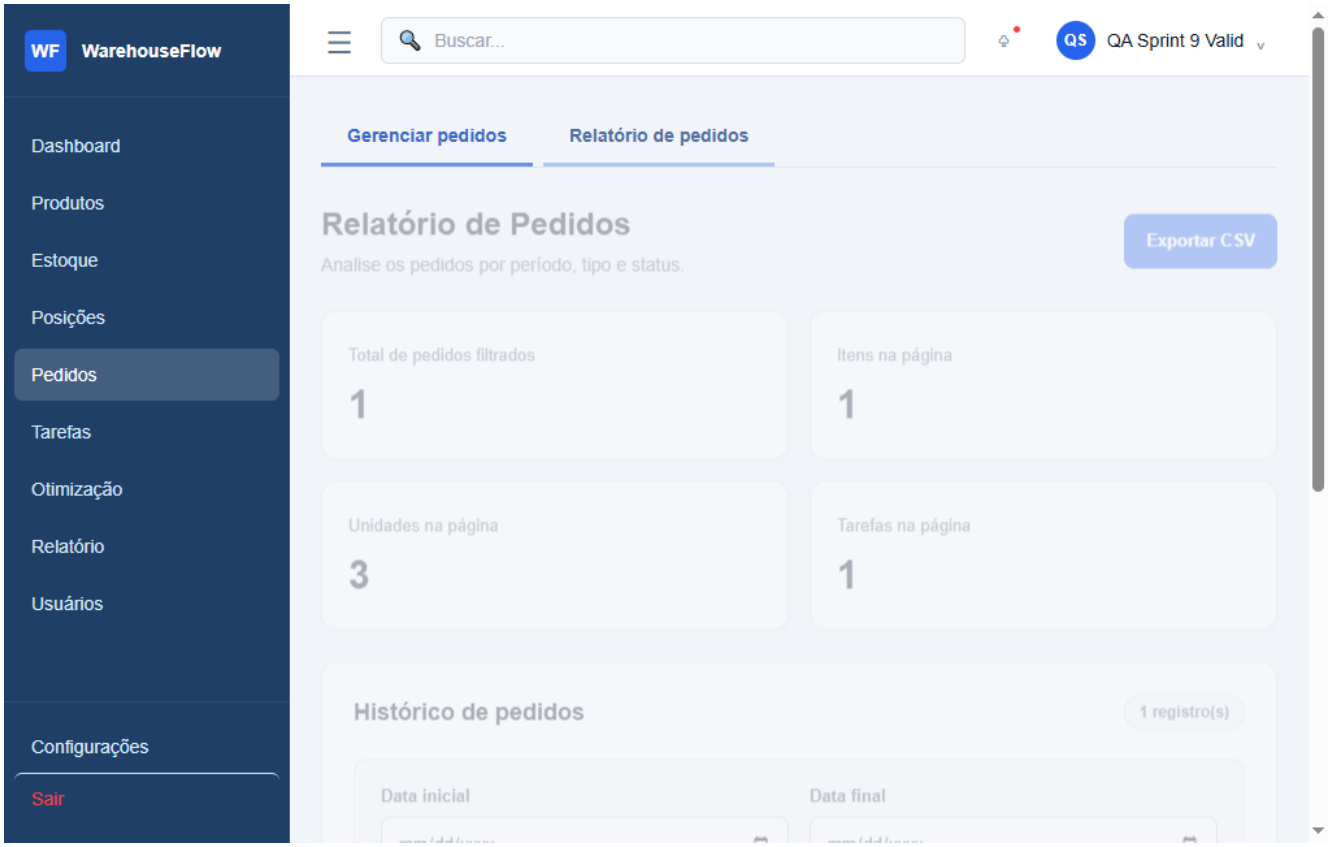
S9-T14-otimizacao-por-pedido.png



S9-T15-relatorio-historico-picking.png



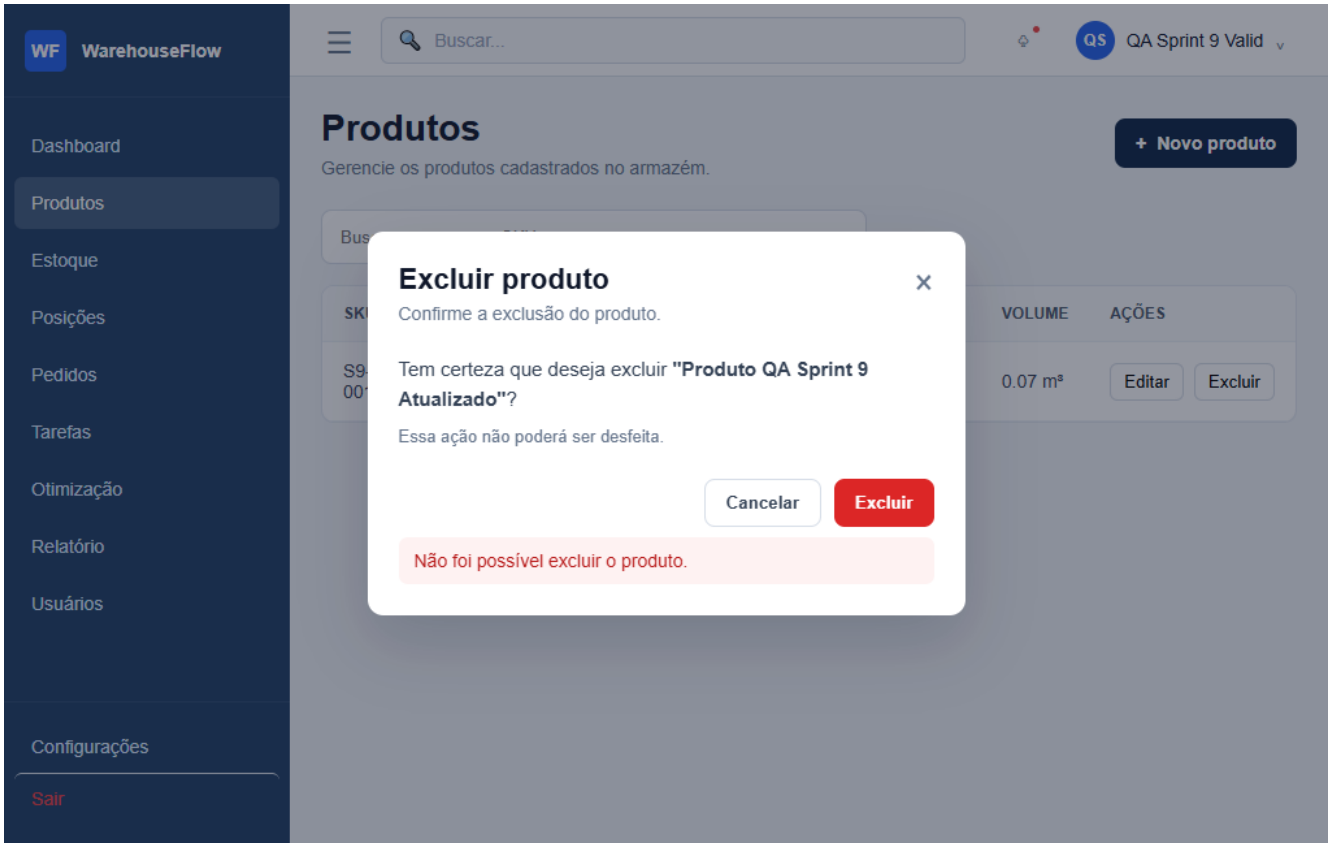
S9-T16-navegacao-relatorio-pedidos.png



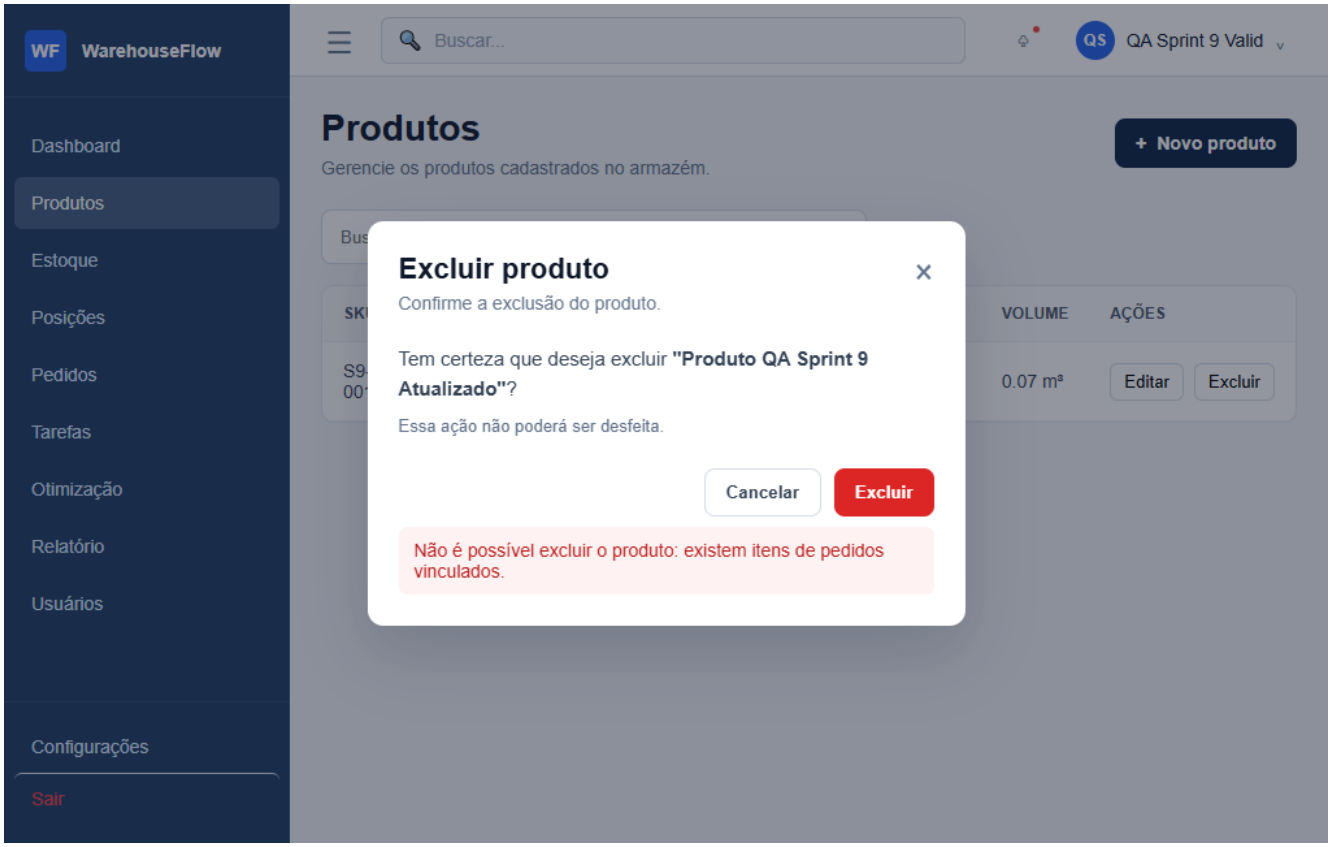
S9-T17-logout-e-redirecionamento.png



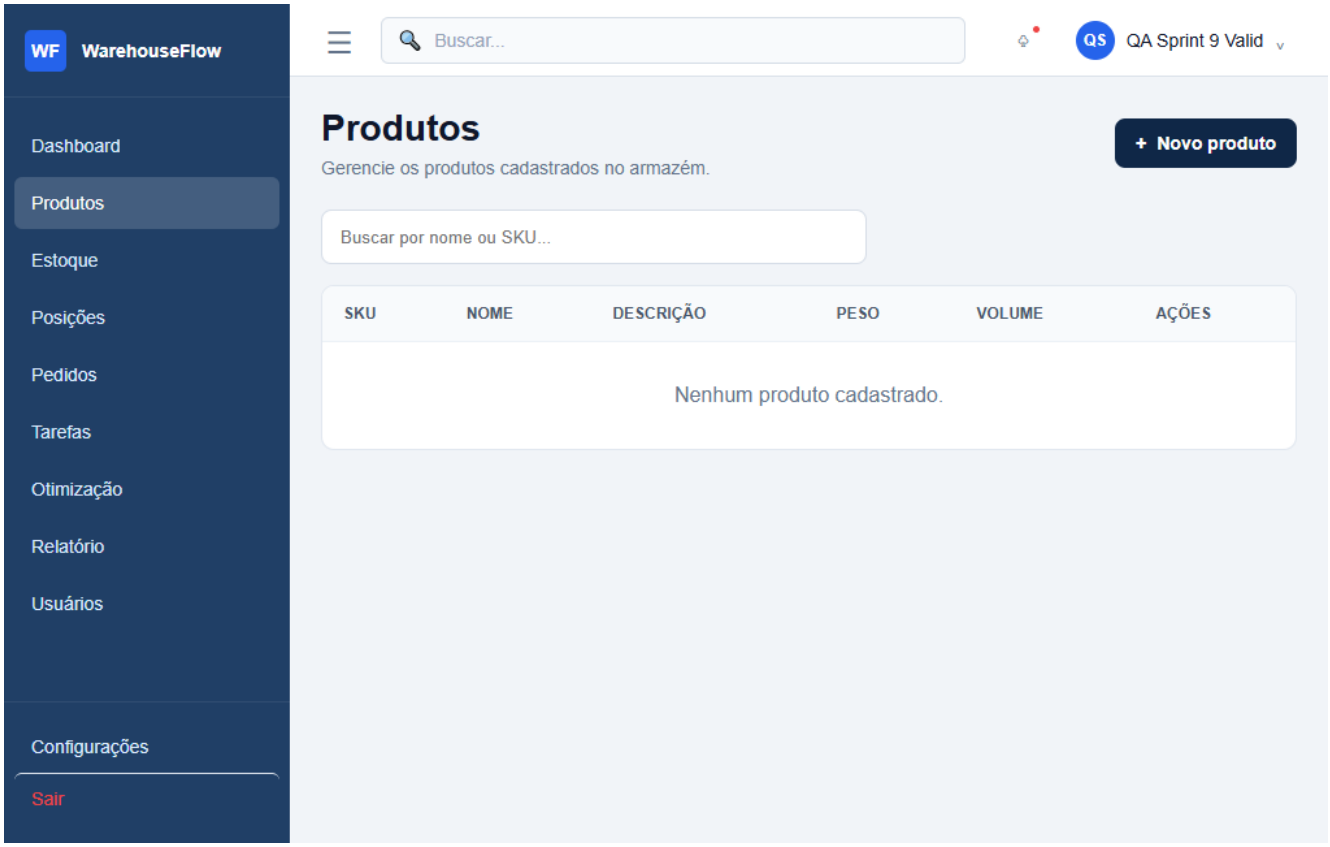
S9-T18-exclusao-com-dependencias-falha.png



S9-T18-exclusao-dependencia-409.png



S9-T19-produto-exclusao-sem-vinculos.png



Sprint 10 — WarehouseFlow | Relatório da Release Candidate

Grupo: 26 · Turma: 8MB · Data da validação: 13/11/2026 Repositório: thiagodemorais87/WarehouseFlow Escopo: documentação, testes e preparação da RC; execução da aplicação e verificação dos contratos existentes. Este relatório distingue validações realizadas de itens bloqueados.

PDF acumulado: Grupo26_WarehouseFlow_Sprint10_RC.pdf — capa RC atualizada, 101 páginas do acumulado Sprints 1–9 preservadas e relatório/evidências da Sprint 10. O apêndice usa fonte Unicode para exibir corretamente os caracteres da documentação em português.

Estado da RC: a API e a interface Jinja foram iniciadas e os testes
executados em uma base SQLite descartável. O Compose foi validado, mas não
iniciou porque o daemon Docker não estava disponível. Portanto, esta entrega
não certifica PostgreSQL, Nginx ou persistência em containers.

A. Descrição da RC

O que entra nesta versão

- Autenticação JWT, cookie de sessão e autorização por ADMIN, GESTOR e OPERADOR.
- API e telas Jinja para produtos, estoque, posições, pedidos e tarefas.
- Dashboard; relatórios de pedidos, movimentações de estoque e picking; CSV nos relatórios implementados.
- Otimização de picking com Nearest Neighbor + 2-opt, distância Manhattan, rota por posições ou por pedido OUTBOUND, métricas e histórico.
- Endpoints de usuários/cargos e regras de acesso. **A administração de usuários não tem tela no menu atual.**
- Contrato de API atualizado em Api-doc.md, README e este conjunto de evidências.

Alterações do repositório após o relatório da Sprint 09

O histórico entre de38073 (documentação de execução da Sprint 09) e o estado analisado inclui commits de 07–10/11/2026 com validação de e-mail e permissões, testes funcionais de CRUD, limites e checks no schema, tratamento de conflitos de integridade, proteção/idempotência do seed, healthcheck do PostgreSQL no Compose e ajustes nas telas de autenticação, estoque, posições, pedidos e relatórios. O log não classifica individualmente esses commits como Sprint 10; eles são registrados aqui como evolução posterior à documentação da Sprint 09, sem reclassificar a autoria ou o escopo dos commits.

Na validação desta entrega foi corrigida a importação ausente de EmailStr, que impedia a coleta dos testes e a importação da aplicação. Também foi alinhado o OpenAPI ao handler de validação da aplicação (400, não 422).

Referência de repositório observada: branch main, tag Git 2.0.0. APP_VERSION respondeu dev na execução isolada; isso não cria nem certifica uma release publicada.

B. Evidências de estabilidade

Componente / verificação	Resultado observado
docker compose config --quiet	Passou (código 0).
docker compose up --build -d	Não iniciado: falha ao conectar ao pipe do Docker Desktop Linux Engine.
Containers, PostgreSQL, migrações e seed do Compose	Não executados; não há status de container nem log de container para relatar.
API local	Uvicorn iniciado em 127.0.0.1:8000, com FRONTEND_DIR do repositório.
Banco da execução	SQLite descartável em %TEMP%, não persistente; não representa validação PostgreSQL.
Health check	GET /health → HTTP 200, {"status":"ok","version":"dev"}.
Pytest	101 passed, 2 skipped, 0 failed, 1 aviso de depreciação; detalhes abaixo.
Smoke tests da API	31 passed, 0 failed, incluindo login, fluxo de pedido, picking, relatórios, RBAC, otimização, validação e logout.
Navegação no navegador	Login/logout e /dashboard, /pedidos, /tarefas, /otimizacao, /relatorios/picking responderam/carregaram com HTTP 200.
Swagger/OpenAPI	/docs e /openapi.json responderam HTTP 200; 39 caminhos OpenAPI, com validação documentada como 400.

O serviço frontend em Compose (nginx:alpine, porta 8080) não foi iniciado. As telas capturadas foram servidas pelo backend local em :8000; portanto não comprovam o proxy Nginx.

Pytest

Resultado final salvo em S10-pytest-results.txt:

- 101 aprovados
- 2 ignorados, sem falha:
- teste de tela de usuários: o link/página ainda não existe;
- geração opt-in da matriz: exige GERAR_MATRIZ=1 e atualiza

docs/matriz-permissoes.md; não foi executada para preservar o documento.

- 1 aviso: depreciação do uso de httpx com starlette.testclient.

Smoke tests e health check

Os 31 resultados individuais estão em S10-smoke-results.json. O fluxo verificou, entre outros:

1. /health e Swagger;
2. login JWT e /auth/me;
3. dashboard;
4. criação/listagem de pedido OUTBOUND e tarefa PICKING;
5. relatórios JSON e CSV;
6. listagem de usuários/cargos e restrição de escrita por OPERADOR;
7. otimização explícita e por pedido e histórico;
8. entradas inválidas (400), ausência de token (401) e logout (303).

As telas de usuário não foram marcadas como aprovadas: o backend expõe a API, mas não existe página de administração no menu. O smoke de tarefas confirmou as rotas e permissões pela API; não substitui uma validação de fluxo manual em PostgreSQL.

Evidências pontuais:

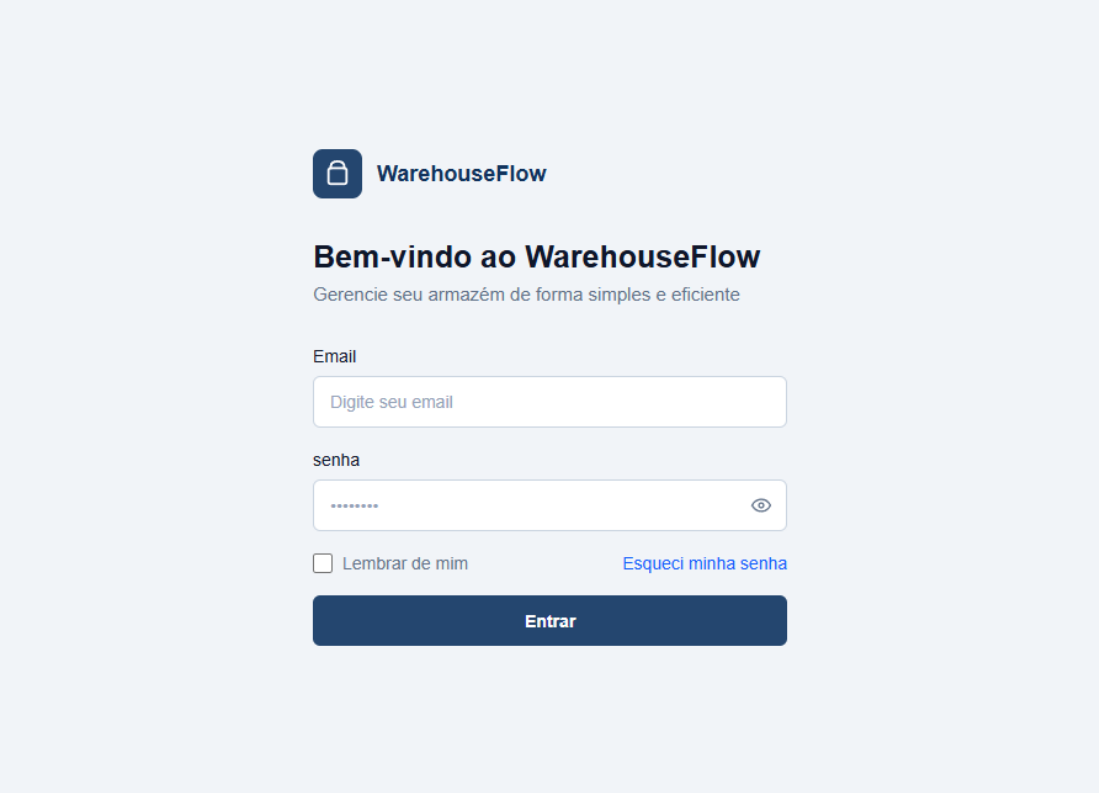
- Health check HTTP
- Log da API local
- Validação do Compose e bloqueio do daemon
- OpenAPI gerado
- Caminhos OpenAPI
- Respostas de validação documentadas no OpenAPI

C. Interface final

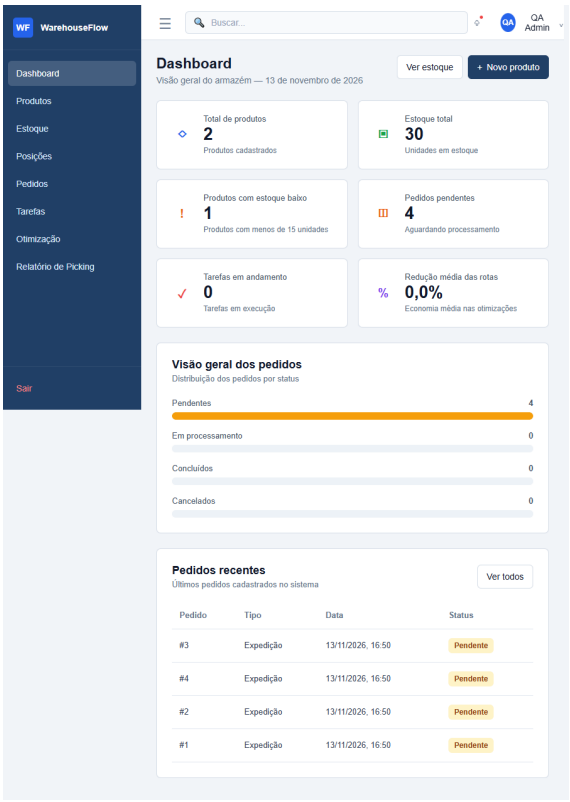
As capturas abaixo são screenshots reais do navegador integrado, obtidas da aplicação local com registros QA temporários na base descartável. Não representam dados de produção nem containers Compose.

Login e dashboard

Tela de login

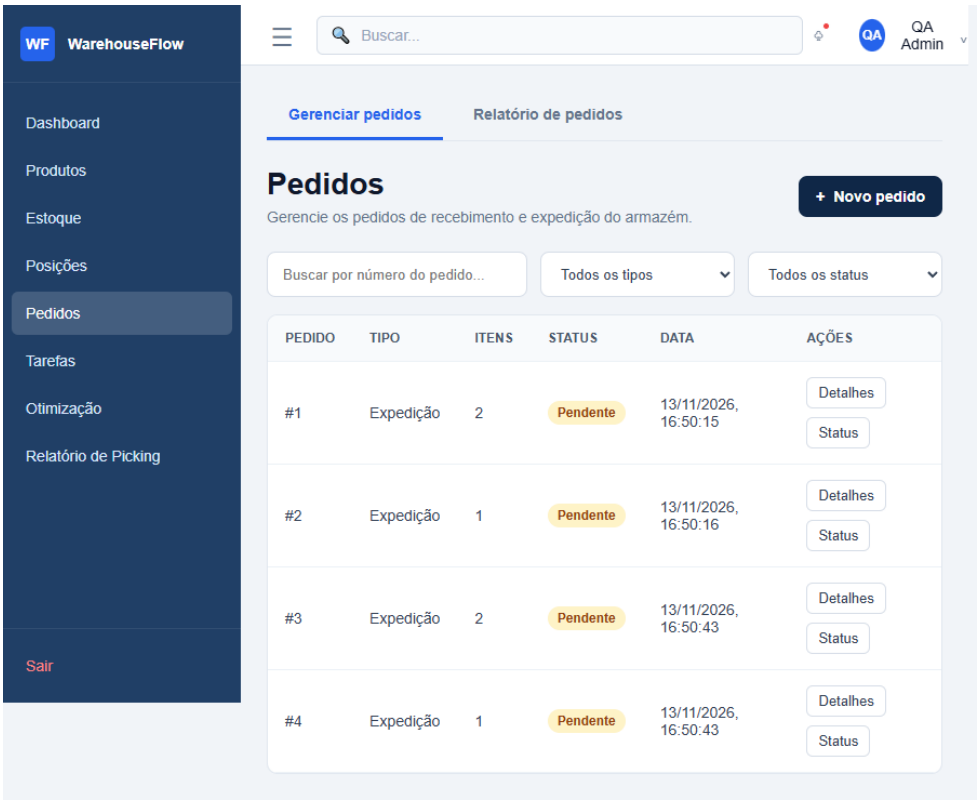


Dashboard do WarehouseFlow

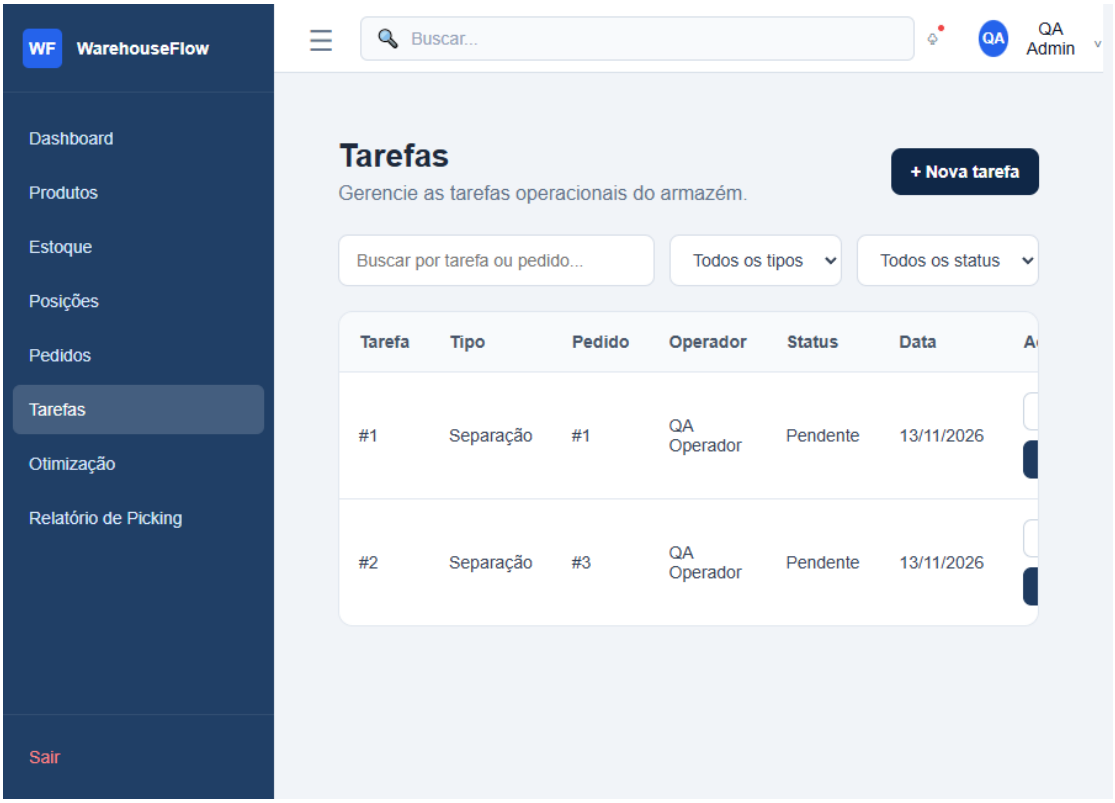


Navegação e fluxos

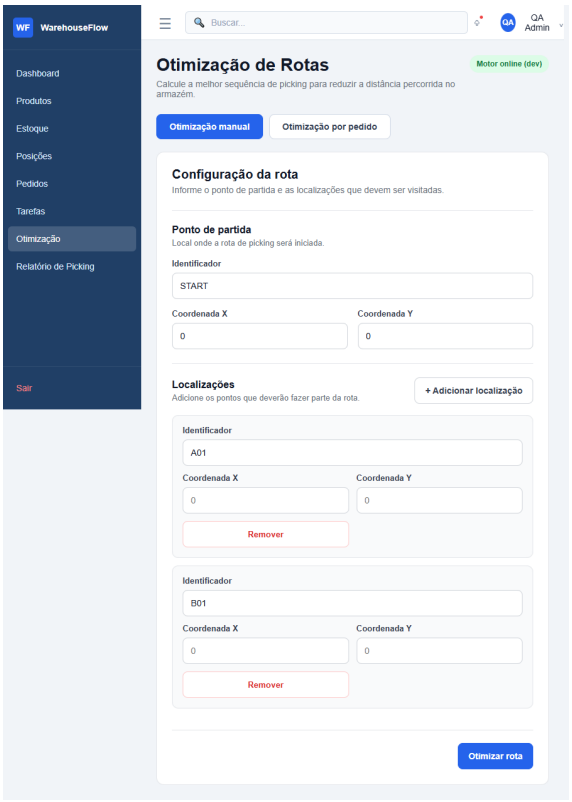
Tela de pedidos



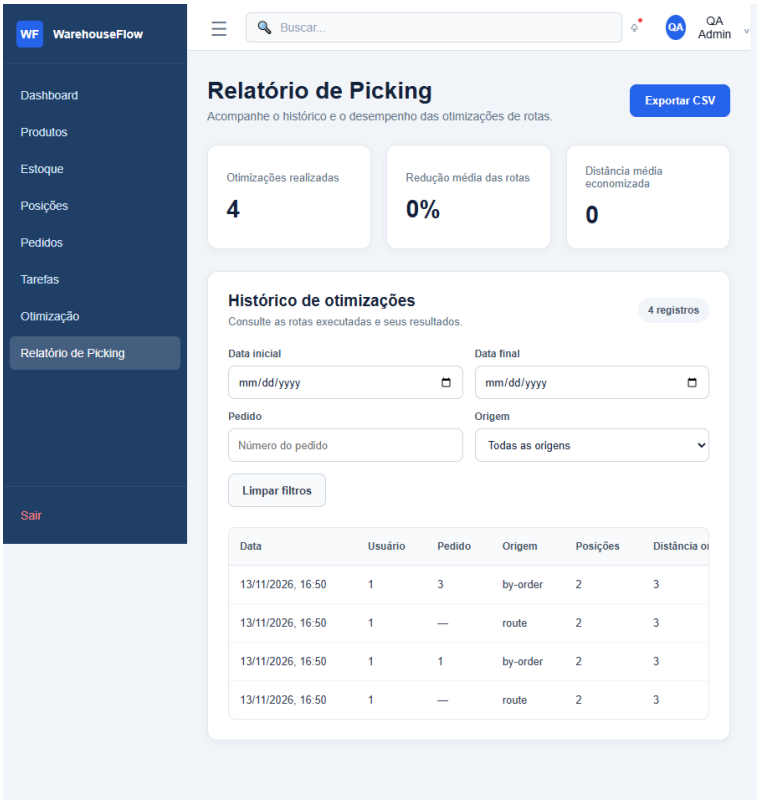
Tela de tarefas



Tela de otimização

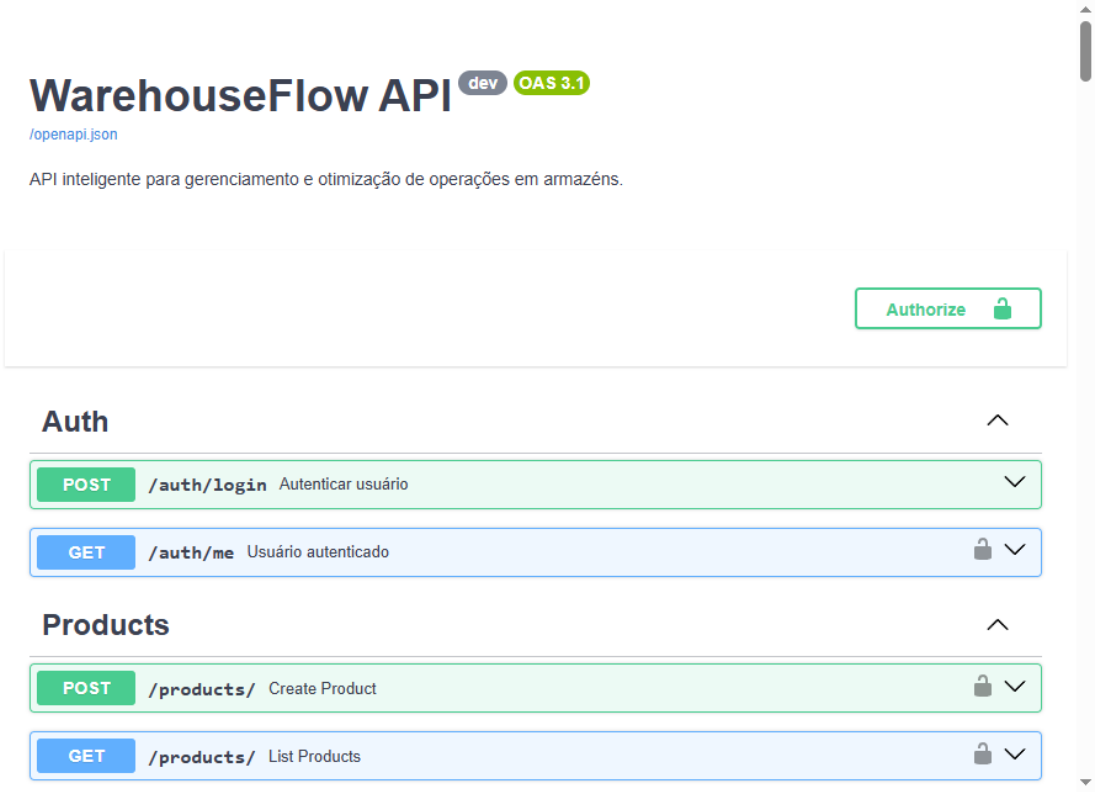


Relatório de picking



Swagger

Swagger UI executado em /docs



D. Banco de dados

Estrutura e MER

O DDL backend/database/schema.sql contém as tabelas roles, users, products, warehouses, locations, stock, orders, order_items, tasks e optimization_results. O modelo ORM backend/app/models/optimization_run.py acrescenta optimization_runs, criada por Base.metadata.create_all quando esse modelo está carregado.

A base SQLite temporária de smoke test apresentou 11 tabelas. Sua introspecção real, sem dados de linha ou credenciais, está em S10-database-schema.txt. Ela não prova que o mesmo schema foi aplicado ao PostgreSQL.

Relacionamentos principais: role→users; warehouse→locations; product/location→stock; order→order_items/tasks; product→order_items; user→tasks atribuídas; task→optimization_results; location→resultado de otimização sugerido. stock restringe duplicidade do par produto/posição; quantidade de estoque é não negativa; itens de pedido exigem quantidade positiva. optimization_runs.user_id e order_id são valores de histórico sem FK declarada no modelo.

MER atualizado: mer-final.md. A confirmação de migrações, seed e estrutura em PostgreSQL depende de executar o Compose em ambiente com daemon disponível.

E. APIs

- Swagger UI: http://127.0.0.1:8000/docs
- OpenAPI: http://127.0.0.1:8000/openapi.json
- Referência textual: docs/Api-doc.md
- Evidência gerada: lista de caminhos e contratos

Foram documentados autenticação (/auth/*), pedidos, tarefas, usuários/cargos, relatórios e /optimization/*, incluindo métodos, roles, entradas, exemplos, respostas e erros verificados no código. A correção do OpenAPI substitui o 422 automático pelo 400 real do handler de validação e documenta o formato detail + errors; há teste de regressão em tests/test_openapi.py.

O snapshot da execução contém 39 paths no schema gerado; o Swagger exibiu 65 operações. Outras rotas (/products, /stock, /locations, /optimization-results) continuam disponíveis no Swagger e são apontadas como fora do recorte textual pedido.

F. Repositório, README e execução

- GitHub: thiagodemorais87/WarehouseFlow
- README: README.md, atualizado com funcionalidades,

arquitetura, tecnologias, requisitos, configuração segura de .env, Compose, testes, URLs, Swagger, documentação de API e Sprint 10.

- Tecnologias observadas: Python, FastAPI, SQLAlchemy, Pydantic,

PostgreSQL, HTML/CSS/Jinja2/JavaScript, Nginx, Docker Compose e Pytest.

- URLs Docker documentadas: UI :8080, API/Swagger :8000, PostgreSQL

host :5433.

Os comandos de instalação e inicialização estão no README. .env não foi aberto nem sobrescrito nesta execução. O acesso efetivamente verificado foi http://127.0.0.1:8000; :8080 e :5433 ficaram sem validação porque o daemon Docker não iniciou.

G. Correções da Sprint 10 e regressão

BUG-S10-01 — import ausente de EmailStr

- Sintoma: antes da alteração, pytest interrompeu a coleta de sete

módulos com NameError: name 'EmailStr' is not defined ao importar backend/app/schemas/__init__.py.

- Correção: importação de EmailStr adicionada ao módulo Pydantic.
- Regressão: depois da correção, a suíte concluiu com 101 aprovados e

nenhum erro de coleta/falha.

DOC-S10-01 — contrato de validação divergente no Swagger

- Sintoma: o handler devolvia HTTP 400, mas a resposta automática

OpenAPI anunciava 422.

- Correção: schema OpenAPI substitui a resposta de validação por 400 e

descreve o JSON {detail, errors}; resposta do endpoint e OpenAPI foram conferidos e há teste em tests/test_openapi.py.

Não houve outra correção de bug de produto confirmada nesta execução. As correções da Sprint 09 permanecem registradas no relatório e evidências anteriores; não foram reescritas.

H. Pendências, riscos e próximos passos

1. Iniciar Docker Desktop e executar docker compose up --build; verificar

estado/logs de API, frontend/Nginx e PostgreSQL, /health através da API, seed, migrações e persistência após reinicialização.

2. Confirmar se optimization_runs é criado/aplicado corretamente em uma

instância PostgreSQL limpa e já existente.

3. Implementar/validar manualmente a interface de administração de usuários,

caso continue no escopo. A API já existe; a tela não.

4. Executar a geração opt-in de matriz (GERAR_MATRIZ=1) em ambiente

controlado, se a equipe optar por atualizar docs/matriz-permissoes.md.

5. Sprint 11 (21/11): manual do usuário, manual técnico, revisão completa

de código e organização final do GitHub.

6. Sprint 12 (28/11): correções finais, code freeze/tag/release, vídeos e

ensaio da apresentação; entrega final permanece em 05/12 conforme o cronograma.

Risco conhecido: a RC ainda não tem evidência de operação integrada em PostgreSQL + Nginx/Compose neste ambiente. Os testes SQLite não substituem essa validação.

I. Integrantes

Integrante	Matrícula	Papel registrado
Gabriel George de Araújo Figueiredo	01605236	O papel individual não está nominalmente identificado nas fontes consultadas.
Guilherme Branco Ferrario	01596391	Documentação e Testes nesta entrega, conforme designação da Sprint 10; documentos anteriores nomeiam a função como “Pessoa 5”, sem associar essa numeração a um integrante.
João Pedro Silva de Araujo	01606470	O papel individual não está nominalmente identificado nas fontes consultadas.
Sérgio José de Araújo Júnior	01590694	O papel individual não está nominalmente identificado nas fontes consultadas.
Thiago de Morais Gonçalves	01609695	O papel individual não está nominalmente identificado nas fontes consultadas.

Os cinco nomes, matrículas, Grupo 26 e turma 8MB foram conferidos no README e no PDF acumulado anterior. As responsabilidades antigas são descritas por número de pessoa, não por nome; os papéis não mapeados permanecem explicitamente sem atribuição para evitar inventar informações.